

ÇOCUKLAR İÇİN PYTHON

Programlamaya kolay yoldan,
güçlü bir şekilde başla!



BÜLENT ÇOBANOĞLU

- ▲ Bol Görselli Basit Anlatım Tekniği
- ▲ Temel Programcılık Kavramları ve Python Dilinde Kullanım Şekilleri
- ▲ Eğlenceli, Oyun Tabanlı Uygulama Projeleri

PUSULA▲

Çocuklar İçin Python



Bülent ÇOBANOĞLU

PUSULA

PUSULA 20 TEKNOLOJİ VE YAYINCILIK A.Ş.

Pusula 402-130

Çocuklar İçin Python

ISBN: 978-605-2359-94-5

Yayıncılık sertifika no: 29644

Bülent Çobanoğlu

1. Baskı: Haziran 2021

Yayın Yönetmeni : Mehmet Arslantunalı

Editör : Derya Karakoyun

Satış : satis@pusula.com

Bu kitabın bütün yayın hakları Pusula 20 Teknoloji ve Yayıncılık A.Ş.'ye aittir. Yayınevimizin yazılı izni olmaksızın kısmen veya tamamen alıntı yapılamaz, kopya edilemez, çoğaltılamaz ve yayınlanamaz.

Pusula 20 Teknoloji ve Yayıncılık A.Ş.

Namık Kemal Mah. Köşk Sk. No:25B Ümraniye / İstanbul

Tel. : (0216) 505 49 45

Faks : (0216) 505 49 43

E-posta : pusula@pusula.com

Web : www.pusula.com www.pusulakitaplik.com

Baskı : Vizyon Basımevi Kağıtçılık Matbaacılık ve Yayıncılık San. Tic. LTD. ŞTİ.
Beylikdüzü O.S.B. Mah. Orkide Cad. No:1/Z Beylikdüzü – İstanbul
Matbaa Sertifika No: 28640

Dr. Bülent ÇOBANOĞLU

İlk ve orta öğrenimini Erzincan'da tamamladı. Sırası ile; Ankara Gazi Üniversitesi Elektronik-Bilgisayar Öğretmenliği bölümünden lisans, İstanbul Bahçeşehir Üniversitesi Bilgisayar Mühendisliğinden yüksek lisans, Sakarya Üniversitesi Elektronik-Bilgisayar Eğitimi bölümünden doktora dereceleri aldı.

Alanında 10'dan fazla kitap, 30'a yakın bilimsel makale yayınlayan yazarımız, ülkemizin çeşitli üniversite ve eğitim kurumlarında programlama alanında dersler vermektedir. Evli ve dört çocuk babasıdır.

Önsöz

Python dili ile diğer diller arasındaki fark; otomatik vitesli araba ile manuel araba arasındaki fark gibidir.

Sevgili okuyucu, su bile kolay yolu seçer. Siz de programlamaya kolay yoldan güçlü bir şekilde başlamak istiyorsanız Python dili en ideal seçim olacaktır.

Elinizdeki kitapta bu öğrenme sürecini kolaylaştıracaktır. Kitapta;

- Algoritma temelli basitten zora giden projeler vardır.
- Bol görselli basit anlatım tekniği tercih edilmiştir.
- Önce kavramlar anlatılmakta, sonra projelere yer verilmektedir.
- Ağırlıklı olarak eğlenceli, oyun tabanlı projelere yer verilmiştir.
- Bölüm sonlarında, anlatılan konuları pekiştiren sorular, cevapları ve bulmacalar yer almaktadır.
- Her bölümde "sıra sende etkinlikleri" ile kendini test edebilirsin.

Kitaptaki tüm kodlara www.pusula.com adresindeki "dosya merkezi"nden ya da şu adresten erişebilirsiniz; <https://github.com/bcobanoglu/PythonforKids>

Kitapla ilgili her türlü olumlu/olumsuz yorumlarınızı cbanoglubulent@gmail.com adresinden bana yapabilirsiniz. Faydalı olması dileklerle...

Bu eseri tamamlamayı ve sizlere ulaştırmayı nasip eden Rabbime teşekkür ederim.

Dr. Bülent Çobanoğlu

2021 / Sakarya

İÇİNDEKİLER

1) Algoritma ve Akış Şemaları	1
Algoritma Nedir?	2
Algoritma Hazırlamanın Faydaları	4
Algoritmayı İfade Şekilleri	4
Düz Metin İfadeleri	5
Kaba-Kod (Pseudo-Code)	5
Akış Şemaları (Flow Charts)	6
Soru ve Cevaplarla Neler Anladık?	10
BULMACA	12
2) Bilgisayar Kavramı ve Python Dili	13
Bilgisayar ve Bileşenleri	14
Donanım (Hardware)	14
Yazılım (Software)	15
Program ve Programlama Dilleri	16
Makine Dili	16
Derleyici (Compiler) ve Yorumlayıcı (Interpreter)	16
Kaynak Program (Source Program) ve Python Editörleri	18
Çalıştırılabilir Dosya (Executable File)	18
Python Dili	19
Python Kurulumu	19
Python Programlarını Saklamak	21
Soru ve Cevaplarla Neler Anladık?	22
BULMACA	24
3) Değişkenler ve Hesaplar	25
Değişken Nedir? Nasıl Değer Aktarılır?	26
Değişkenlere İsim Verme Kuralları	26
Yer değiştirme (İki Değişkenin İçeriğini Birbirine Aktarma) İşlemi	27
Başlangıçta Değişkenlere Değer Aktarmanın Mantığı	28

Operatörler	29
Matematiksel Operatörlerin İşlem Önceliği	30
Değer Atama İşlemlerini Basitleştirme	31
Veri Tipleri	32
Veri Tipi Dönüşümü	33
Açıklama (Yorum) Satırları: #	34
Soru ve Cevaplarla Neler Anladık?	35
BULMACA	36
4) Veri Giriş-Çıkış İşlemleri	37
Giriş	38
Veri girişi komutu: input()	38
Veri çıkışı Komutu: print()	40
print() Komutu Parametreleri	40
format() Kullanımı	41
print() Kullanımında Sık Karşılaşılan Bazı Hatalar ve Çözümü	43
Veri Giriş-Çıkış Uygulamaları	43
Soru ve Cevaplarla Neler Anladık?	46
BULMACA	48
5) Koşul İfadeleri (if/else) ve Karşılaştırma Operatörleri	49
Giriş	50
Karşılaştırma Operatörleri	50
Mantıksal Operatörler	51
bool İfadelerin Sonuçları	52
if Komutu	52
Python Kod Bloğu Yapısı Ve Girinti Mekanizması	54
if - else Komutu	55
if - elif Komutu	57
Karar Ağacı (Bir Yapay Zekâ) Oyunu	60
Soru ve Cevaplarla Neler Anladık?	62
BULMACA	64

6) Döngüler (Loops)	65
Döngü Yapılarının Mantığı	66
Döngü Kullanmanın Ne Faydası Olabilir?	66
for Döngüsü	67
Liste	67
in / not in Operatörleri	67
range Fonksiyonu	68
while Döngüsü	70
while True	73
break Komutu	74
continue Komutu	75
İç İçe Döngüler	76
Soru ve Cevaplarla Neler Anladık?	78
BULMACA	80

7) Fonksiyonlar	81
Fonksiyon Nedir?	82
Fonksiyonlar Niçin Kullanılır?	83
Python Dilinde Fonksiyon Nasıl Tanımlanır?	83
Bir Fonksiyonun Girdileri (Parametreleri)	84
Varsayılan parametre değeri	85
Bir Fonksiyonun Çıktıları (return Komutu)	86
Fonksiyonlarda global ve yerel (lokal) değişken tanımlaması	87
İçeriği Olmayan Fonksiyon Tanımlama	88
Fonksiyon Yazımını Kısaltma: Lambda	88
Özyinelemeli (Rekürsif) Fonksiyonlar	89
Soru ve Cevaplarla Neler Anladık?	92
BULMACA	94

8) String (Karakter Dizisi) İşlemleri	95
Giriş	96
İndis (index) Kavramı	96

İki Stringi Birleştirme	97
Stringi Dilimleme (Parçalara Ayırma)	98
Bir String İfadeyi Güncelleme/Değiştirme	99
Bir Stringin Tamamını Veya Belli Bir Kısımını Silme	99
Bir Stringin Karakterlerine For Döngüsü İle Erişim	100
Bir Stringi Ayırıştırarak Listeye Dönüştürme	100
Bir Stringin Karakter Sayısını Öğrenme	101
İki String İfadeyi Karşılaştırma	101
String İfade İçerisine Değişken Değerini Yazdırma	102
String İfadeyi Ters Çevirme	102
Bir Stringin Harflerini (Büyük/ Küçük Harf) Dönüştürme	102
Bir String İçerisinde Başka Bir Stringi Arama	103
Soru ve Cevaplarla Neler Anladık?	104
BULMACA	106

9) Diziler ve Listeler 107

Dizi Nedir?	108
Diziler Niçin Kullanılır?	108
Listeler	109
Liste elemanlarına erişim	109
Listedeki Bir Elemanın İndis Numarasını Öğrenme	110
Liste Dilimleme	110
Listeye Eleman Ekleme	110
Listenin Eleman Sayısını Öğrenme	111
İki Listeyi Birleştirme	112
Bir Listeyi Ters Çevirme	112
Bir Listenin En Küçük ve En Büyük Elemanını Öğrenme	112
Bir Stringi (Karakter Dizisini) Listeye Dönüştürme	113
Bir Listedeki Eleman Silme Veya Listeyi Tamamen Boşaltma	113
Listelerde Arama	114
Liste Elemanlarını Sıralama	114
enumerate() Fonksiyonu ile Liste Elemanlarını Numaralandırma	115

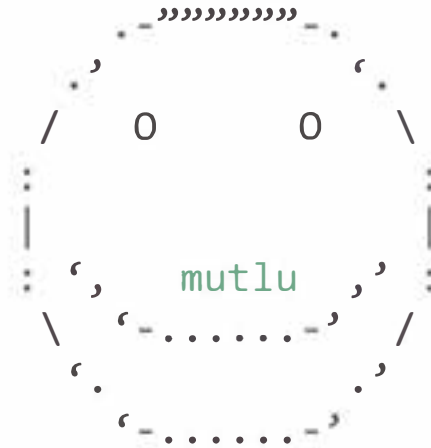
Listelerden Yığın (Stack) Oluşturma	116
Listelerden Kuyruk (Queue) Oluşturma	117
Soru ve Cevaplarla Neler Anladık?	118
BULMACA	120
10) Sözlükler	121
Giriş	122
Sözlük (Dict) Fonksiyonları	123
Küme (set) ve Demet (tuple) Veri Yapıları	127
Soru ve Cevaplarla Neler Anladık?	128
BULMACA	130
11) Nesne ve Sınıf Kavramları	131
Giriş	132
Sınıf (Class) Tanımlama	132
self ve pass Deyimleri	133
Nesne (Object) Tanımlama	133
Nesne Özellikleri:	134
Nesne Metotları:	134
Sınıf Metotlarının Normal Fonksiyonlardan Bir Farkı Var Mıdır?	134
__init__() Metodu:	135
Bir Nesnenin Metot ve Özellikleri Nasıl Öğrenilebilir?	136
Sınıf Yapıları ile Yeni Veri Tipleri Tanımlama	136
Kalıtım	137
Modül Kavramı	138
Farklı import Şekilleri	138
__name__ Özelliği	139
PIP ile Yeni Modüller Ekleme	140
Soru ve Cevaplarla Neler Anladık?	141
BULMACA	142

12) Rastgele Sayı Üretimi ve Matematiksel Oyunlar	143
Matematiksel İşlem Fonksiyonları	144
Rastgele Sayı Üretimi ve Oyun Programlama	148
Soru ve Cevaplarla Neler Anladık?	155
BULMACA	156
13) Turtle ile Çizim Uygulamaları	157
Giriş	158
Grafik Koordinat Sistemi	158
Doğru Çizimi veya Doğrusal Hareket	159
Farklı Turtle Simgeleri	160
Renk Komutları	163
Renk Tablosu	164
Daire Çizimi	166
Çokgen (Poligon) Çizimi	168
Veri Girişi için İletişim (Diyalog) Kutularını Kullanma	169
Çerçeve İçine Resim Yerleştirmek	171
Turtle Metotlarına Toplu Bakış	172
Soru ve Cevaplarla Neler Anladık?	174
BULMACA	176
14) Olay Tabanlı Projeler ve Yılan Oyunu	177
Giriş	178
Sonlu Durum Makinesi (Finite State Machine) Algoritması	182
Adım Adım Yılan Oyunu Tasarlayalım	183
Yılan Oyunu Programı (Tek parça)	192
Projelere Ses Ekleme	193
Soru ve Cevaplarla Neler Anladık?	195
BULMACA	196
SONSÖZ	197
KAYNAKÇA	198

Bölüm

1

Algoritma ve Akış Şemaları



Bazen sayfalarca yazılan bir metin yerine bir karikatür veya şema, istenileni daha güzel anlatabilir.

Algoritma Nedir?

Algoritma, herhangi bir problemi çözmek için adım adım izlenecek yol anlamına gelmektedir. Bir problemi belirli bir zaman diliminde çözebilmek için geliştirilmiş net, çalıştırılabilir, sıralı ve basit işlem adımlarına algoritma denir. Bu tanıma göre bir algoritmanın özellikleri aşağıdaki gibi özetlenebilir;

- Net olmalı, belirsizlik içermemelidir.
- Bilgisayar ortamında çalıştırılabilir olmalıdır.
- Sıralı işlem basamaklarından oluşmalıdır.
- Belirli bir zaman diliminde çözülebilmelidir.

Algoritma yaşamın her alanında faaliyet gösterir. O kendini; ayağınızı bastığınızda harekete geçen yürüyen merdivende veya elinizdeki akıllı telefon üzerinde çalışan herhangi bir uygulamada gösterir.

Evet, iyi yazılımların (uygulamaların) arkasında hep iyi algoritmalar vardır. Ve bu iyi algoritmalar arka planda çalıştıklarından pek görünmezler. Tıpkı Google arama motorunu popüler kılan şeyin, arka planda çalışan "PageRank (Sayfa Değeri)" algoritmasının olması gibi.

TDK (Türk Dil Kurumu) sözlüğünde Algoritma; "Harezmi yolu" olarak tanımlanır. Harezmi; günümüzdeki "Algoritma" kavramına ismini veren kişidir.

Bunları Biliyor musunuz?



Algoritma kelimesi ismini Harezmi adındaki bilim insanından almıştır. Asıl adı Ebu Abdullah Muhammed İbn Musa Al Harezmi'dir. Al-Harezmi ismi Latinceye Alkhorizmi, Fransızcaya ise Algorithmme şeklinde geçmiştir. Bu sözcük daha sonra algoritmaya dönüşür ve yaygın olarak kullanılır.

Algoritma kavramının çıkmasını sağlayan kitabın hikayesi ise ilginçtir. Harezmi, bir matematik kitabı yazar ve o zamanki Abbasi halifesine sunar. Halife: "Harezmi, bu yazılanlar çok güzel ama bunları halkım anlayıp kullanamaz. Haydi, git yeniden yaz ve öyle yaz ki herkes bu kurallara uyarak problem çözebilsin" der. Bu emirle Harezmi konuyu yeniden inceler ve kitabını, herkesin anlayıp uygulayabileceği sistemli bir anlatım yapısı ile yeniden düzenler.

İşte, "Öyle yaz ki herkes bu kurallara uyarak problem çözebilsin" sözü algoritma tasarımının ve algoritmik düşünce sisteminin özünü oluşturmaktadır.

Peki, siz algoritmik düşünceye sahip biri misiniz?

Günlük yaşantısında yapılacak işleri, sıralı işlem basamaklarına ayıran başka bir ifade ile algoritmik düşünceyi karakterleri haline getiren insanlarla karşılaşabiliriz. Bu tip insanlara;

- Pikniğe gitmeden önce kibritinden tuzuna kadar “piknik listesi”,
- Alışverişe çıkmadan önce bir “alışveriş listesi”,
- Bir sınava hazırlanırken günlük “ders çalışma planı”

hazırlayanları örnek gösterebiliriz.

Algoritmik düşünce yapısına sahip olsun/olmasın esasen tüm insanlar gündelik yaşamlarında algoritmayı kullanırlar. Sabah kalkarken, gece yatarken, okula giderken, telefonla birini ararken, araba kullanırken, çay demlerken, bir tarife göre yemek yaparken veya birine yapması gerekenleri söylerken farkında olmasak da bir algoritma kullanırız. Nasıl mı?

Okula Hazırlık Algoritması:	Çay Demleme Algoritması:
1. Başla	1. Başla
2. Kalk , yataktan;	2. Doldur , çaydanlığa suyu;
3. Yık , elini-yüzünü;	3. Koy , çaydanlığı ocağa;
4. Yap , kahvaltını;	4. Yak , ocağın altını;
5. Fırçala , dişlerini;	5. Bekle , kaynamasını suyun;
6. Çıkar , pijamalarını;	6. Eğer Su kaynadı ise Demle çayı;
7. Giy , okul kıyafetlerini;	7. Bekle , 5 dakika dem almasını;
8. Hazırla , okul çantayı;	8. Servis et , çayı;
9. Çık , evden;	9. Dur
10. Dur	

İşte bu şekilde adım adım takip edilen, sıralı ve basit ifadelerden oluşan işlem basamaklarına algoritma denir.

Algoritmada yer alan her bir emir kipi (yık, yap, bekle, yak, eğer ise gibi), birer komuttur. Bir algoritma, programlama dillerine (Python, C, Visual Basic, Swift, Java gibi) uygulandığında program adını alır.

Bazı problemlerin birden fazla çözüm yolu olabilir? Örneğin çay demleme veya okula gidiş hazırlığımızı farklı yollardan yapabiliriz. Bu durumda hangi yolu tercih etmeliyiz sorusu aklımıza gelebilir. Doğrusu;

En kısa (en az adımlı) ve en güvenli yolu tercih etmek olmalıdır.

Algoritma tasarlarlarken dikkat etmemiz gereken konulardan biri de hedefe; en az işlem adımı (en az kod satırı) ile doğru ve güvenli bir şekilde ulaşmak olmalıdır.

Algoritma Hazırlamanın Faydaları

Algoritma hazırlamanın faydalarını aşağıdaki gibi sıralayabiliriz;

- Programın kodlanmasını kolaylaştırır,
- Olası kodlama hatalarını azaltır,
- Programın kontrolünü kolaylaştırır.

Algoritma Hazırlarken Nelere Dikkat Etmeliyiz?

Algoritma hazırlarken dikkat edilecek hususları maddeler halinde açıklayacak olursak:

1. Bilgisayar ortamına aktarılacak problem iyice incelenir.
2. Programlama mantığına en uygun çözüm yolu (algoritma) seçilir. En az komutla, en kısa sürede, en doğru sonuca ulaştıracak algoritma belirlenir.
3. Algoritmanın net ve basit ifadelerden oluşmasına dikkat edilir.
4. Algoritmada her işlem adımına bir satır numarası verilir.
5. Algoritma yazımı problem çözümünün başladığını gösteren **"Başla"** ifadesi ile başlar ve işlemlerin bittiğini belirten **"Dur"** ifadesi ile sonlanır.
6. Algoritma akışını değiştiren atlama komutu **"Git"**, gidilecek satır numarası ile kullanılır. {Örneğin **"3. adıma git"** gibi}.
7. Mümkün olduğu kadar az atlama **'Git (Goto)'** komutu kullanmaya özen gösterilir.
8. Son olarak algoritmanın doğru çalışıp/çalışmadığının testi (sağlaması) yapılır.

Algoritmayı İfade Şekilleri

Algoritmik olarak ifade edilebilen bir problemin, bilgisayar ortamında kodlanması çok kolaydır. Bir problemin algoritmasını aşağıdaki gibi farklı şekillerde ifade edebiliriz. Bunlar;

- Düz metin ifadeleri;
- Kaba kod (Pseudo code);
- Akış şemaları

Düz Metin İfadeleri

Algoritmanın belki de en kolay sunumu günlük konuşma dili ile yapılan düz metin ifadeleridir. Çocukluğumuzdan beri anne ve babalarımızın tatlı-sert uyarıları ile büyürüz. Örneğin "Ellerini yıka ve sofraya otur", "yemekten sonra dişlerini fırçala" şeklindeki kısa düz ifadelerle karşılaşırız. Bu şekildeki algoritma sunumu, komutların alt alta ve sıralı bir şekilde yazılması ile yapılır ve başlarına satır numaraları konur.

Örneğin; İki sayının toplamını hesaplayan basit bir matematiksel işlemin algoritmasını;

1. Başla
2. **Gir**, iki sayı (**sayı1**, **sayı2**);
3. **Topla**, girilen iki sayıyı (**t = sayı1 + sayı2**);
4. **Yaz**, **toplam** sonucu (**t**);
5. Dur

Şeklinde ifade edebiliriz. Bu şekildeki sunum, algoritmanın günlük konuşma dili ile "düz metin" kodlamasıdır.

Not. Bu örnekteki "Gir", "Topla", "Yaz" gibi emir kipleri birer komut, "sayı1, sayı2, t" gibi sayıların tutulduğu semboller ise birer değişkendir. Değişken kavramı kitabın 3.bölümünde detaylı anlatılmaktadır.

Kaba-Kod (Pseudo-Code)

Kaba (İngilizcesi: Pseudo) kod ifadeleri, yarı Türkçe, yarı İngilizce ifadelerden oluşan kod parçalarıdır. Tam bir programdan ziyade programcının bildiği bir dilde yazılmış ön hazırlık kodlarıdır. Gerçek bir programlama dilinin söz dizim kurallarını taşımazlar. Sadece genel olarak algoritmanın işleyişini gösterir.

Kullanılacak olan programlama dili ne ise kaba-kodu o dili basitleştirerek elde ederiz. Örneğin; iki sayının toplamını hesaplayan basit bir matematiksel işlemin kaba kod algoritmasını;

1. Başla
2. input (sayı1, sayı2)
3. Toplam = sayı1 + sayı2
4. print (Toplam)
5. Dur

Şeklinde ifade edebiliriz.

Dikkat ederseniz kaba-kodlar, bitmiş programlar değil algoritmalarımızı kaba program özetleri halinde sunan yarı konuşma dili - yarı programlama dili (yarı Türkçe - yarı İngilizce) ifadelerinden oluşan kod parçalarıdır.

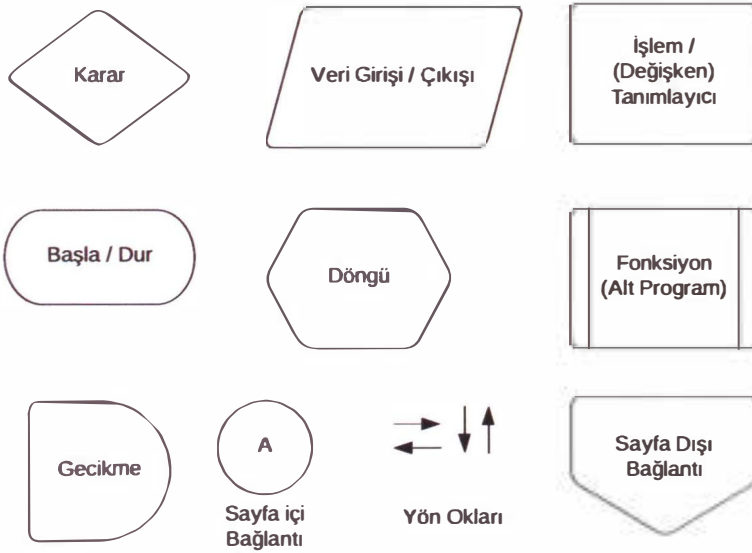
Not. Kaba-kod sunumu, daha önce bir programlama dili bilen programcılar tarafından tercih edilen bir sunum şeklidir. Doğal olarak programlamaya yeni başlayan veya yeni öğrenenler için uygun bir algoritma sunum şekli değildir.

Akış Şemaları (Flow Charts)

Algoritmanın, görsel olarak simge ya da sembollerle ifade edilmiş şekline akış şemaları (flowchart) adı verilir. Akış şemalarında her bir işlem adımı farklı bir şekil ile, şekiller arası bağlantı ise yön okları ile gösterilir. Akış şemasındaki her bir adım bir sonraki adımı işaret eden okla ifade edilir.

Akış şemaları evrensel bir dildir. Akış şemalarının avantajı, işlem adımı takibinin kolay olmasıdır. Akış şemalarının uzun olması ise bir dezavantajdır. Zira bazen problemlerin çözümü sayfalarca sürebilir.

Amerikan Ulusal Standartları Enstitüsünün (ANSI) belirlediği akış diyagramları ve kullanım amacı aşağıdaki şekilde verilmiştir.



Şekil 1. Akış Şeması Simgeleri

Algoritmayı sunum şekli önemlidir. Uzun uzun anlatım yerine kısa ve öz akış şemaları ile programı sunmak çok daha akılda kalıcıdır. Akış şemaları, karikatür tarzı anlatılmak isteneni kısa ve öz, şekillerle ifade etmenin bir yoludur.

Proje 1.1. Algoritma: İki Sayının Toplamı

İki sayının toplamını hesaplayan basit bir matematiksel işlemin algoritmasını hem metinsel düz ifade hem de akış şemaları ile gösterelim.

Algoritması (metinsel düz ifade)	Algoritması (akış şeması)
1. Başla 2. Gir, iki sayı (A, B); 3. Topla, girilen iki sayıyı ($T=A+B$); 4. Yaz, toplam sonucu (t); 5. Dur	<pre>graph TD; A([Başla]) --> B[/A, B/]; B --> C[T = A + B]; C --> D[/T/]; D --> E([Dur]);</pre>

Algoritmaya dikkat edilirse işlemlerin sıralanmasında, işlem önceliklerinin göz önünde bulundurulduğu görülür. Ayrıca algoritma yazımı problem çözümünün başladığını gösteren "Başla" ifadesi ile başlamakta ve işlemlerin bittiğini belirten "Dur" ifadesi ile sona ermektedir.

Proje 1.2. Algoritma: Karenin Çevresini Hesaplama

Bir kenar uzunluğu girilen karenin çevresini hesaplayan programın algoritmasını hem metinsel düz ifade hem de akış şemaları ile gösterelim.

Kare, 4 kenarı eşit uzunlukta olan bir dörtgendir. Karenin çevresini bulmak için bir kenar uzunluğunu 4 ile çarpmak gerekir. Bir kenar uzunluğunu " k " ile gösterirsek karenin çevresini " $Cevre = 4*k$ " formülü ile hesaplayabiliriz.

Algoritması (metinsel düz ifade)	Algoritması (akış şeması)
1. Başla 2. Gir, karenin bir kenar uzunluğunu (k) 3. Hesapla, karenin çevresini (Cevre = $4*k$); 4. Yaz, işlem sonucunu (Cevre); 5. Dur	<pre> graph TD A([Başla]) --> B[/k/] B --> C[Cevre=4*k] C --> D[/Cevre/] D --> E([Dur]) </pre>

Proje 1.3. Algoritma: Geçti - Kaldı

Girilen bir ortalama not değerine göre (50 ve üzeri ise) ekrana "Geçti" değilse (50'den küçükse) "Kaldı" yazan programın algoritmasını hem kaba kod hem de akış şemaları ile gösterelim.

Algoritması (kaba kod)	Algoritması (akış şeması)
1. Başla 2. Input Ortalama; 3. Eğer Ortalama ≥ 50 ise print "Geçti" Değilse print "Kaldı" 4. Dur	<pre> graph TD A([Başla]) --> B[/Ortalama/] B --> C{Ortalama >= 50} C -- E --> D[/Geçti/] C -- H --> E[/Kaldı/] D --> F([Dur]) E --> F </pre>

Proje 1.4. Algoritma: Trafik Lambası

Bir robot araba tasarladınız ve bu arabanın bir trafik kavşağına geldiğinde trafik lambasının durumuna göre nasıl hareket etmesi gerektiğini programlamanız gerekiyor. Ne yapmanız gerekir?



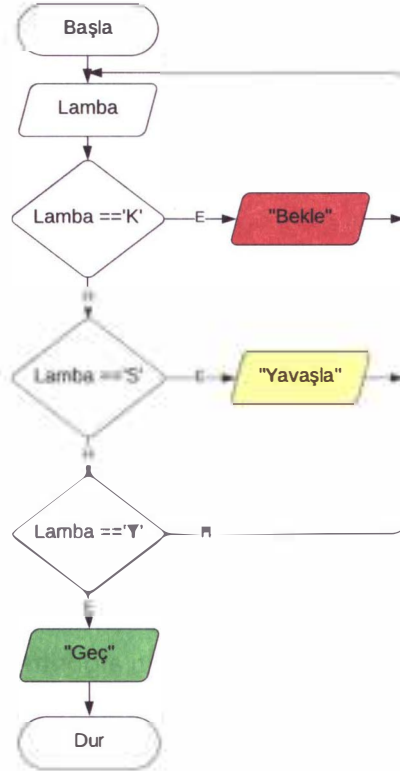
Trafik lambalarının yanması sırasıyla, "yeşil - sarı - kırmızı" veya "kırmızı - sarı - yeşil" şeklinde olduğundan bu sıra dikkate alınarak algoritma hazırlanacaktır. Yeşil ışıktaki geçilecek, sarı ışıktaki yavaşlanacak ve kırmızı ışıktaki beklenektir.

Programda "kırmızı - sarı - yeşil" renk düzeneği "K - S - Y" karakterleri ile gösterilmiştir.

Algoritması (metinsel düz ifade)

1. Başla
2. Oku, trafik lambasını (Lamba)
3. Eğer (Lamba == 'K') ise
"Bekle" ve 2. adıma Git,
4. Eğer (Lamba == 'S') ise
"Yavaşla" ve 2. adıma Git,
5. Eğer (Lamba == 'Y') ise
"Geç"
- Değilse 2. adıma Git
6. Dur

Algoritması (akış şeması)



Not. Algoritmayı akış şeması şeklinde ifade eden ve adım adım çalıştırarak test eden, aynı zamanda herhangi bir programlama dilinde (C, Python, Java gibi) koda dönüştüren akış şeması yorumlayıcıları da vardır. Bu programlara; Flowgorithm ve RAPTOR örnek gösterilebilir.

Flowgorithm programını <http://www.flowgorithm.org/> adresinden indirebilir, akış şemaları hazırlayabilir ve otomatik olarak Python dilinde koda dönüştürebilirsiniz.

Sıra Sende Etkinliği

Trafik lambası örneğindeki gibi günlük yaşamında karşılaştığın veya gözlemlediğin bir etkinliğin algoritmasını akış şemaları ile kodlayabilir misin?

{Örneğin bir kek yapma işleminin algoritmasını yazabilirsin}.

Soru ve Cevaplarla Neler Anladık?

1 Algoritma nedir?

Algoritma, herhangi bir problemi çözmek için adım adım izlenecek yol anlamına gelmektedir.

2 Algoritma kelimesine adını veren kişi kimdir?

Harezmi.

3 Algoritmayı sunum şekilleri nelerdir?

Metinsel düz ifade, kaba kod, akış şemaları.

4 Kaba program özetleri halinde yarı konuşma dili yarı programlama dili ifadelerinden oluşan algoritmaya ne ad verilir?

Kaba-kod (Pseudo-code).

5 Bazı problemlerin birden fazla çözüm yolu olabilir? Bu durumda hangi yolu (algoritmayı) tercih etmeliyiz?

En kısa (en az adımli), en güvenli ve doğru yolu tercih etmeliyiz.

6 Algoritma hazırlamanın faydaları nelerdir?

- Programın kodlanmasını kolaylaştırır,
- Olası kodlama hatalarını azaltır,
- Programın kontrolünü kolaylaştırır.

7 Bir algoritmanın özelliklerini sıralayınız?

- Net olmalı, belirsizliklere yer vermemeli.
- Bilgisayar ortamında çalıştırılabilmeli.
- Sıralı işlem adımlarından oluşmalı.
- Sonlu bir zamanda (belirli bir vakitte) çözülebilmeli.

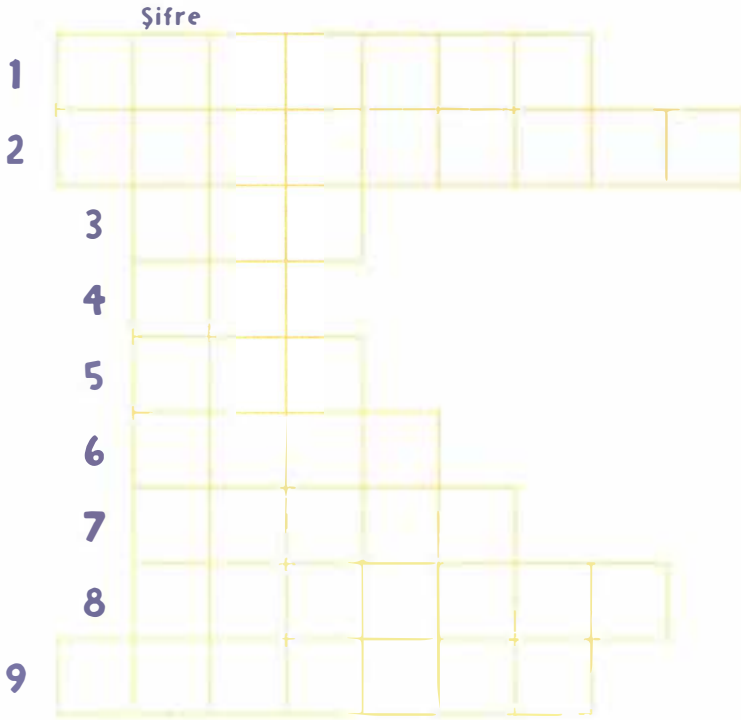
8 Akış şeması (flowchart) nedir?

Akış şemaları (flowcharts); algoritmanın, görsel olarak simgelerle ya da sembollerle ifade edilmesine denir. Akış şeması simgeleri evrenseldir.

9 Klavyeden girilen A ve B gibi iki sayının ortalamasını alan programın algoritmasını oluşturunuz.

1. Başla
2. Gir, iki sayı (A,B);
3. Hesapla, Ortalamayı ($\text{Ortalama} = (A+B) / 2$);
4. Yaz, Ortalamayı (Ortalama);
5. Dur

BULMACA



1. Algoritmaya ismini veren bilgin.
2. "Akış şeması" İngilizcesi
3. "Gir, iki sayı (S1, S2)" ifadesindeki komut.
4. Akış şemasındaki bir sonraki adımı işaret eden simge.
5. (Tersi) Algoritmanın bittiğini belirten ifade.
6. (Tersi) "Amerikan Ulusal Standartları Enstitüsü" kısaltması.
7. (Tersi) Algoritmadaki "gir", "topla", "yaz" gibi emir kipleri.
8. (Tersi) Bir algoritmanın bir programlama dili (Python, C, Java gibi) ile kodlandığında aldığı isim.
9. "Pseudo-code" Türkçesi.

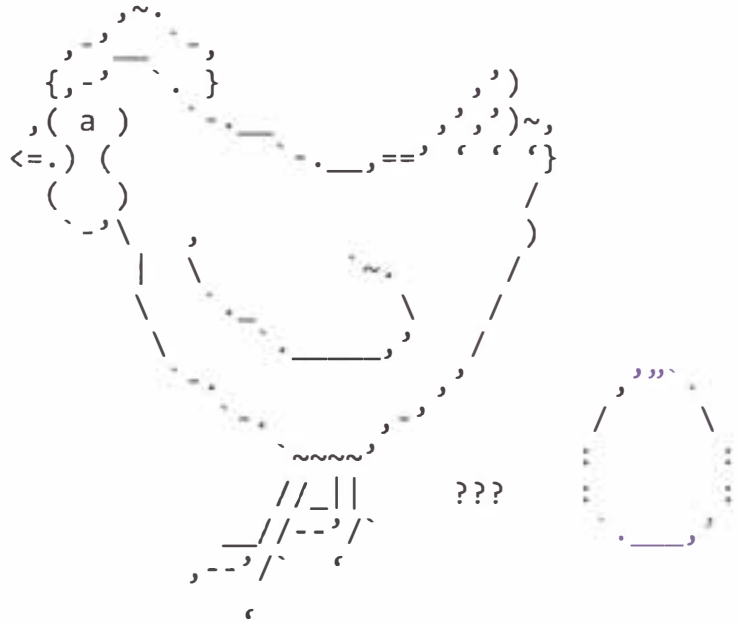
Aşağıya bulduğunuz şifreyi kodlayınız.

--	--	--	--	--	--	--	--	--	--

Bölüm

2

Bilgisayar Kavramı ve Python Dili



Tavuk mu yumurtadan, yumurta mı tavuktan çıkar?

Bilgisayar ve Bileşenleri

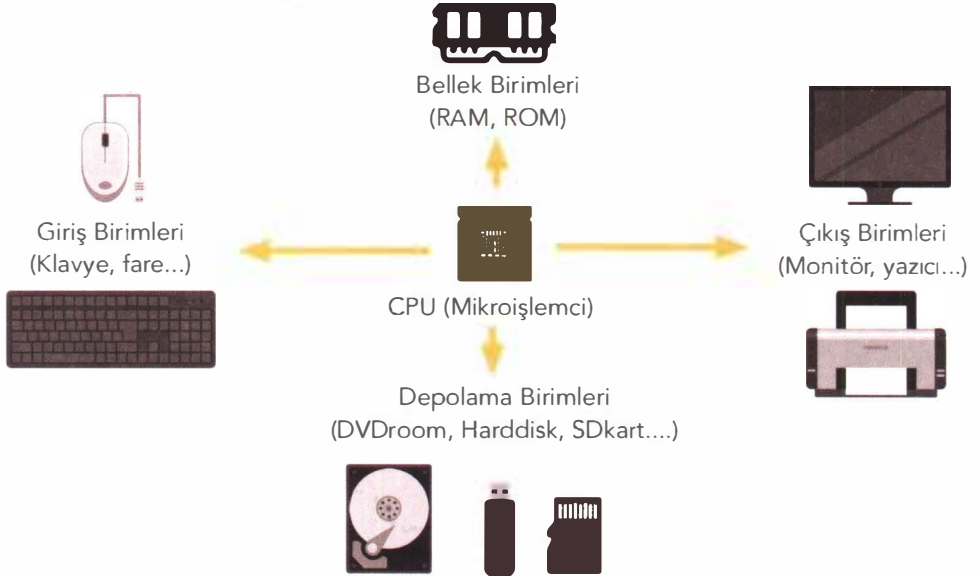
Bilgisayar kelimesinin İngilizcesi, "Computer" dir. "Computer" kelimesi ise matematiksel problemleri hesaplama anlamındaki "compute" kelimesinden türetilmiştir. Kelime anlamından yola çıkılarak bilgisayarların en önemli işi hesaplama değildir denilebilir.

TDK (Türk Dil Kurumu) sözlüğünde ise bilgisayar; Çok sayıda aritmetiksel veya mantıksal işlemlerden oluşan bir işi, önceden verilmiş bir programa göre yapıp sonuçlandıran elektronik aygıt, elektronik beyin, yani "kompüter" olarak tanımlanmaktadır.

Bilgisayar; çoğu icat gibi doğadaki varlıklardan biri (insan) ilham alınarak geliştirilmiştir. İnsanın beden ve ona canlılık veren ruhtan oluşması gibi bilgisayarlarda donanım ve yazılım olmak üzere iki ana bileşenden oluşmaktadır.

Donanım (Hardware)

Bilgisayarın fiziksel yapısını ifade eder. Bilgisayarın gözle görülen, elle dokunulan parçalarına "donanım (hardware)" denir. Bir bilgisayarı oluşturan en temel donanımsal birimler aşağıda gösterilmiştir.



Bir bilgisayar sistemi en azından aşağıdaki birimlerin bir araya gelmesi ile oluşur;

1. Giriş Birimleri (klavye, fare, mikrofon, tarayıcı, hard disk gibi): Bilgisayarı kullanan kişilerin bilgisayarın işlemcisi ile haberleşmesini sağlayan birimlerdir (cihazlardır).

2. **Çıkış Birimleri (monitör, yazıcı, hoparlör, projeksiyon, hard disk gibi):** Bilgisayarın işlediği verileri kullanıcıların anlayabileceği şekle/formata dönüştüren birimlerdir (cihazlardır).
3. **Mikroişlemci (CPU):** Aritmetik ve mantık işlemlerinin yapıldığı, yapılan işlemlerin denetlendiği/yönlendirildiği birimdir (cihazdır). Bilgisayarın kalbi ve beyni mikroişlemcidir, denilebilir.
4. **Bellek (Hafıza) Birimleri (RAM/ROM Bellek):** Bilgisayar üzerinde çalışan verilerin sürekli veya geçici olarak saklandığı hafıza birimleridir.
5. **Depolama birimleri (hard disk (HDD), flash disk, DVD-ROM gibi):** Verilerin kalıcı olarak depolandığı mekanik veya fiziksel depolama alanlarıdır.

Yazılım (Software)

Bilgisayarda kullandığımız (donanım üzerinde çalışan) her türlü programa "yazılım (software)" denir. Yazılımlar, bilgisayarlara hayat ve canlılık veren programlardır. Yazılım yüklü olmayan bilgisayarın ölü bir bedenden farkı yoktur.

Aşağıdaki tabloda donanım ve yazılım örneklerine yer verilmiştir.



Tüm yazılımlar bir programlama dili ile yazılırlar.

Soru: Tüm yazılımlar, mutlaka bir programlama dili kullanılarak yazıldığına göre programlama dilleri ne ile yazılmıştır?

Cevap: Bu sorunun cevabını başka bir soru ile verelim. - Tavuk mu yumurtadan çıktı, yumurta mı tavuktan çıktı? Başlangıçta bir yumurta ya da tavuğun varlığını kabullenmemiz gerekir, öyle değil mi?



Program ve Programlama Dilleri

Program, bilgisayara ne yapması gerektiğini söyleyen bir grup komuta (kod topluluğuna) verilen isimdir. Bu komutları veren kişiye programcı, programcının konuştuğu dile ise programlama dili denir. Bir programlama dili, programcının bilgisayara ne yapması gerektiğini anlatan komutlar kümesidir.

Programcı, komutları farklı programlama dilleri kullanarak yazabilir/kodlayabilir. Günümüzde yüzlerce programlama dili mevcuttur. En yaygın kullanılan programlama dillerine Python, C, C++, C#, Visual Basic, Java, Javascript, Swift, PHP gibi diller örnek gösterilebilir.

Makine Dili

Bilgisayarın konuştuğu dile makine dili denir. İlk programlama dilleri de makine dilinde yazılmıştır. Bu dilde yazılan tüm komutlar 1 ve 0'lardan oluşur. Örneğin "hello" yazısının makine dilinde kodlanması aşağıdaki gibidir;

```
01101000 01100101 01101100 01101100 01101111
```

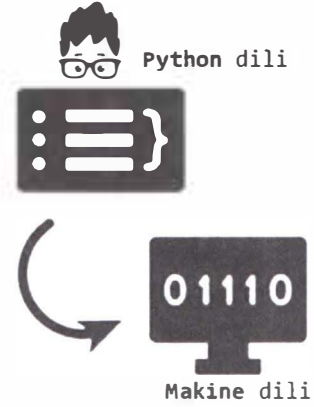
Derleyici (Compiler) ve Yorumlayıcı (Interpreter)

Python dili ile konuşan programcı ile makine dili (1 ve 0 rakamlarından oluşan dil) ile konuşan bilgisayar arasındaki iletişim nasıl gerçekleşir?

Nasıl ki birbirinin konuşma dilini bilmeyen iki yabancı devlet adamı arasındaki görüşme; tercüman veya çevirmenler aracılığı ile yapılabiliyorsa, benzer şekilde

programcının konuşma dilini bilgisayarın makine diline çevirmek/dönüştürmek için (yani bir nevi tercümanlık yapmak için) derleyici veya yorumlayıcı adı verilen yazılımlar kullanılır.

Yazılan programları makine diline çeviren, makine dilinde elde edilen sonuçları programcının anladığı biçime dönüştüren programlara derleyici veya yorumlayıcı programlar denir. Bazı programlama dilleri derleyici kullanırken bazıları ise yorumlayıcı kullanırlar.



Derleyici (Compiler): Derleyici, herhangi bir programlama dili ile yazılmış bir programı, makine diline (çalıştırılabilir koda) çevirir. Derleyiciler, bir programı makine diline çevirirken kurallarına ters düşen hata bildirimlerini; programın tamamı için oluşturur ve aynı anda tüm hataların bir listesini verirler. Fortran, C, C ++, C#, Pascal, Delphi gibi diller derleyici kullanırlar.

Yorumlayıcı (Interpreter): Yorumlayıcı, herhangi bir programlama dili ile yazılmış bir programı, ilk satırından son satırına doğru satır-satır (adım adım) çalıştırır. Yorumlayıcı; herhangi bir komut satırının çevrilmesinde ya da çalıştırılmasında bir hatayla karşılaştığında anında çalışmayı durdurur ve hatalı satırı programcıya bildirir. Yorumlayıcılar genellikle anlık olarak kod dönüşümü yaptıkları için, derleyicilere göre daha yavaş çalışırlar. Basic, Python, PHP, Perl, Lisp, Java gibi diller yorumlayıcı kullanırlar.

Derleyici ile Yorumlayıcı arasındaki farkı kısaca şöyle açıklayabiliriz:

Derleyici programın tamamını kontrol eder, bir satırında hata varsa tüm program satırlarını derledikten sonra hatayı gösterir. Yani kaynak kodu bir defa da derler.

Yorumlayıcı ise programı yukarıdan aşağıya satır satır derler bir satırında hata varsa hatalı satıra geldiğinde durur ve hata mesajı verir. Yorumlayıcı bir satırdaki hata giderilmeden bir alttaki komut satırına geçmez. Hata düzeldikten sonra tekrar ilk satırdan itibaren kodu derler.

Kaynak Program (Source Program) ve Python Editörleri

Herhangi bir programlama dili ile yazılmış/kodlanmış ham, derlenmemiş programa kaynak program (kod) denir. Python programı veya kodu denildiğinde Python dilinde yazılmış kaynak program kastedilir.

Kaynak program dosyasının uzantısı yazıldığı dile göre değişir. Örneğin

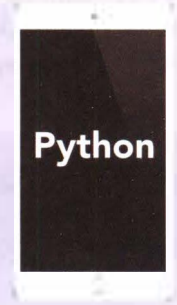
- C dilinde yazıldı ise .c
- C++ dilinde yazıldı ise .cpp
- Assembly dilinde yazıldı ise .asm
- Java dilinde yazıldı ise .java
- Python dilinde yazıldı ise .py

Python programı dahil tüm programlar bir kod editörü ile yazılır. Python editörlerine örnek olarak; IDLE, Notepad++, Atom, Eclipse PyDev, PyCharm, Geany, Visual Studio Code, Jupyter Notebook gibi programlar gösterilebilir.

Not. Bu kitaptaki uygulamalar için Python kurulumu ile gelen IDLE editörü tercih edilmiştir.

Soru: Mobil telefonumdan veya tablet bilgisayarımdan kitaptaki Python programlarını yazıp çalıştırabilir miyim?

Cevap: Evet. Bunun için akıllı telefon ya da tabletin uygulama indirme mağazasının (App Store veya Play Store gibi) arama alanına "Python IDE" yazmanız yeterlidir. Çok yıldız alan (beğenilen) herhangi bir uygulamayı (Örneğin Android telefonlar için Pydroid 3 veya QPython 3, iPhone telefonlar için Python3 IDE gibi) tercih edebilirsiniz. Böylece Python kodlarını yazıp çalıştırabilirsiniz.



Çalıştırılabilir Dosya (Executable File)

Bir programlama dilinde yazılmış kaynak programın derlenmesi ile elde edilen, makine diline dönüştürülmüş ve çalışmaya hazır programlara denir. Bu dosyaların Windows, MS-DOS tabanlı sistemlerdeki dosya uzantısı .exe'dir. Bir .exe uzantılı dosyayı çalıştırmak için "run (çalıştır)" komutu vermek yeterlidir.

Not. Bir Python kaynak dosyasını (.py uzantılı) çalıştırılabilir dosya (.exe uzantılı) formatına dönüştürmek için PyInstaller, py2exe gibi programlar kullanılabilir.

Python Dili



Guido van Rossum

1991 yılında, Hollandalı bilgisayar mühendisi Guido van Rossum tarafından geliştirilmeye başlanmıştır. Python ismini sanılanın aksine bir yıldan değil Monty Python adlı altı kişilik bir İngiliz komedi grubunun "Monty Python's Flying Circus" adlı gösterisinden almıştır. Günümüzde "Python Yazılım Vakfı" gönüllüleri tarafından geliştirilen dil; basitliği, nesne yönelimli olması, hemen hemen her ortamda çalışması (Linux, MacOS, Windows, ...) ve ücretsiz dağıtımını gibi sebeplerle yazılımcılar tarafından tercih edilmektedir.

Python; modüler, %100 nesne yönelimli, açık kaynaklı (ücretsiz), platform bağımsız, yorumlayıcı kullanan, hızlı ve dinamik bir dildir. Python açık kaynaklı, ücretsiz bir dildir. Çok geniş kütüphanelere (modüllere) sahiptir.

Python dili ile konsol uygulamaları, robotik uygulamalar, görsel masaüstü (GUI) uygulamaları, web tabanlı uygulamalar, veri analizi ve veri görselleştirme, yapay zeka uygulamaları gibi çok farklı programlar yazılabilir.

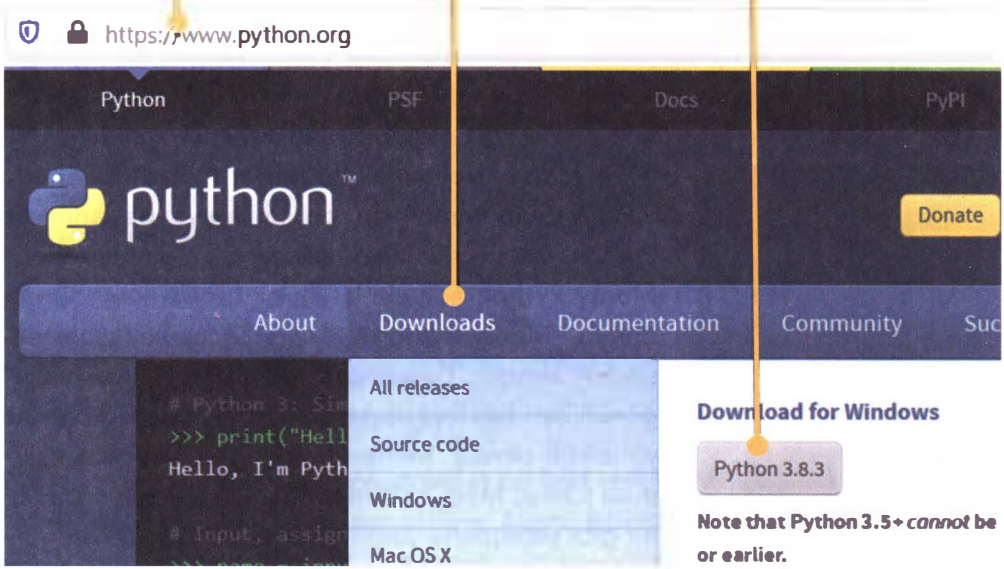
Python Kurulumu

Python dilinin en güncel sürümü www.python.org adresinden indirilmelidir. Ayrıca Python hakkındaki tüm dokümanlara bu adresten erişilir.

Python kurulumu için herhangi bir arama motoruna (örneğin Google) "www.python.org" yazıldığında aşağıdaki gibi bir sayfa gelir.

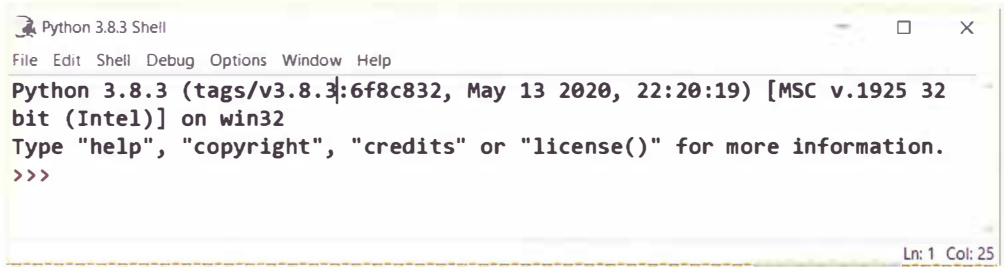
Download sekmesi tıklatıldığında bilgisayarınız için en uygun son sürüm sağda görülür.

Web adresi Download (indirme) sekmesi En son Python sürümü



Python kurulumu tamamlandıktan sonra Başlat menüsünden (veya arama sayfasına IDLE yazılarak) IDLE seçilerek Python editörü çalıştırılabilir.

IDLE editörü açıldığında Python kabuk (shell) ekranı gelecektir. Python kabuk ekranı bir interaktif (etkileşimli) komut istemi (>>>....) ekranıdır. Python interaktif komut istemini (>>>....) aynı zamanda çok gelişmiş bir hesap makinesi olarak kullanabilirsiniz.

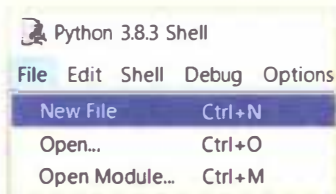


Yani bu ekranda kısa (birkaç satırlık) kod satırları yazılıp çalıştırılacağı gibi çok ileri matematiksel hesaplamalar da yapılabilir.

```
===== RESTART: Shell =====
>>> 3*5
15
>>> 111*5/3
185.0
>>> "Python " * 5
'Python Python Python Python Python '
>>> print("Merhaba Python")
Merhaba Python
```

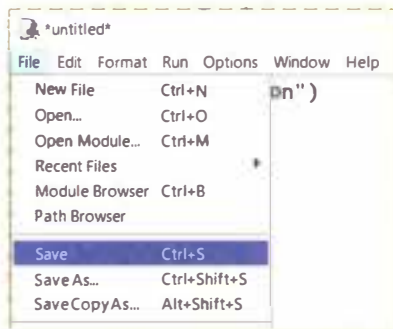
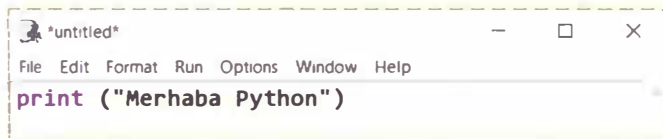
Python Programlarını Saklamak

Python kabuk ekranında yaptığımız işlemler, yazdığımız programlar ekranı veya bilgisayarını kapattığımızda kaybolur, gider.



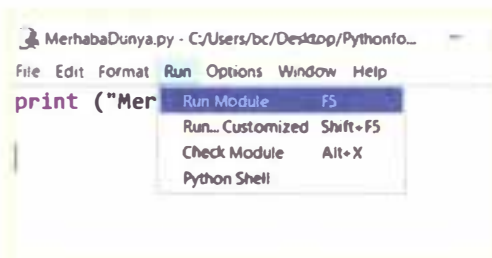
Yazdığımız programların kaybolmaması için bilgisayarın depolama birimlerinde (hard diskinde) saklanması gerekir. Bunun için File menüsünden New File seçilir.

Ve açılan pencereye program yazılır.



Yazılan programın çalıştırılabilmesi için kaydedilmesi gerekir. Bunun için File menüsünden Save seçilerek dosyaya bir isim verilir ve kaydedilir.

Daha sonra ise Run menüsünden Run Module seçilerek program çalıştırılır.



Not. Farklı bir yol olarak; programı kaydetmek için **Ctrl+S**, çalıştırmak için ise **F5** klavye tuşları kullanılabilir.

Sıra Sende Etkinliği

Python kurulu olmayan bir bilgisayara Python programını kurabilir misin? Daha sonra da IDLE editörünü açıp Python kabuk (shell) ekranına aşağıdaki kodu yazdığında alt satırda neler gördüğünü söyleyebilir misin?

```
>>> "müdür " * 3
```

Soru ve Cevaplarla Neler Anladık?

1 Bilgisayarın tanımını yapabilir misiniz?

Çok sayıda aritmetiksel veya mantıksal işlemlerden oluşan bir işi, önceden verilmiş bir programa göre yapıp sonuçlandıran elektronik aygıt.

2 Donanım (hardware) nedir?

Bilgisayarın gözle görülebilen, elle dokunulabilen parçaları donanım olarak isimlendirilmektedir.

3 Bir bilgisayarı oluşturan en temel donanım birimleri (parçaları) nelerdir?

1. Giriş birimleri (Klavye, Mouse, dokunmatik ekran gibi...)
2. Çıkış birimleri (Monitör, Yazıcı, Hoparlör gibi),
3. İşlemci (CPU),
4. Hafıza birimleri (RAM),
5. Depolama birimleri (Hard disk, USB Bellek gibi)

4 Yazılım (software) nedir?

Bilgisayarda kullandığımız (donanım üzerinde çalışan) her türlü programa yazılım denir.

5 Programlama dili nedir?

Bir programlama dili, programcının bilgisayara ne yapması gerektiğini anlatan komutlar kümesidir. Programcının konuştuğu dildir.

6 Derleyici (compiler) ile yorumlayıcı (interpreter) arasındaki fark nedir?

Derleyici programın tamamını bir defa da derlerken, yorumlayıcı satır satır derler. Yorumlayıcı bir satırdaki hata giderilmeden bir alttaki komut satırına geçmez. Hata düzeldikten sonra tekrar ilk satırdan itibaren kodu derler.

7 Kaynak program dosyasının uzantısı yazıldığı dile göre değişir. Python dilinde yazılan bir kaynak programın uzantısı nedir?

".py" dir.

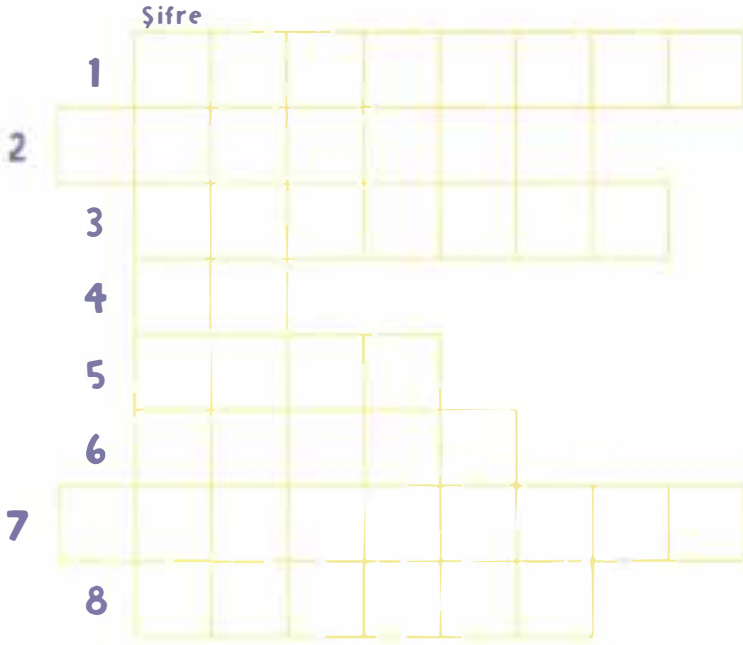
8 IDLE nedir?

Python kurulumu ile birlikte gelen resmi Python editörüdür.

9 Python dili ilk kim tarafından geliştirilmiştir?

Hollandalı bilgisayar mühendisi Guido van Rossum tarafından geliştirilmiştir.

BULMACA



1. "Bilgisayar" kelimesinin İngilizcesi.
2. Bilgisayarın gözle görülen, elle dokunulan parçalarına verilen isim.
3. (Tersi) Bilgisayara ne yapması gerektiğini söyleyen bir grup komut.
4. Python dilinde yazılan programların uzantısı.
5. Python kurulumu ile birlikte gelen resmi Python editörü.
6. (Tersi) Python kabuk ekranının (interaktif komut isteminin) İngilizcesi.
7. Bir satırdaki hatayı tüm program satırlarını derledikten sonra gösteren program.
8. Python dilini geliştiren kişinin soyadı.

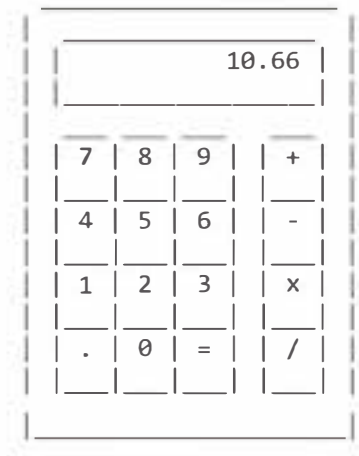
Aşağıya bulduğunuz şifreyi kodlayınız.

--	--	--	--	--	--	--	--

Bölüm

3

Değişkenler ve Hesaplar

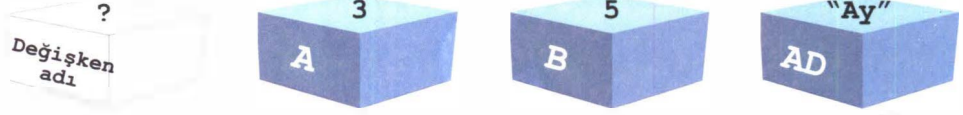


Python komut istemi (Python shell ekranı); bir hesap makinesi gibi kullanılabilir:

```
>>> 1453 * 19 / (2000 + 20) - 3  
10.666831683168317
```

Değişken Nedir? Nasıl Değer Aktarılır?

Değişkenler içerisinde veri (değer) sakladığımız küçük bellek (hafıza) alanlarıdır. Değişkenleri üzerine etiket yapıştırılan saklama kaplarına benzetebiliriz. Programın çalışması boyunca bu saklama kaplarındaki veriler (değerler) değişebilir. Bu nedenle bu saklama kaplarına "değişken" adı verilmiştir.



Not. Bilgilerin geçici olarak tutulduğu yere bellek (hafıza) denir. Değişkenler de bilgisayar belleğinde tutulurlar/saklanırlar.

Matematiksel ya da sözel bir ifadeyi bir değişkene aktarmak/atamak için "=" eşit operatörü kullanılır ($A = 3$, $B = 5$, $AD = "Ali"$ gibi).

Python dili aynı satırda birden fazla değişkene "=" operatörü ile değer atamasına izin verir. Örneğin;

$A = B = 5$ şeklindeki bir atamada hem A hem de B değişkenlerinin değeri 5 değerini alır.

Uyarı! Değişkenlere değer atanırken değişken ismi eşit simgesinin sol tarafında yer almalıdır. Örneğin $A=5$ doğru iken $5=A$ yanlış bir ifadedir.

Değişkenlere İsim Verme Kuralları

Değişken isimleri programcı (yani sen 😊) tarafından verilir. Fakat gelişigüzel isim verilmez. Aşağıdaki kurallara dikkat edilmesi gerekir. Bunlar;

1. Bir değişkenin ilk karakteri mutlaka harf ya da alt çizgi _ olmalıdır.
2. Değişken isimlerinin içerisinde alt çizgi _ hariç, boşluk veya diğer özel karakterler bulunmamalıdır (.,@ ? * : ; !, {},[] / - + = % & " # gibi karakterler kullanılamaz).
3. Değişken ismi bir Python komutu olamaz. Python diline ait komutlara anahtar sözcükler (keywords) de denilmektedir. Değişken ismi olarak kullanılamayacak Python komutlarının güncel bir listesini aşağıdaki gibi `kwlist` komutu ile öğrenebiliriz;

```
>>> import keyword
>>> keyword.kwlist
['False', 'None', 'True', 'and', 'as', 'assert', 'async', 'await',
'break', 'class', 'continue', 'def', 'del', 'elif', 'else', 'ex-
cept', 'finally', 'for', 'from', 'global', 'if', 'import', 'in', 'is',
'lambda', 'nonlocal', 'not', 'or', 'pass', 'raise', 'return', 'try',
'while', 'with', 'yield']
```

4. Python kabul etse de değişken isimlerinde Türkçe karakter kullanmamaya özen gösterilmelidir.

5. Python büyük-küçük harf ayrımı yapan bir dildir. Yani 'AD' ile 'Ad' farklı değişkenler olarak kabul edilir. Bu duruma dikkat edilmelidir.

Python dili için geçerli/geçersiz değişken isimlendirmeleri:

Geçersiz değişken isimlendirme örnekleri		Geçerli değişken isimlendirme örnekleri	
is	Bir Python komutu	adNo	Geçerli
4ad	Rakamla başlamış	ada54	Geçerli
Ad.no	Özel karakter kullanılmış	ad_No	Geçerli
#sayı	Özel karakter kullanılmış	sayı	Geçerli
not	Bir Python komutu	nt	Geçerli
Ad No	Arada boşluk bırakılmış	_adNo	Geçerli
mi?	Özel karakter kullanılmış	miSoru	Geçerli
None	Bir Python komutu	none	Geçerli

Not. Eğer bir değişkenin içeriği, program boyunca hiç değişmeyecekse sabit kalacaksa büyük harfle gösterilir. TCNO = 2456789345 gibi.

Yer değiştirme (İki Değişkenin İçeriğini Birbirine Aktarma) İşlemi

İki değişkenin içeriklerini birbirine aktarma işlemi "yer değiştirme" olarak isimlendirilir. Aşağıda gösterildiği gibi Python dilinde değişkenler doğrudan birbirine aktarılabilirken çoğu programlama dilinde bu işlem için üçüncü (C gibi) bir değişkene ihtiyaç duyulur.

Python'da yer deęiřtirme

```
A=3
B=5
A, B = B, A
# A = 5
# B = 3 ise olur
```

C/C++/Java gibi dillerde yer deęiřtirme

```
A=3
B=5
C = A #A nın deęeri C'ye aktarıldı.
řimdi A boş
A = B #B nin deęeri A'ya aktarıldı.
řimdi B boş
B = C #C nin deęeri B'ye aktarıldı.
řimdi C boş
# A = 5
# B = 3 ise olur
```

Bařlangıřta Deęiřkenlere Deęer Aktarmanın Mantıęı

Bazen programda kullandıęımız deęiřkenleri tanımlama yaparken bařlangıř deęeri atamak gerekir. Örneęin bir toplama deęiřkenini program bařında sıfıra eřitlemek gibi. Peki, bu iřlemi niçin yaparız?

Bunu bir örnek üzerinden açıklayalım:

- 5 kiři aramızda para toplayıp, arkadaşımızın doğum günü için bir yař pasta alacaęız. Ben para toplama iřini üstlendim ve herkesten 5–10 TL arası bir para aldım ve topladıęım paraları pantolonumun saę cebine koydum. řimdi benim kař TL para topladıęımı nasıl bileceęiz? Saę cebimdeki tüm parayı çıkarıp bu kadar para topladık desem, bu ne kadar doęru olur?
- Bunu bilemeyiz deęil mi? Emin olmak için para toplama- dan önce saę cebimi boşaltmam gerekmez miydi? řşte bizde toplama veya çıkarma iřlemleri ile ilgili deęiřkenleri programın bařında sıfıra (etkisiz elemana) eřitleyerek bir nevi cep boşaltma iřlemi yapmıř oluyoruz.



Proje 3.1. Kodlama: İki Sayının Toplamı

İki sayının toplamını hesaplayan iřlemi yapalım.

Algoritması

1. Bařla
2. **T = 0**
3. **Gir**, iki sayı (**A**, **B**)
4. **Topla**, girilen sayıları
(**T = A + B**)
5. **Yaz**, Toplam deęeri (**T**)
6. **Dur**

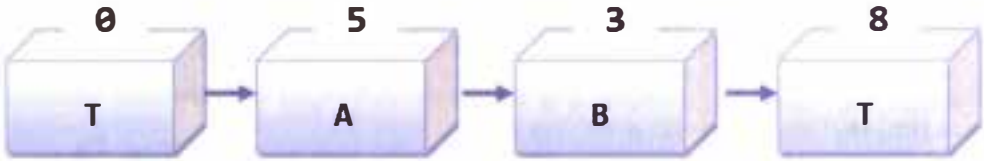
Python kodlaması

```
T = 0
A = 5
B = 3
T = A + B
print (T)
```

Not. Bu program, IDLE editöründe File → New File seçilerek kodlanabilir.

Açıklama. Bu örneğimizdeki; A, B ve T birer değişkendir. =, + simgeleri ise birer operatördür.

Adım adım değişkenlerin aldığı değerleri takip edelim;



“Yaz” bir komuttur. Yaz komutunun Python dilindeki eşdeğeri “print” komutudur. T’nin son değeri print komutu ile ekrana yazdırılmaktadır. Programın ekran çıktısı 8 olacaktır.

Operatörler

Hesaplama işlemlerinde kullanılan işaretlere operatör (işleç) adı verilir. Operatörlerle işlem yaparken işlem önceliğine ve çalışma sırasına dikkat etmek gerekmektedir.

Matematiksel işlemlerde kullanılan operatörler ve örnek kullanımı:

İşlem	Operatör	Örnek kullanım	Sonuç
Toplama	+	3 + 5	8
Çıkarma	-	12 - 7	5
Çarpma	*	8 * 4	32
Bölme	/	10 / 4	2.5
Tamsayı bölme	//	10 // 4	2
Üs alma	**	3 ** 2	9
Mod (Kalanı alma)	%	8%3	2

Not. Mod alma (%) operatörü; bir bölme işlemi sonucunda kalan değeri verir.

Proje 3.2. Basit Hesap Makinesi

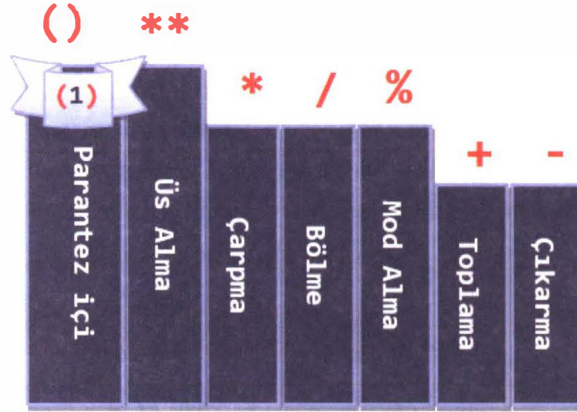
Haydi basit bir hesap makinesi yapalım.

Python kodlaması	Ekran çıktısı
<pre>A = 3 B = 5 print ("## Basit Hesap Makinem ##") print (A, '+', B, '=', A + B) print (A, '-', B, '=', A - B) print (A, '*', B, '=', A * B) print (A, '/', B, '=', A / B) print (A, 'mod', B, '=', A % B) print (A, '^', 2, '=', A ** 2)</pre>	<pre>## Basit Hesap Makinem ## 3 + 5 = 8 3 - 5 = -2 3 * 5 = 15 3 / 5 = 0.6 3 mod 5 = 3 3 ^ 2 = 9</pre>

Matematiksel Operatörlerin İşlem Önceliği

Matematiksel işlemlerde kullanılan operatörlerin öncelik sırasını 3 seviyede ifade edebiliriz;

1. Parantez içleri ve üs alma
2. Çarpma, bölme ve mod (kalanı) alma
3. Toplama ve çıkarma



Not. Aynı önceliğe sahip işlemlerde (+,- gibi) soldaki daha önce işlem görür. Bir matematiksel işlemin önceliğini (parantez içine) alarak değiştirebiliriz.

Aşağıdaki işlemlerde işlem önceliğinin ve parantez içine almanın sonucu nasıl değiştirdiğine dikkat ediniz.

```
>>> 3+5*2/4
5.5
>>> (3+5)*2/4
4.0
>>> 12%8+(3-5)*4
-4
>>> 6--5
11
>>> #İki '-' işaretin çarpımı '+' dır.
```

Proje 3.3. Matematiksel İşlemlerin Kodlanması

Aşağıdaki matematiksel işlemi Python dilinde kodlayalım.

$$e = \frac{4^2 + 3/5}{3^2 - 2 * 5}$$

Matematiksel işlemleri basitleştirerek kodlamakta fayda vardır. Bu işlemi doğrudan $e = (4^{**2} + 3/5)/(3^{**2} - 2*5)$ şeklinde kodlayabiliriz. Fakat her bir işlemi ayrı bir değişkene atayarak kodlamak programı daha anlaşılır hale getirir.

Python kodlaması	Ekran çıktısı
<pre>a = 4**2 b = 3/5 c = 3**2 d = 2*5 e = (a + b) / (c - d) print ('e =', e)</pre>	<pre>e = -16.6</pre>

Değer Atama İşlemlerini Basitleştirme

Python dilinde matematiksel işlemlerde; değer atama (eşittir) operatörü "=" ve matematiksel operatörlerin birleşmesi ile yeni operatörler oluşturulabilir. Bu yeni operatörlere aritmetiksel atama operatörleri adı verilir. Bu operatörlerin gösterimi, işlem operatörü ile "=" eşittir simgesinin (operatörünün) birleştirilmesi { +=, -=, *=, /= gibi } şeklinde olur. Örnek kullanım ve eşdeğeri;

Kısa gösterim	Açılımı (eşdeğeri)
Toplam += 5	Toplam = Toplam + 5

Bu örnekte '+=' operatörünün yaptığı işlevi;

"değişkene eşitliğin sağındaki değeri ekle ve sonucu yine değişkene aktar" şeklinde açıklanabilir.

Not. Eğer Toplam += 5 satırı bir döngü içerisinde yazılacak olursa ardışık toplama (beşerli ritmik sayma) işlemi gerçekleşmiş olur.

Yukarıdaki örneğe benzer şekilde bütün matematiksel operatörler "=" eşittir simgesi ile birleştirilerek aşağıda gösterildiği gibi kullanılabilir.

Matematiksel işlemlerde kullanılan operatörlerin atama operatörleri ile birleşimi:

İşlem	Operatör	Kısa gösterim	Uzun gösterim (eşdeğeri)
Aktar/Ata	=	A = B = 3	A = 3 B = 3
Topla ve aktar	+=	A += B	A = A + B
Çıkar ve aktar	-=	A -= B	A = A - B
Çarp ve aktar	*=	A *= B	A = A * B
Böl ve aktar	/=	A /= B	A = A / B
Kalanı al ve aktar	%=	A %= B	A = A % B
Tamsayı böl ve aktar	//=	A //= B	A = A // B

Proje 3.4. İşlem Satırını Kısaltma

Aşağıdaki kod satırları daha kısa nasıl ifade edilebilir?

```
A = A * B
i = i + j * 8
x = x - y / 2
```

Çözüm:

```
A *= B
i += j * 8
x -= y / 2
```

Veri Tipleri

Her değişken içerisinde farklı tipte veriler saklanabilir. Veri tipi (type) bir değişkenin bellekte kapladığı alan ve alacağı değerlerin biçimi hakkında bilgi veren temel bir özelliktir.

Python dilindeki en temel veri tipleri şunlardır;

Veri tipi	Açıklama
<code>int</code>	Tamsayı veri tipidir.
<code>float</code>	Kesirli sayı veri tipidir.
<code>bool</code>	True (doğru) ya da False (yanlış) biçiminde değer tutan boolean veri tipidir.
<code>str</code>	Sözel ifadeleri tutan String veri tipidir. Yazılar/Metinler tek tırnakla ya da iki tırnakla belirtilebilirler. Str veri tipindeki veriler '+' operatörü ile birleştirilebilir.

```

>>> A = True    #Bu örnekte A değişkeni bool tipinde veri tutmaktadır.
>>> A = 'True'   #Bu örnekte A değişkeni str tipinde veri tutmaktadır.
>>> A = 5        #Bu örnekte A değişkeni int tipinde veri tutmaktadır.
>>> A = 5.0      #Bu örnekte A değişkeni float tipinde veri tutmaktadır.

```

Python dilinde her değişken, veri veya nesnenin bir veri tipi vardır. Bir değişkenin içerisinde hangi tip veri sakladığını öğrenmek için 'type' komutu kullanılır. Örnek;

```

>>> A=3
>>> B=4.5
>>> type(A)
<class 'int'>
>>> type(B)
<class 'float'>
>>> type("Ay")
<class 'str'>
>>> type(3<5)
<class 'bool'>

```

Veri Tipi Dönüşümü

Python dilinde tüm veri tipleri birbirine dönüştürülebilir. Sözel veriler, sayısal verilere, kesirli sayılar tamsayılara, tamsayılar da kesirli sayılara dönüştürülebilir. Bu işlem için tip dönüşümüne uğrayacak değişken, dönüştürülecek veri tipi sarmalına alınır. Örnek kullanım şekli aşağıdaki gibi olmalıdır;

```

(dönüştürülecek veri tipi) değişken;
>>> (int) (4.3) #Sonuç 4 olacaktır

```

Tip dönüşüm örnekleri;

```
>>> int (3.14)          >>>Tamsayıya dönüştürüldü.
3
>>> float(3)           >>>Kesirli sayıya dönüştürüldü.
3.0
>>> str(5) + str (5)   >>>Sözel (string) ifadeye dönüştürüldü.
'55'
>>> 5 + 5              >>>Herhangi bir dönüşüm yapılmadı.
10
```

Açıklama (Yorum) Satırları:

Programcılar yazdıkları programlara kod satırlarını açıklayıcı kısa notlar düşerler. Bu kısa notları, Python yorumlayıcısının komut olarak algılamaması için # (diyez) simgesi kullanılır.

açıklama (yorum) satırı simgesinden sonra yazılan ifadeleri Python yorumlayıcısı dikkate almaz. Böylece programcı Python derleyicisine, **#bu bir komut satırı değil, derleme!** demek ister.

- Eğer tek satırlık bir açıklama yapılacaksa '#' karakteri,
- Birden fazla satırlık açıklama yapılacaksa """ """ 3 çift tırnak veya '''... ''' 3 tek tırnak karakterleri kullanılır.

Proje 3.5.

Verilen bir para miktarındaki kaç çeyrek, onluk, beşlik ve birlik olduğunu hesaplayan basit bir programı açıklama satırları ile kodlayalım.

```
#Verilen Para miktarı:
Miktar = 69 #TL

ceyrek = Miktar // 25   #çeyrek hesabı
Miktar = Miktar % 25    #kalan
onluk = Miktar // 10    #onluk hesabı
Miktar = Miktar % 10    #kalan
beslik = Miktar // 5    #beşlik hesabı
Miktar = Miktar % 5     #birlik hesabı

#Değişken değerleri ekrana yazdırılıyor
print(ceyrek, "Çeyrek") #2 Çeyrek
print(onluk, "Onluk")   #1 Onluk
print(beslik, "Beşlik") #1 Beşlik
print(Miktar, "Birlik") #4 Birlik
```

Sıra Sende Etkinliği :

Ortalama hızı saatte 100 km/saat olan bir aracın N saat sonra kaç metre yol alacağını bulan programı kodlayınız. {N=5 alınacaktır}

Soru ve Cevaplarla Neler Anladık?

- 1 A, B ve C değişkenlerine "Python" verisini tek satırda aktarınız?

```
A = B = C = "Python"
```

- 2 Bir değişkenin tipini öğrenmek için hangi komut kullanılır?

```
type
```

- 3 Aşağıdaki değişken isimlerinden hangileri Python dilinde geçersizdir?

a. **as** b. **54goto** c. **not** d. **ad soy** e. **Foto** f. **False**

a, c, d, f

- 4 "i = i + j" kod satırı daha kısa nasıl kodlanabilir?

```
i += j
```

- 5 B ve C değişkenlerinin içeriklerini tek satırda yer değiştiriniz?

```
B, C = C, B
```

- 6 Aşağıdaki kod satırlarının çalıştırılmasından sonra x değişkeninin değeri ne olur?

```
t=22+25-15
```

```
x=t/4+t%4
```

8.0

- 7 Python komutları, değişken ismi olarak kullanılamaz. Python komutlarının güncel listesini hangi komut satırları ile öğrenebiliriz?

```
>>> import keyword
```

```
>>> keyword.kwlist
```

- 8 Aşağıdaki kod satırlarının çalıştırılmasından sonra B değişkeninin değeri ne olur?

```
B = 40
```

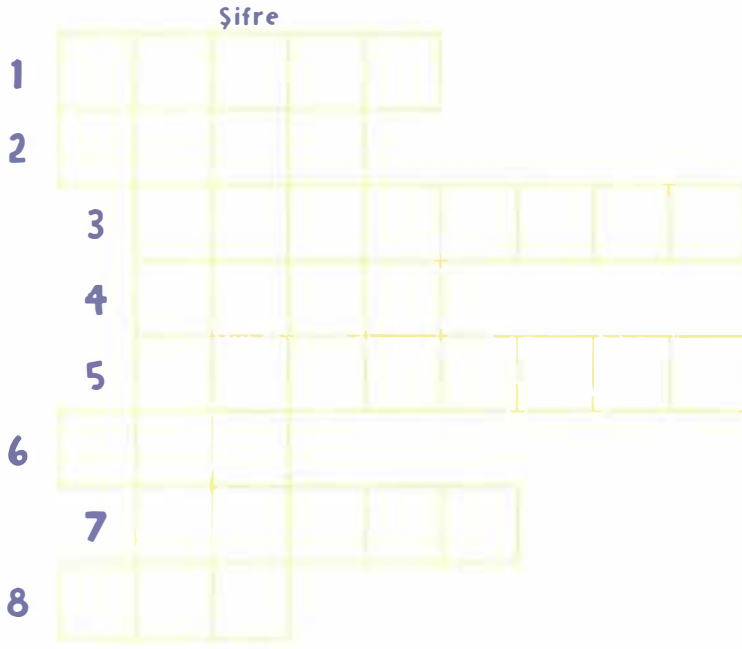
```
C = str(B) + "10"
```

4010

- 9 Python programında o kod satırının bir açıklama satırı olduğunu (yani bir komut satırı değil, derleme anlamında) vurgulamak/belirtmek için hangi simge kullanılır?

```
#
```

BULMACA



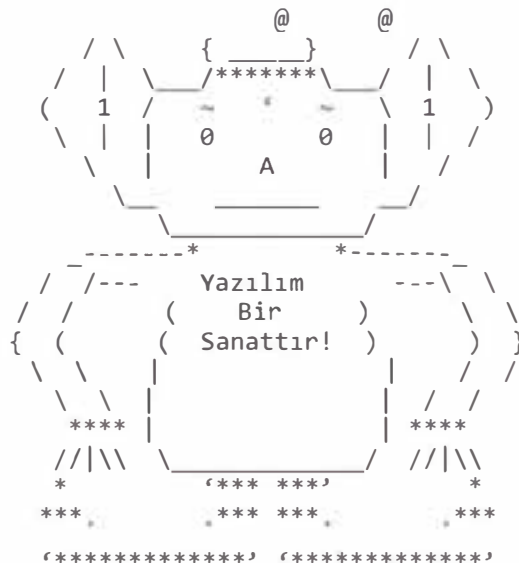
1. Kesirli sayı veri tipi.
2. Bir değişkenin veri tipini öğrenmek için kullanılır.
3. İçerisinde veri (değer) sakladığımız küçük hafıza (bellek) alanları.
4. Bool veri tipindeki bir değişkenin tuttuğu değerlerden biri.
5. Bir matematiksel işlemin önceliğini onun içine alarak değiştirebiliriz.
6. Tamsayı veri tipi.
7. "/" operatörü.
8. Sözel (string) veri tipi.

Aşağıya bulduğunuz şifreyi kodlayınız.

--	--	--	--	--	--	--	--	--	--

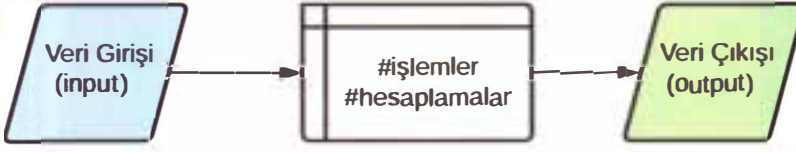
4

Veri Giriş-Çıkış İşlemleri



Giriş

Genel olarak her programın çalışma mantığı aşağıdaki gibi gerçekleşir;



Programa veriler girilir, gerekli işlem ve hesaplamalar yapıldıktan sonra ise istenen çıktı elde edilir.

Bu bölümde klavyeden/konsoldan veri girilmesi ve bu veriler üzerinde basit işlemler/hesaplamalar yapılması ve sonucun ekrana yazdırılması şeklindeki örneklerle yer verilecektir.

Veri Girişi Komutu: input()

Klavyeden (konsoldan) veri girişi için Python dilinde `input()` komutu kullanılır. Bu komut Türkçe "Gir" anlamındadır. Kullanım şekli: `değişken = input('mesaj')`

```
ad= input("Adınız..")
print ("Ad:", ad)
#Açıklama: Klavyeden 'Bülent' girilirse ekrana 'Ad: Bülent' mesajı yazılır.
```

Örnekteki gibi `input()` komutu ile alınan veri **sözel (string)** tipte kabul edilir. Eğer bu veri üzerinde sayısal (matematiksel) işlem yapılacak ise aşağıdaki tablodaki gibi istenen veri tipine (int, float) dönüşüm yapılması gerekir.

`input()` komutu kullanım örnekleri:

Girdi komutu	Açıklama
<code>a= input('Ad?')</code>	Klavyeden girilen veri, sözel (str) tipindeki 'a' değişkeninde tutulur. Varsayılan tip değeri str'dir.
<code>b= int(input('Numara?'))</code>	Klavyeden girilen veri, int (tamsayı) tipine dönüştürülerek 'b' değişkeninde tutulur.
<code>c= float(input('Maas?'))</code>	Bu kullanımda klavyeden girilen veri, float (kesirli sayı) tipine dönüştürülerek 'c' değişkeninde tutulur.
<code>hata = input("Ad')</code>	Hatalı kullanım örneği: "çift tırnak ile başlandı ise çift tırnakla bitirilmelidir". "... ' veya '..." şeklindeki kullanımlar hatalıdır.

Örneklerden görüldüğü üzere `input()` komutu parantezleri arasında 'tek' veya "çift" tırnaklar kullanılabilir. Parantezler arasına "girilmek istenen veriyi açıklayıcı mesajlar" yazılır.

Proje 4.1. Girilen Sayıların Toplamı

İki sayının toplamını hesaplayan basit matematiksel işlemi yapalım. Programın istenen çıktısı aşağıdaki gibi olmalıdır:

```
1.sayıyı gir:5
2.sayıyı gir:7
Toplam= 12
```

Algoritması	Python dili kodlaması
1. Başla 2. Gir, iki sayı (A, B) 3. Topla, girilen sayıları ($T = A + B$) 4. Yaz, Toplam değeri (T) 5. Dur	<pre>A = int (input ("1.sayıyı gir:")) B = int (input ("2.sayıyı gir:")) T = A + B print ("Toplam= ", T)</pre>

Açıklama: Bu örneğimizde; A, B ve T birer değişkendir. =, + simgeleri ise birer operatördür. 'Gir' ve 'Yaz' ise birer komuttur. Bu komutların Python dilindeki eşdeğerleri `input` ve `print` komutlarıdır.

Soru: Eğer bu programda `input()` ile alınan değerler 'int' sarmalına alınarak tip dönüşümü yapılmasa idi, program nasıl etkilenirdi?

Cevap: Aşağıdaki ekran çıktısında da görüldüğü gibi tip dönüşümü yapılmadan `input()` komutu ile alınan veri, sözel (string) kabul edildiği için A ve B değerleri birleştirilir, toplanmaz. Kısaca '5+7=12' yerine '5' + '7' = '57' sonucu görülür.

Python kodlaması (Tip dönüşümü yapılmamış)	Ekran Çıktısı
<pre>A = input ("1.sayıyı gir:") B = input ("2.sayıyı gir:") T = A + B print ("Toplam= ", T)</pre>	<pre>1.sayıyı gir: 2.sayıyı gir: Toplam= 57</pre>

Veri Çıkışı Komutu: print()

Veri çıkışı için Python dilinde `print()` komutu kullanılır. Bu komut Türkçe "Yaz" anlamındadır. Kullanım şekli: `print('çıkıttı mesajı')`

Veri çıkış işlemi, verileri ya bilgisayar ekranına ya da bilgisayardaki herhangi bir dosyaya yazdırma şeklinde gerçekleşir. Aynı komutu örneklerinde olduğu gibi `print("çıkıttı mesajı")` içerisindeki çıkıttı mesajı 'tek' veya "çift" tırnak arasına alınabilir, fark etmez.

Not. Genellikle 'tek' veya "çift" tırnak arasındaki ifadeler, programı kullanan kişiye bir mesaj vermek, onu yönlendirmek amacı ile kullanılırlar.

print() Komutu Parametreleri

Ekran çıktısının biçimini/formatını farklı şekillerde ayarlamak için `print()` içerisinde bazı parametreler kullanılır. Bunlar;

`sep=` : Çıkıttı mesajındaki karakterler arasına ayraç koymak için kullanılır.

`end=` : Çıkıttı mesajı sonunu düzenlemek için kullanılır.

`\n` : Çıkıttı mesajındaki istenen karakterleri bir alt satıra (new line) basmak için kullanılan bir çıkış (escape) karakteridir.

`\t` : Çıkıttı mesajındaki karakterler arasına bir tab boşluk bırakmak için kullanılan bir çıkış (escape) karakteridir.

`\\` : Ters taksim (backslash) "\\" karakterini ekrana basar. Ters taksim "\" işareti çıkış mesajı formatını düzenlemede kullanıldığı için tek başına kullanılmaz.

Aşağıdaki tüm örnekler dikkatlice incelenirse; `print` komutunun çok zengin çıkış alternatifleri sunduğu görülecektir.

`print()` komutu kullanım örnekleri:

print (Çıkıttı) komutu kullanım şekilleri	Açıklama ve ekran çıktısı
<code>print("python")</code>	" " arasındaki mesaj ekrana yazdırılır.
<code>print('p','i','t','o','n')</code>	Her bir karakter arasına boşluk ' ' bırakır. Ekran Çıktısı: p i t o n

<code>print('p','i','t','o','n',sep=',')</code>	sep (ayraç) fonksiyonu ekrana basılacak her değerin arasına ',' ile belirtilen karakteri koyar. Ekran Çıktısı: p,i,t,o,n
<code>print('p','i','t','o','n',sep='-')</code>	Ekran Çıktısı: p-i-t-o-n
<code>a=5 print(a)</code>	#a tamsayı tipinde bir değişkendir. Ekran çıktısı: 5
<code>a=3, b=5 print(a,'+',b,'=', a+b)</code>	Ekran çıktısı: 3 + 5 = 8
<code>print("piton " * 4)</code>	#Yanyana 4 kez 'piton' yazar. Ekran çıktısı: piton piton piton piton
<code>print("Ad.:") ad=input() print("Ad.:", end=' ') ad=input()</code>	Ad.: Ali Ad. : Ali #gibi mesaj yanına bir isim girilmesini bekler.
<code>print("piton", end='?')</code>	Mesaj sonuna '?' karakterini ekler. Ekran çıktısı: piton?
<code>print("Can\nEvi")</code>	#Alt satıra geçerek yazar. Ekran Çıktısı: Can Evi
<code>print("Can\tEvi")</code>	#Bir tab boşluk bırakarak yazar. Ekran Çıktısı: Can Evi
<code>print ("\\")</code>	#Tek bir ters taksim '\\' işaretini yazar. Ekran çıktısı: \
<code>print("Can dedi: 'Haydi gel!'")</code>	Ekran çıktısı: Can dedi: 'Haydi gel!'

format() Kullanımı

Bir sözel ifade içerisine veya print() içerisindeki mesaja değişken değerini yazdırmak için **format()** (veya kısaca **f**) komutu kullanılabilir. Aşağıdaki dört kullanım şekli de aynı sonucu üretir;

```
A = 3
1. print('A =',A)
2. print("A = {0}".format(A))
3. print(f"A = {A}")
4. print('A = ' + str(A))
```

Kesirli (virgüllü) sayıların virgülden sonraki basamak sayısını düzenlemek için `'f'` kullanılabilir. Örnek kullanım;

```
PI = 3.14159
#'.2f' ile PI sayısının virgülden sonraki 2 basamağı gösterilir;
print (f"PI = {PI:.2f}")    #Ekran çıktısı: PI =3.14
```

Proje 4.2. print() ile Çizim

a. Aşağıdaki balık resmini çizen kodu print() komutu ile tek satırda kodlayalım.

```
 /_ |
( ' )<|
 \_ |
```

Çözüm:

```
print (" /_ |\\n( ' )<|\\n \\_ |")
```

b. Aşağıdaki balık resmini çizen kodu print() komutu ile tek satırda kodlayalım.

```
 /
 /\
 \\
  \
```

Çözüm:

```
print (" /\n\\/\n\\/\n \\\n")
```

Not. `print ("...\n")` şeklinde print fonksiyonunu tek `'\n'` ile sonlandırma EOL (satır sonu) hatasına sebep olur. Bu sebeple en sondaki `'\n'` karakterini ekrana bastırmak için programda çift `'\\n'` ters taksim (İngilizce ismi: slash) karakteri kullanıldı.

Sıra Sende Etkinliği

Aşağıdaki balık resmini çizdirecek kodu tek satırda kodlayabilir misin?

```
 /_ |
( ' )<|
 \_ |
```

print() Kullanımında Sık Karşılaşılan Bazı Hatalar Ve Çözümü

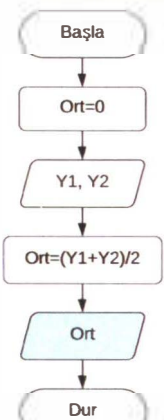
Hatalı kod	Doğrusu
<pre>>>> print ("piton")</pre> SyntaxError: EOL while scanning string literal	<pre>print ("piton")</pre> #Çift tırnak kapanmamış
<pre>>>> print (" /\n<><\n \")</pre> SyntaxError: EOL while scanning string literal	<pre>print (" /\n<><\n \\\")</pre> #Çıktı mesajı tek '\\' ters taksim işareti ile sonlanamaz.
<pre>>>> print 'piton'</pre> SyntaxError: Missing parentheses in call to 'print'.	<pre>print ("piton")</pre> #Mesaj parantez içine alınmamış
<pre>>>> a=3 >>> print ("a^2=" *a)</pre> SyntaxError: invalid syntax	<pre>a=3 print ("a^2=", a*a)</pre> #Arada virgül unutulmuş.

Veri Giriş-Çıkış Uygulamaları

Proje 4.3. Ders Başarı Ortalaması

1. ve 2. yazılı sınav notlarına göre bir öğrencinin ders başarı ortalamasını hesaplayan programı kodlayalım.

Açıklama: Bir dersin başarı ortalaması yazılıların (sınavların) toplamının ikiye bölümü alınarak hesaplanacaktır.

Algoritması	Python kodlaması
 <pre>graph TD A([Başla]) --> B[Ort=0] B --> C[/Y1, Y2/] C --> D[Ort=(Y1+Y2)/2] D --> E[/Ort/] E --> F([Dur])</pre>	<pre>Ort= 0 Y1 = int(input("1.yazılı:")) Y2 = int(input("2.yazılı:")) Ort = (Y1 + Y2) / 2 print ("Ortalama:", Ort)</pre> Programın Örnek Ekran Çıktısı: <pre>1.yazılı.:75 2.yazılı.:65 Ortalama= 70.0</pre>

Proje 4.4. Yaş Hesabı

2031 yılında kaç yaşında olduğunuzu hesaplayan programı kodlayalım.

Çözüm: Öncelikle doğum tarihinin (dT) girilmesi ve 2031 yılından bu tarihin çıkarılması gerekir.

Algoritması	Python kodlaması
	<pre>dT = int(input("Doğum tarihiniz.:")) Yas = 2031 - dT print ("Yaşınız.: ", Yas)</pre> Programın Örnek Ekran Çıktısı: <pre>Doğum tarihiniz.: 2012 Yaşınız.: 19</pre>

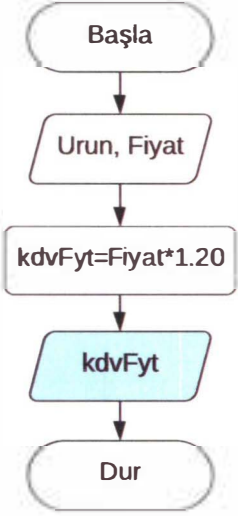
Proje 4.5. KDV Hesabı

Fiyatı ve ismi girilen bir ürünün KDV dâhil fiyatını hesaplayan programı kodlayalım. KDV oranı %20 alınacaktır. Programın istenen çıktısı aşağıdaki gibi olmalıdır:

```
Ürünün Adı.: Python
Ürünün Fiyatı.:100
Ürün adı.: Python
KDV'li Fiyatı.: 120.0 TL
```

Çözüm: Bir ürünün KDV dahil fiyatı `kdvFyt` isimli değişkende tutulmakta ve aşağıdaki gibi hesaplanmaktadır:

```
kdvFyt = Fiyat * (1 + KDV Oranı /100)
        = Fiyat * (1 + 20/100)
        = Fiyat * 1.20
```

Algoritması	Python kodlaması
 <pre> graph TD A([Başla]) --> B[/Urun, Fiyat/] B --> C[kdvFyt=Fiyat*1.20] C --> D[/kdvFyt/] D --> E([Dur]) </pre>	<pre> Urun = input("Ürünün Adı.:") Fiyat= float(input("Ürünün Fiyatı.:")) kdvFyt = Fiyat*1.20 print ("Ürün adı:", Urun) print ("KDV'li Fiyatı.:", kdvFyt, "TL") </pre>

Soru ve Cevaplarla Neler Anladık?

- 1 input() fonksiyonu varsayılan (default) olarak hangi tipte veri alır?

`str`

- 2 Aşağıdaki komut satırının ekran çıktısı ne olur?

```
print (" o\n()<\n o")
o
()<
o
```

- 3 Aşağıdaki komut satırının ekran çıktısı ne olur?

```
print("ab\\c")
ab\c
```

- 4 Aşağıdaki programın klavyeden girilen adı göstermesi için ii. satırda boş bırakılan alana ne yazılmalıdır?

```
i. Ad = input('Ad.:')
ii. print("Girilen.: ", _____)
Ad
```

- 5 Aşağıdaki komut satırlarının ekran çıktısı ne olur?

```
print("pisi *3")
print("pisi ``*3")
#tırnak işaretlerinin yerine dikkat ediniz
pisi *3
pisi pisi pisi
```

- 6 Aşağıdaki programın a ve b sayılarını toplaması gerekmektedir. Bu programın doğru çıktıyı vermesi için ne yapılmalıdır?

```
1 a=input("sayi1.:")          <terminated>
2 b=input("sayi2.:")          sayi1.:5
3 print(a+b)                  sayi2.:4
                               54
```

a ve b değişkenleri int veri tipine dönüştürülmelidir.

- 7 `print('p','i','t','o','n',sep='>')` komut satırının çıktısı ne olur?

`p>i>t>o>n`

- 8 `print()` komutu çıktı mesajındaki karakterler arasına bir tab boşluk bırakmak için hangi çıkış (escape) karakteri kullanılır?

`\t`

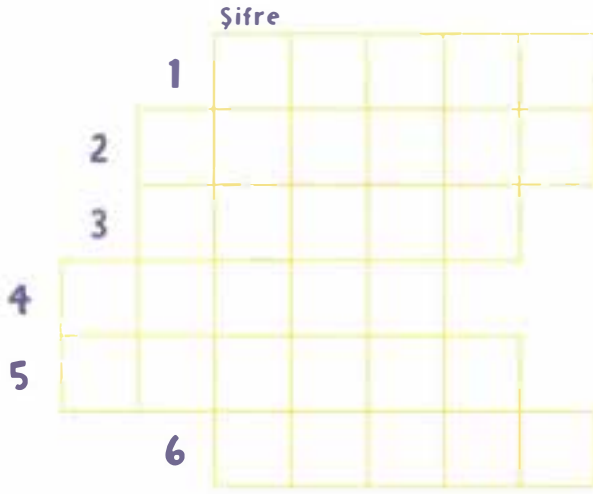
- 9 KDV dahil fiyatı 360 TL olan bir malın KDV siz fiyatını ekranda gösteren aşağıdaki programda eksik satıra ne gelmelidir?
(KDV oranı: %18 alınmıştır)

```
fyf = 360/1.18
```

```
print ("KDV'siz Fiyatı.:", _____)
```

```
fyf
```

BULMACA



1. `float(input("Fiyatı:"))` komut satırı hangi tipte veri alır.
2. `print ('KOD' * 2)` komutunun sonucu.
3. "Yaz" komutunun Python dilindeki karşılığı.
4. `print('CEM' + 'AL')` komutunun sonucu.
5. (Tersi) "Çıkış karakterleri"nin İngilizcesi.
6. (Tersi) "Gir" komutunun Python dilindeki karşılığı.

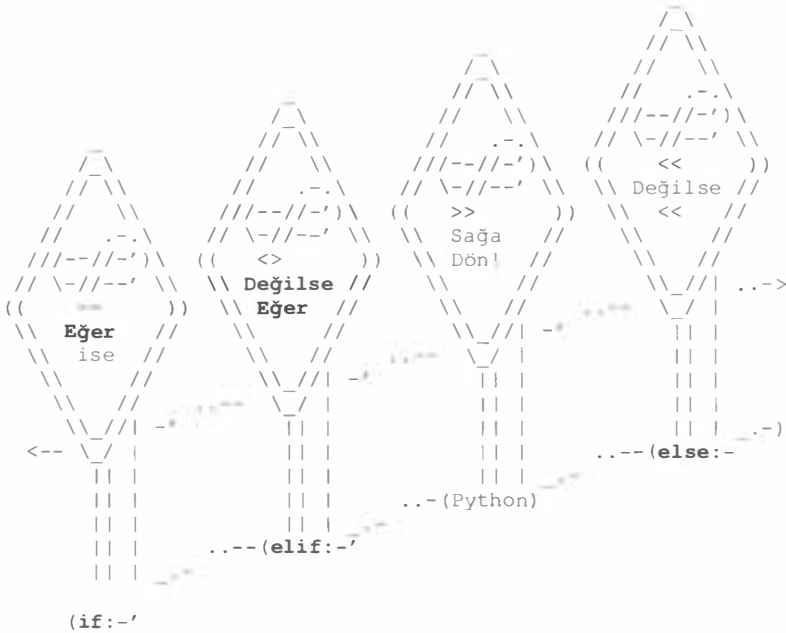
Aşağıya bulduğunuz şifreyi kodlayınız.

--	--	--	--	--	--	--

Bölüm

5

Koşul İfadeleri (if/else) ve Karşılaştırma Operatörleri

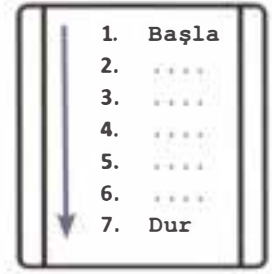


Koşullara verdiğimiz cevaplar, yolumuzu belirler.

Giriş

Normal bir programın çalışması, ilk satırdan son satıra doğru sırası ile adım adım gerçekleşir. Fakat bu çalışma sırası, bir koşula bağlı olarak değiştirilebilir.

Çoğu zaman bir programın bir veya daha fazla koşula (şarta) bağlı olarak farklı işlemleri yerine getirmesi gerekebilir. Program; koşula bakarak hangi işlemi yapacağına karar verir.



Python dilinde;

- Tek koşula bağlı olarak gerçekleşecek işlemler için; if
- Çift koşula bağlı olarak gerçekleşecek farklı işlemler için; if else
- Çoklu koşula bağlı olarak çok farklı işlemler için ise if-elif else

koşul ifadeleri kullanılır.

Koşullu ifadelerin koşul kısmında karşılaştırma operatörleri (<,>, ==, !=,<=,>=) kullanılır. Eğer koşul kısmında ikiden fazla değer karşılaştırması yapılacaksa aralarında mantıksal operatörler (and, or) kullanılabilir. Öncelikle bu operatörleri tanıyalım.

Karşılaştırma Operatörleri

Koşul içerisindeki verilerin birbiriyle karşılaştırılmasında kullanılır. Bu operatörler, sonuç doğru (geçerli) ise **True**, yanlış (geçersiz) ise **False** değerini üretir. Bir karşılaştırmının koşulun söz konusu olduğu koşul ifadelerinde, döngülerde bu karşılaştırma operatörlerinden en az biri mutlaka kullanılır.

Karşılaştırma operatörleri:

İşlem	Python eşdeğeri	Örnek kullanım	Sonuç
Büyük mü?	>	5 > 3	True
Küçük mü?	<	5 > 3	False
Büyük Eşit mi?	>=	5 >= 3	True
Küçük Eşit mi?	<=	3 <= 5	True
Eşit mi?	==	5 == 5	True
Eşit değil mi?	!=	5 != 5	False

Not. Programlamaya yeni başlayanlar "=" ile "==" operatörlerini sıklıkla birbirine karıştırabilmektedir.

= operatörü ; eşitliğin sağ tarafındaki ifadeyi (değeri) sol taraftaki değişkene aktarır.

== operatörü ; iki değer birbirine "eşit mi?" sorgulamasını yapar.

Örneğin, a=4 ile a sayısına 4 değeri aktarılırken, a==4 ile "a sayısı 4'e eşit mi?" sorgulaması yapılır.

Mantıksal Operatörler

Mantıksal operatörler, ikiden fazla verinin birbiri ile karşılaştırmasında kullanılır. Mantıksal operatörlerin sonucu, bool tipi değişkenlerde tutulur ve sonuç olarak sadece birbirinin tersi olan iki değerden True (Doğru) veya False (Yanlış) biri üretilir.

Mantıksal operatörler:

İşlem	Operatör	Örnek kullanım	Ekran çıktısı
Mantıksal VE	and	(3<5) and (7<3)	False
Mantıksal VEYA	or	(3<5) or (7<3)	True
DEĞİL	not	not (3<5)	False

Mantıksal operatörlerin işlevleri aşağıdaki gibi açıklanabilir;

- **and (VE) operatörü:** Koşul kısmında karşılaştırılan ifadelerden her ikisinin de doğru olması halinde True sonucunu aksi takdirde False sonucunu üretir. Çalışma mantığı tabloda gösterildiği gibidir:

A	B	A and B
True	True	True
True	False	False
False	True	False
False	False	False

- **or (VEYA) operatörü:** Koşul kısmında karşılaştırılan ifadelerden herhangi birinin doğru olması halinde True sonucunu aksi takdirde False sonucunu üretir. Çalışma mantığı tabloda gösterildiği gibidir:

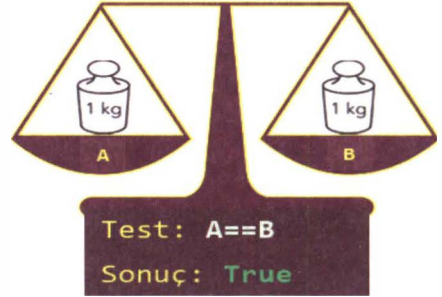
A	B	A or B
True	True	True
True	False	True
False	True	True
False	False	False

• **not (DEĞİL) operatörü:** Koşulun tersini (değilini) alır. Örneğin; not(True)= False, not(False)= True sonuçlarını üretir. Çalışma mantığı tabloda gösterildiği gibidir:

A	not A
True	False
False	True

bool İfadelerin Sonuçları

Karşılaştırma (<,>, ==, !=,<=,>=) veya mantıksal (and, or, not) operatörlerin hepsi sonuç olarak bool veri tipinde değer üretirler. Yani **True** ya da **False** değerlerinden birini üretirler.

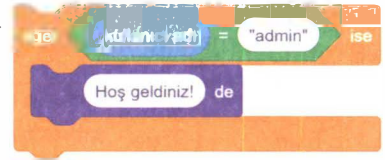


Not. **A==B** işlemi; "A değişkeninin değeri B değişkenin değerine eşit mi?" veya "A is B mi?" anlamına gelmektedir.

if Komutu

if, koşula (şarta) bağlı olarak tek bir işlemi gerçekleştirmek için kullanılır.

- Eğer yeşil ışık yanıyor ise geç.
- Eğer karanlık ise ışığı aç.
- Eğer (kullanıcı adı= "admin") ise "Hoş geldiniz" de.



Örneklerdeki gibi mevcut durumu (devam eden bir işlemi) koşula bağlı olarak değiştirirken "if" komutu kullanılır.

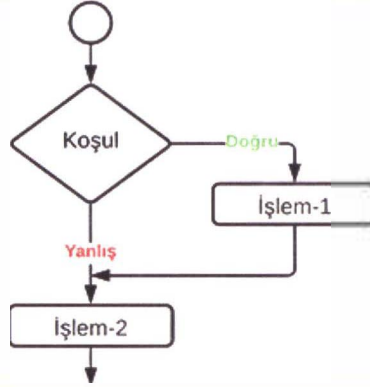
"if" deyimi Türkçe "eğer" anlamında kullanılan bir komuttur. Aşağıdaki if yapısında "eğer koşul geçerli (doğru - True)" ise "işlem-1" yapılır. Doğru değilse herhangi bir işlem yapılmaz ve program işleyişi bir alt satırdan "işlem-2"den devam eder.

Kullanım şekli

```

if koşul:
    İşlem-1
    İşlem-2

```

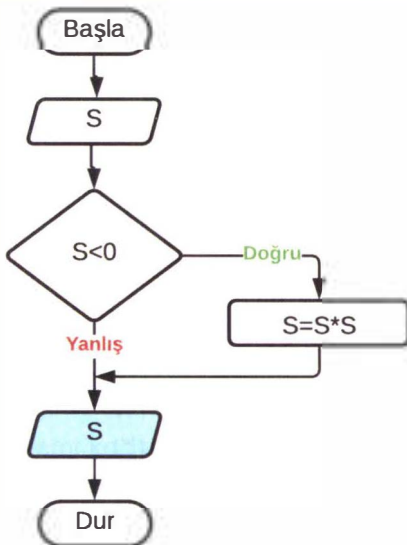
Algoritması

Not. if sorgularında koşuldan sonra ":" koymayı unutmayınız ve büyük / küçük harf ayırımına (yani If veya IF yazmamaya) dikkat ediniz.

Proje 5.1. Negatiflerin Karesi

Eğer girilen sayı sıfırdan küçükse karesini, değilse kendisini ekranda gösteren programı kodlayalım.

Çözüm: Program, normalde girilen sayıları olduğu gibi ekranda göstermektedir. Fakat sayı negatif girilirse karesini göstermektedir.

Algoritması**Python dili kodlaması**

```

S = int (input('Gir bir sayı.:'))
if S<0:
    S=S*S
print (S)

```

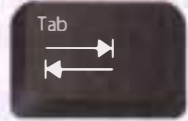
Python Kod Bloğu Yapısı ve Girinti Mekanizması

Çoğu programlama dillerinde kod bloklarını belirlemek için {...} süslü parantezler kullanılırken Python dilinde girinti (indentation) mekanizmasından yararlanılır.

Python dilinde dikey olarak aynı hizada başlayan ardışık kod satırları bir blok kabul edilir. Pratikte her bir blok girintisi için 4 boşluk (1 tab) karakteri bırakılması tavsiye edilir.

Doğru girinti kullanımı	Hatalı girinti kullanımı
<pre>1 S = -5 2 if S<0: 3 print("Negative") 4 if S>0: 5 print("Positive") 6 if S==0: 7 print("Zero")</pre>	<pre>1 S = -5 2 if S<0: 3 print("Negative") 4 if S>0: 5 print("Positive") 6 if S==0: 7 print("Zero") 8</pre> Bad Indentation (2 spaces)
Doğru: Hepsinde aynı (4) boşluk bırakılmış.	Yanlış: 3. Satırda girinti için 1 boşluk, 7. Satırda ise 2 boşluk bırakılmış

Not: Aynı hizada girinti bırakmak için klavyedeki Tab tuşuna 1 defa basmak yeterlidir.



Proje 5.2. Sıfır mı, Negatif mi, Pozitif mi?

Girilen bir sayının sıfır mı, negatif mi veya pozitif mi olduğunu ekranda gösteren programı kodlayalım. Programın örnek ekran çıktısı:

Gir bir sayı.: 0

Sayı Pozitif

Algoritması	Python dili kodlaması
1. Başla 2. Gir bir sayı (S) 3. Eğer (S<0) ise Yaz "Sayı Negatif" 4. Eğer (S>0) ise Yaz "Sayı Pozitif" 5. Eğer (S==0) ise Yaz "Sayı Sıfır" 6. Dur	<pre>S = int(input('Gir bir sayı.:')) if S<0: print("Sayı Negatif") if S>0: print("Sayı Pozitif") if S==0: print("Sayı Sıfır")</pre>

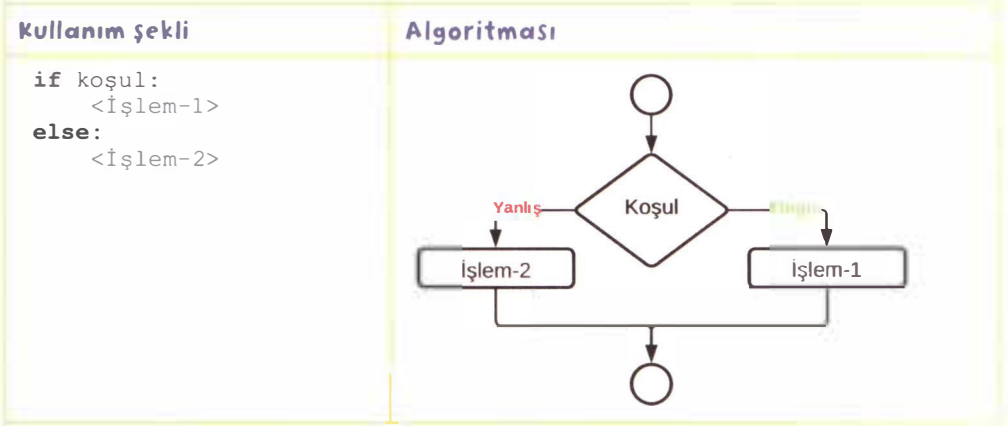
if - else Komutu

"if - else" deyimi Türkçe "eğer-değilse" anlamında kullanılan bir yapıdır. Koşula bağlı olarak iki farklı işlemden birini gerçekleştiren yapıdır. Tüm programlama dillerinde olduğu gibi Python dilinde bu işlem için if / else deyimi kullanılır.

- Eğer ortalama > 50 ise "Geçtiniz" değilse "Kaldınız" de.
- Eğer Hava==Güneşli ise "Top oynayalım" değilse "TV izleyelim"



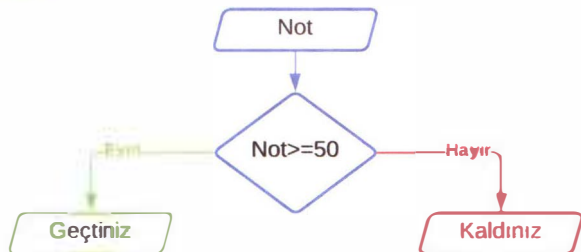
Şeklindeki ifadelerde bir koşula bağlı olarak iki işlemden biri gerçekleşir. Koşul geçerli/doğru (True) ise if (eğer) bloğundaki işlem: <işlem-1>, geçersiz/yanlış (False) ise else (değilse) bloğundaki işlem: <işlem-2> yapılır.



Not. Hem if koşul: hem de else: den sonra ":" koymayı unutmayınız!

Proje 5.3. Geçti mi, Kaldı mı?

Girilen not değerine göre öğrencinin o dersten geçip geçmediğini ekranda gösteren programı kodlayalım.



Algoritması

1. Başla
2. Gir, öğrencinin notunu (Not)
3. Eğer Not ≥ 50 ise
Yaz "Geçtiniz"
- Değilse
Yaz "Kaldınız"
4. Dur

Python dili kodlaması

```
Not=int(input("Öğrencinin Notu.:"))  
if Not>= 50:  
    print ("Geçtiniz")  
else:  
    print ("Kaldınız!")
```

Açıklama: "Not" değişkenini "not" şeklinde kullanırsanız hata alırsınız.

Çünkü "not" "değil" anlamında kullanılan bir Python komutudur.

Proje 5.4. Kullanıcı Adı ve Parola Sorgulama

Girilen kullanıcı adı (user) ve parola (password) bilgilerine göre "Hoş geldiniz!" değilse "hatalı kullanıcı adı veya parola" girişi mesajı veren programı kodlayalım. Programın istenen çıktısı:

```
Kullanıcı Adı.:admin  
Parola.:12345  
Hatalı kullanıcı adı veya parola girişi
```

Çözüm: Programda hem kullanıcı adı (user) hem de parola (password) 'admin' olarak ayarlanmıştır.

Algoritması

1. Başla
2. Gir, kullanıcı adını (user)
3. Gir, parolayı (password)
4. Eğer user == 'admin' VE password== 'admin' ise
Yaz "Hoş geldiniz!"
- Değilse
Yaz "Hatalı kullanıcı adı veya parola"
5. Dur

Python dili kodlaması

```
user = input('Kullanıcı Adı.:')  
password = input('Parola.:')  
if user == 'admin' and password=='admin':  
    print ("Hoş geldiniz!")  
else:  
    print ("Hatalı kullanıcı adı veya parola girişi")
```

Proje 5.5. Sayı Doğrusu

X sayısının alabileceği değerler (çözüm) kümesi sayı doğrusu üzerinde gösterilmiştir.



$$12 \leq x < 15$$

Buna göre girilen X değeri bu aralıkta ise "X sayı doğrusu üzerinde" değilse "X sayı doğrusu üzerinde değil" mesajını ekranda gösteren programı kodlayalım.

Algoritması	Python dili kodlaması
1. Başla 2. Gir, X değerini (x) 3. Eğer (12<=x<15) ise Yaz "Sayı doğrusu üzerinde" Değilse Yaz "Sayı doğrusu üzerinde değil" 4. Dur	<pre>x=int(input("X değeri. :")) if (12<=x<15): print ("Sayı doğrusu üzerinde") else: print ("Sayı doğrusu üzerinde değil")</pre>
(12<=x<15) ifadesinin açılımı: (12<=x) and (x<15) dir.	

Sıra Sende Etkinliği - 1

Bir sayının tek mi çift mi olduğunu bulan programın algoritması verilmiştir. Verilen algoritmaya göre Python dili kodlamasını da sen yapabilir misin?

Algoritması	Python dili kodlaması
1. Başla 2. Gir, A değerini 3. Eğer (A%2 == 0) ise Yaz "Sayı Çifttir" Değilse Yaz "Yaz Tektir" 4. Dur #(A%2):A'nın 2'ye bölümünden kalanı verir	

if - elif Komutu

Üç ve daha fazla koşul durumunda if ve elif komutları merdiven şeklinde alt alta kullanılır. elif: "değilse eğer" anlamında kullanılan bir komuttur.

Kullanım şekli	Algoritması
<pre>if koşul1: <İşlem-1> elif: <İşlem-2> elif: <İşlem-3> else: <İşlem-N></pre>	<pre>graph TD Start(()) --> K1{Koşul-1} K1 -- Doğru --> I1[İşlem-1] K1 -- Yanlış --> K2{Koşul-2} K2 -- Doğru --> I2[İşlem-2] K2 -- Yanlış --> K3{Koşul-3} K3 -- Doğru --> I3[İşlem-3] K3 -- Yanlış --> I4[İşlem-4] I1 --> Join(()) I2 --> Join I3 --> Join I4 --> Join Join --> End(())</pre>

Proje 5.6. Park Ücreti

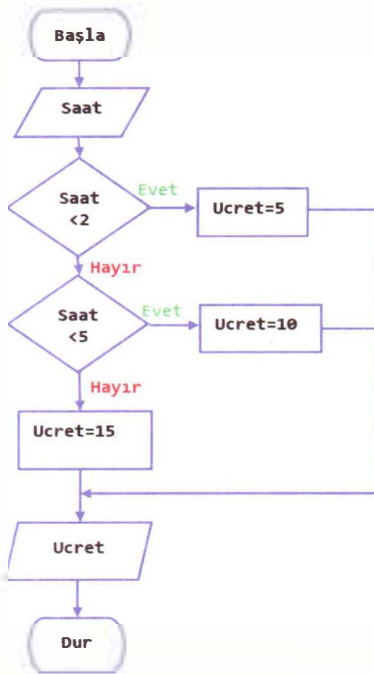
Yanda verilen ücret tarifesine göre otoparkta kalan araçlardan alınacak park ücretini hesaplayan programı kodlayalım.

Çözüm:

Programda saat aralığı: "Saat", park ücreti: 'Ucret' isimli değişkenler ile ifade edilmiştir.

Saat Aralığı (Saat)	Ücret (TL)
0-2 saat arası	5 TL
2-5 saat arası	10 TL
5-24 saat arası	15 TL

Akış Şeması (algoritması)



Python dili kodlaması

```
Saat = float(input('Kalma Süresi.: '))
if Saat < 2:
    Ucret = 5
elif Saat < 5:
    Ucret = 10
else:
    Ucret = 15
print ("Park Ücreti.:", Ucret, "TL")
```

Proje 5.7. Başarı Notunu Harfli Sisteme

Dönüştürme

100'lük sisteme göre girilen başarı notunu yanda verilen tabloya göre harfli sisteme çeviren programı kodlayalım.

Not Aralığı	Harf Notu
00 - 49	F
50 - 59	E
60 - 69	D
70 - 79	C
80 - 89	B
90 - 100	A

Programın örnek ekran çıktısı:

Gir Notu.: 55

Ders Notu.: E

Algoritması

1. Başla
2. Gir başarı notunu (Nt)
3. Eğer notunuz 90 ve üzeri ise
Yaz "A"
4. Değilse Eğer notunuz 80 VE 89 arası ise
Yaz "B"
5. Değilse Eğer notunuz 70 VE 79 arası ise
Yaz "C"
6. Değilse Eğer notunuz 60 VE 69 arası ise
Yaz "D"
7. Değilse Eğer notunuz 50 VE 59 arası ise
Yaz "E"
8. Değilse Eğer notunuz 0 VE 49 arası ise
Yaz "F"
9. Değilse
Yaz "Lütfen 0-100 arası not giriniz"
10. Dur

Python dili kodlaması

```
Nt=int(input('Gir Notu.: '))
print ("Ders Notu.:")
if Nt >= 90 and Nt<=100:
    print ("A")
elif Nt >= 80 and Nt<=89:
    print ("B")
elif Nt >= 70 and Nt<=79:
    print ("C")
elif Nt >= 60 and Nt<=69:
    print ("D")
elif Nt >= 50 and Nt<=59:
    print ("E")
elif Nt >= 0 and Nt<=49:
    print ("F")
else:
    print ("Lütfen 0-100 arası not giriniz!")
```

Proje 5.8 TCDD İndirimli Bilet Fiyatlandırması

Yaş	İndirim Oranı
0 - 6 arası	Ücretsiz
7 – 12 arası	%50
13 – 26 arası	%20
27 - 59	%0 (indirim yok)
60 – 64	%20
65 ve üzeri	%50

TCDD trenlerinde bilet fiyatında yaşa göre yapılacak indirimler yukarıda verilen tabloya göre hesaplanmaktadır. Girilen yaş değerine göre indirimli bilet fiyatını gösteren programı kodlayalım.

Python dili kodlaması	Programın ekran çıktısı
<pre> Fiyat = 100 yas = int (input('Yaşınız:')) print ("Normal Bilet Fiyatı: ", Fiyat) if yas>0 and yas<=6: iFyt = 0 #ücretsiz elif yas>=7 and yas<=12: iFyt=Fiyat/2 # 50 indirim elif yas>=13 and yas<=26: iFyt=Fiyat- Fiyat*0.20 elif yas>=27 and yas<=59: iFyt=Fiyat #indirim yok elif yas>=60 and yas<=64: iFyt=Fiyat- Fiyat*0.20 elif yas>=65: iFyt=Fiyat/2 #50 indirim print ("İndirimli Bilet Fiyatı: ", iFyt) </pre>	<pre> Yaşınız.:60 Normal Bilet Fiyatı.: 100 İndirimli Bilet Fiyatı.: 80.0 </pre>

Karar Ağacı (Bir Yapay Zekâ) Oyunu

Karar ağacı oyununda doğru cevaba ulaşmak için; yukarıdan aşağıya doğru verilen cevapları izlemek yeterlidir. Bunun için de iç içe if-elif sorgusu yapılarak doğru cevaba ulaşmaya çalışılır.

Oyunun Amacı

Kaliteli (iyi) sorular sorabilmektir. İyi sorular sizi doğru cevaba daha fazla yaklaştıran sorulardır.

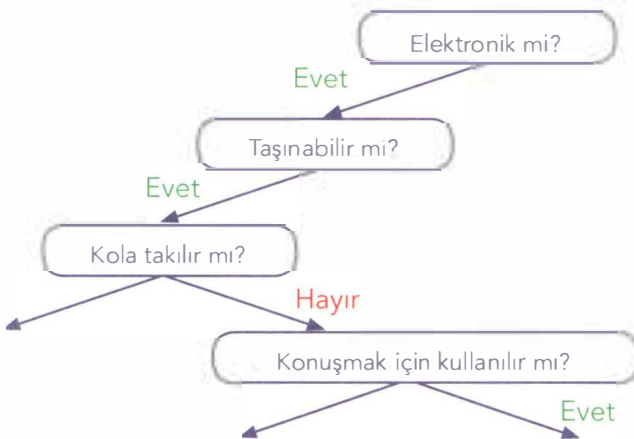
Nasıl Oynanır?

Bu oyunda oyunculardan biri (sen) aklından bir nesne tutar ve diğer oyuncu da (bilgisayar) ona en fazla 20 adet "Evet-Hayır" cevabı verilebilecek sorular sorarak o nesneyi bulmaya çalışır.

Proje 5.9. Karar Ağacı

"Telefon, radyo, TV, kamera, akıllı saat" gibi bir elektronik eşyayı aklımızdan tutalım. Bilgisayara bu elektronik nesneyi, aşağıdakine benzer bir karar ağacını kodlayarak buldurmaya çalışalım.

Bakalım bilgisayar tutulana bilebilecek mi? Programda kullanıcının evet cevabı için kısaca "E", hayır cevabı için kısaca "H" demesi yeterlidir.



Tabii, karar ağacındaki if: else: durumları daha da uzatılabilir. Biz kod satırları uzamaması için biraz kısalttık.

Python dili kodlaması

```

print ("Karar Ağacı Oyunu:\n Aklından elektronik bir eşya tut!")
cvp = input ("Elektronik mi? [E/H]")
if cvp == 'E' or cvp == 'e':
    cvp1 = input ("Taşınabilir mi? [E/H]")
    if cvp1 == 'E' or cvp1 == 'e':
        cvp2 = input ("Kola takılabilir mi? [E/H]")
        if cvp2 == 'E' or cvp2 == 'e':
            print("Tamam, Saat!")
        else:
            cvp3 = input ("Konaşmak için kullanılır mı? [E/H]")
            if cvp3 == 'E' or cvp3 == 'e':
                print("Tamam, Telefon!")
            else:
                cvp4 = input ("Bir uygulama çalıştırır mı? [E/H]")
                if cvp4 == 'E' or cvp4 == 'e':
                    print("Tamam, Tablet ya da Laptop!")
                else:
                    cvp5 = input ("Video çeker mi? [E/H]")
                    if cvp5 == 'E' or cvp5 == 'e':
                        print("Tamam, Kamera!")
                    else:
                        cvp6 = input ("Fotoğraf çeker mi? [E/H]")
                        if cvp6 == 'E' or cvp6 == 'e':
                            print("Tamam, Fotoğraf Makinesi!")
                        else:
                            cvp7 = input ("Müzik dinlenir mi? [E/H]")
                            if cvp7 == 'E' or cvp7 == 'e':
                                print("Tamam, Radyo!")
                            else:
                                print("Bilemedim! Elektronik değil mi?")

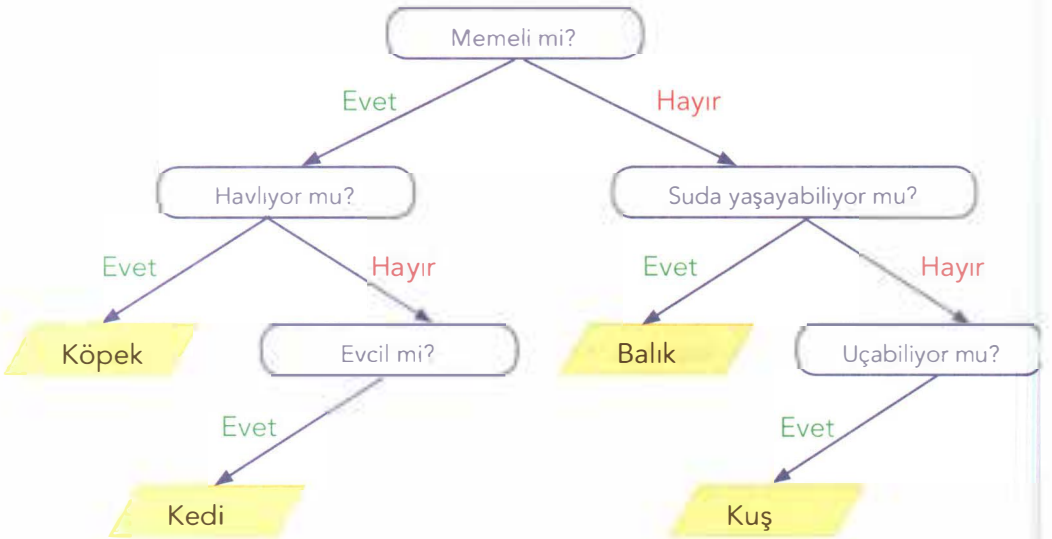
#Taşınabilir değilse
else:
    print ("Sanırım, Beyaz Eşya!")
#Elektronik değilse
else:
    print("Elektronik eşya tutmalısınız!")
  
```

Programın ekran çıktısı:

```
Karar Ağacı Oyunu:  
Aklından elektronik bir eşya tut!  
Elektronik mi? [E/H]E  
Taşınabilir mi? [E/H]E  
Kola takılabilir mi? [E/H]E  
  
Tamam, Saat!
```

Sıra Sende Etkinliği - 2

Sen de hayvanlar alemi için aşağıdakine benzer şekilde bir karar ağacını oluşturup kodlayabilir misin?



Soru ve Cevaplarla Neler Anladık?

- 1 Koşul ifadelerinin (if veya elif) koşul kısmında hangi operatörler kullanılır?
Koşul kısmında karşılaştırma operatörlerinden <, >, ==, !=, <=, >= en az biri mutlaka bulunur.
- 2 $-1 < y < 4$ eşitsizlik ifadesini açılımı mantıksal operatörler ile nasıl kodlanabilir?
 $(-1 < y)$ and $(y < 4)$

3 Aşağıdaki programın ekran çıktısı ne olur?

```
x = -3
print (x!=3)
```

True

4 Aşağıdaki programın ekran çıktısı ne olur?

```
x = -3
if x!=3:
    x=-x
print (x)
```

3

5 Karşılaştırma (<,> ,==, !=,<=,>=) veya mantıksal (and, or, not) operatörlerin hepsi sonuç olarak hangi veri tipinde değer üretirler?

bool veri tipinde (True ya da False) değer üretirler.

6 Aşağıdaki programın ekran çıktısı ne olur [True/False]?

```
print (2*2 == 5)
```

False

7 Aşağıdaki programın ekran çıktısı ne olur [True/False]

```
print (False or not (6 < 5))
```

True

8 not = 40

```
if not>= 50: print ("Geçti")
```

Yukarıdaki programın ekran çıktısı ne olur?

Hata verir. Çünkü "not" bir değişken adı olarak kullanılamaz.

9 Aşağıdaki programın girilen cinsiyet değerine göre ismin önüne "Bay" ise Mr, "Bayan" ise Mrs eklemesi için boşluğa ne yazılmalıdır?

```
ad = "Ada"
cnsyt = input ("Cinsiyetiniz [Bay/Bayan]?")
if _____:
    print ("Mr.", ad)
else:
    print ("Mrs.", ad)
cnsyt == "Bay"
```

BULMACA

Şifre

1. (Tersi) "OR" operatörünün Türkçesi.
2. Bu bölümde anlatılan yapay zekâ oyunu.
3. Bu kitapta anlatılan programlama dili.
4. "VE" operatörü.
5. "FALSE" ifadesinin Türkçesi.
6. Aşağıdaki programın çıktısı;

```
a=5  
if a>3: print ("Zİ")  
else: print ("RAR")
```
7. "DEĞİLSE" anlamındaki komut.
8. "if, if-else, elif" ifadeleri ona bakarak işlem yapar.
9. Koşul ifadelerinde, döngülerde kullanılan operatörlerin genel adı.

Aşağıya bulduğunuz şifreyi kodlayınız.

Bölüm

6

Döngüler (Loops)

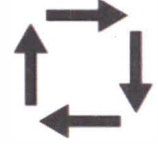
```
...Pyt""hon...  
..88888888o.. "8b.  
..1888P""Y8888b "' f  
o888 (gece) 888) "0  
o88888döngü8888B 'r  
p8888888888888Ü 8  
(8888for8888L 8)  
8888888888E 8  
Y88888888N (gündüz) lo  
Y888888T op  
"Y88888b oP"  
"Y8888o.. ..oP"  
~~~~~ Python ~~~~~
```

Gece gündüze, gündüz geceye katılarak dönüp gitmektedir.

Döngü Yapılarının Mantığı

Hayatta aynı şeyleri tekrar etmek kadar sıkıcı bir şey yoktur. Bu sıkıcılığı programlama dili geliştiricileri de anlamışlar ki tekrarlanan işlemleri göstermenin ya da ifade etmenin daha basit ve kolay bir yolunu bulmuşlar. Buldukları bu yola da döngü (loop) yapıları adını vermişlerdir.

Hemen hemen bütün programlama dillerinde olduğu gibi Python dilinde de iki farklı döngü yapısı vardır. Bunlar;



- for döngüleri
- while döngüleri

Bu döngülerin birbirinden farkı:

- **for döngüleri:** tekrar sayısının baştan belli olduğu döngülerdir. Örnek: 0'dan 24'e kadar çift sayıları ekranda göster.
- **while döngüleri:** tekrar sayısının baştan belli olmadığı, koşul geçerli olduğu sürece işlemlerin tekrar edildiği döngülerdir. Bu yüzden tekrar sayısı koşula bağlı olan döngüler olarak da adlandırılırlar. Örnek: Sıfır girilmediği sürece sayıların karesi al.

Döngü Kullanmanın Ne Faydası Olabilir?

Döngü yapıları, tekrar eden işlemleri oldukça basitleştirmekte, bir anlamda otomatikleştirmektedir.

Örneğin ekrana 5 kez "Python" yazan aşağıdaki programda;

- Döngü kullanılmadığında 5 kez aynı kod satırının tekrarlandığı görülmektedir.
- Döngü kullanıldığında ise kod satırının 1 kez yazılıp, 5 kez çağrıldığı görülmektedir.

Döngü kullanılmadığında	Döngü kullanıldığında
<pre>print("Python") print("Python") print("Python") print("Python") print("Python")</pre>	5 defa tekrarla print("Python") <pre>#for döngüsü ile for a in range(5): print("Python")</pre>

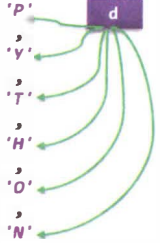
for Döngüsü

Tekrar sayısının baştan belli olduğu, ardışık eleman listesinin sırayla işleme konulduğu döngü yapısıdır. for döngüsü in operatörünü kullanarak liste üzerinde ardışık (iteratif) ilerler.

for komutunun kullanım şekli ve çalışma mantığı;

```
for d in Liste:
    #işlem1
    #işlem2
    #....
```

Liste = "PYTHON"
for d in Liste:
 print(d)



Şimdi for döngü yapısında yer alan komut ve değişkenleri açıklayalım;

Liste

Sıralı eleman dizisidir. Liste değişkeni "Python" gibi bir karakter dizisi olabileceği gibi "0, 1, 2, 3, 4, 5" gibi bir sayı dizisi de olabilir. Sayı dizisi elde etmek için range() fonksiyonu kullanılabilir.

in / not in Operatörleri

For döngüleri veya listeler gibi sıralı yapılarda (range, list) eleman sorgulamasında kullanılırlar. Örnek kullanım:

- **X in S:** Eğer X, S içerisinde varsa True yoksa False değerini döndürür.
- **X not in S:** Eğer X, S içerisinde varsa False yoksa True değerini döndürür.

Proje 6.1. "Ali" Listede mi?

Verilen bir listedeki adları alt alta ekrana yazdıran programı kodlayalım.

Çözüm: Programda öncelikle "Ali"nin listede olup olmadığı "in" komutu ile sorgulanmaktadır. Ekran çıktısındaki False sonucu listede olmadığını bildirmektedir. Sonra da listedeki adlar alt alta for döngüsü ile yazdırılmaktadır.

Python dili kodlaması	Ekran Çıktısı
<pre>liste = ["ekrem", "can", "berat", "zehra"] #Ali listede var mı? print ("ali" in liste) print ("Listede olanlar...") for ad in liste: print (ad)</pre>	<pre>False Listede olanlar... ekrem can berat zehra</pre>

range Fonksiyonu

İstenilen aralıkta sayı dizisi oluşturmak için kullanılır. Aynı zamanda for döngü yapısında tekrar sayısını belirtmek için kullanılan bir fonksiyondur. Kullanım şekli:

```
range(ilk, son, adım)
```

Örnekteki gibi range fonksiyonu, tekrar sayısını ifade ederken 3 parametre alabilir. İlk parametresi belirtilmez ise 0, adım değeri belirtilmez ise 1 kabul edilir. Kullanım Örnekleri:

```
>>> range(5)
[0, 1, 2, 3, 4] şeklinde 0-5 (5 hariç) arasındaki sayıları üretir.
>>> range(3)
[0, 1, 2] sayılarını üretir. 0 ile başladığına dikkat ediniz.
>>> range(1, 6)
[1, 2, 3, 4, 5] (1-6 arasındaki sayıları) üretir (1 dahil, 6 hariçtir).
>>> range(3, 9)
[3, 4, 5, 6, 7, 8] (3'ten 9'a kadar ki sayıları) üretir.
>>> range(3, 30, 3)
[3, 6, 9, 12, 15, 18, 21, 24, 27] (3-30 arasındaki sayıları üçer üçer artırarak) üretir.
>>> range(4, 1, -1)
[4, 3, 2] sayılarını geriye doğru üretir.
```

Soru: range() komutu ile ardışık bir şekilde sayılar listelendiğine göre for döngü yapısında doğrudan liste kullanılabilir mi?

Cevap: Evet, kullanılabilir. Aşağıdaki iki kullanım da eşdeğerdir;

```
for a in range(0,5):
    print("Python")
```

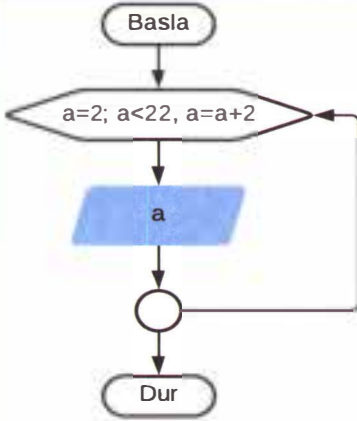
```
for a in [0,1,2,3,4]:
    print("Python")
```

Not: `list(range(0,5))` ile de üretilen sayılar döngü kullanılmadan liste halinde `[0,1,2,3,4]` alınabilir.

Proje 6.2. Çift Sayıları Yazdırma

2' den 20'ye kadar (20 dahil) çift sayıları ekranda gösteren programı for döngüsü ile kodlayalım.

Algoritması



Python dili kodlaması

```
for a in range(2,22,2):
    print(a, end=' ')
```

Açıklama

Program, 2'den 20'ye (22 hariç) kadarki sayıları 2'şer 2'şer (ritmik) ekrana yazdırır.

Ekran Çıktısı:

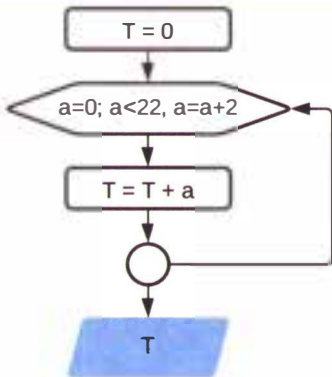
2 4 6 8 10 12 14 16 18 20

Not. Rakamları alt alta değil de yan yana yazdırmak için `print(a)` komutunun `print(a,end=' ')` şeklinde kodlanması gerekir.

Proje 6.3. Çift Sayıların Toplamı

0' dan 20' e kadar ki (20 dahil) çift sayıların toplamını ekranda gösteren programı for döngüsü ile kodlayalım.

Algoritması



Python dili kodlaması

```
T = 0
for a in range(0,22,2):
    T = T + a
#döngü sonu
print("Toplam= ", T)
```

Açıklama ve ekran çıktısı

Program, döngü içerisinde 0'dan 22'ye (22 hariç) kadar ki çift sayıları birbiri ile toplar ve toplam sonucu döngüden çıktıktan sonra ekrana yazar. Ekran Çıktısı:

Toplam= 110

Not. Bu bölümde algoritma akış şemalarında Başla/Dur simgeleri gösterilmemiştir.

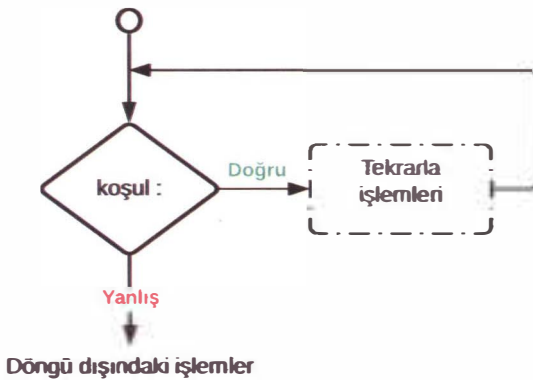
Sıra Sende Etkinliği-1

Belli bir aralıktaki (0'dan 52'e kadarki) tek sayıları ve toplamını ekranda gösteren programı for döngüsü ile kodlayabilir misin?

while Döngüsü

İşlemlerin ne kadar tekrarlanacağını koşulun doğru ya da yanlış olmasına bağlı olduğu, koşul geçerli olduğu sürece işlemlerin tekrarlandığı döngülerdir.

while döngüsünün çalışma mantığını ve akış şeması;



```
while koşul:
    işlem1
    işlem2
```

<koşul>: geçerli olduğu sürece tekrarla (işlemleri)
Verilen koşul geçerli (doğru) olduğu sürece döngü içerisindeki işlemler tekrar edilir. Koşul kontrol işlemi, döngü başında yapılır. Koşul geçersiz (yanlış) olduğunda döngü sonlanır.

Proje 6.4. While Döngüsü: Karesini Al

Girilen sayı sıfırdan farklı olduğu sürece o sayının karesini alan programı kodlayalım. Programın örnek ekran çıktısı:

```
Çıkmak için 0 giriniz!
Sayı gir.: 
Karesi.: 25
Sayı gir.: 
Karesi.: 9
Sayı gir.: 
Karesi.: 0
```

Program sıfır "0" girilmediği sürece girilen sayıların karesini ekranda göstermektedir. Sıfır girildiğinde ise programı sonlandırmaktadır. Diğer bir ifade ile döngüden çıkış koşulu, sıfır girilmesidir.

Algoritması	Python dili kodlaması
1. Başla 2. $S = 1$ 3. S sıfıra eşit olmadığı sürece: Gir bir sayı (S), Yaz karesini ($S*S$) 4. Dur	<pre>S = 1 print ("Çıkmak için 0 giriniz!") while (S!=0): S = int (input("Sayı gir.:")) print ("Karesi.: ", S*S)</pre>

Soru: Niçin programın ilk satırında $S=1$ komut satırı ile S değişkenine "1" değeri atanmıştır?

Cevap: 3. satırdaki while döngüsünün geçerli olabilmesi (çalışması) için $S!=0$ şartının sağlanması gerekir. O nedenle S'ye başlangıç değeri olarak "1" değeri atanmıştır.

Proje 6.5. Tek Sayıların Toplamı

1'den 100'e kadar olan tek sayıların toplamını bulan programı kodlayalım. Programın örnek ekran çıktısı;

Toplam= 2500

Algoritması	Python dili kodlaması
1. Başla 2. Döngü değişkeni ($a=1$) 3. ($a \leq 100$) olduğu sürece: a) Tek sayıları birbirleri ile topla ($Top = Top + a$) b) Tek sayıları artır ($a=a+2$) 4. Sonlandır döngüyü 5. Yaz sayıların toplamını (Top) 6. Dur	<pre>a = 1 Top = 0 #Toplama değişkeni while a <= 100: Top = Top + a #Yığılmalı toplama a = a + 2 #Döngü sonu print ("Toplam=",Top)</pre>

Proje 6.6. ASCII Kod Tablosu

A dan Z ye kadar İngilizce karakterleri ekranda gösteren programı while döngüsü ile kodlayalım. Programın örnek ekran çıktısı;

A B C D E F G H I J K L M N O P Q R S T U V W X Y Z

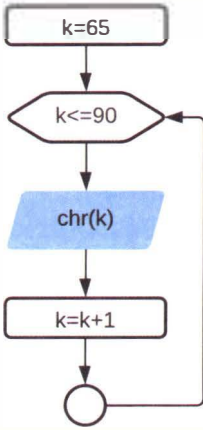
A'dan Z'ye karakterler, ASCII kod tablosunda **65** ile **90** rakamları arasında yer alır. A'dan Z'ye karakterleri ekranda gösterirken aslında bu rakamların ASCII kod karşılıklarını görürüz.

chr(k)	k sayısının ASCII kod karakter karşılığını verir.	chr(65) -> 'A'
ord('A')	'A' nın ASCII kod tablosundaki sayı karşılığını verir.	ord('A') -> 65

**ASCII
Tablosu**

A	65
B	66
C	67
D	68
E	69
F	70
G	71
H	72
I	73
J	74
	75
L	76
M	77
N	78
O	79
P	80
Q	81
	82
S	83
T	84
U	85
V	86
W	87
X	88
Y	89
Z	90

Algoritması



Python dili kodlaması

```

k = ord('A') #k=65 oldu
while k<=ord('Z'):
    #chr(65) = 'A' demektir
    print (chr(k), end=' ')
    k = k + 1
  
```

Not. ASCII, (İngilizcesi: **A**merican **S**tandard **C**ode for **I**nformation **I**nterchange, Türkçesi: Bilgi Değişimi İçin Amerikan Standart Kodlama Sistemi) klavyedeki karakterlerin bir takım sayılarla eşleştirildiği basit bir kodlama sistemidir. ASCII kod tablosunda yer alan karakterleri ve sayısal değerlerin tamamını görmek için <http://www.asciitable.com> veya <https://www.ascii-code.com> adreslerini inceleyebilirsiniz.

Sıra Sende Etkinliği-2

1' den N sayısına kadar ki sayıların karesini ekrana yazan programı while döngüsü ile kodlayabilir misin? Programın örnek ekran çıktısı;

```

N = 5
1 in karesi= 1
2 in karesi= 4
3 in karesi= 9
4 in karesi= 16
5 in karesi= 25
  
```

while True

Sonsuz döngü oluşturur. Döngü içerisindekiler koşulsuz çalıştırılır. Döngüden çıkış ancak break komutu ile döngü kırılarak gerçekleşir.

Trafik sinyalizasyon sistemleri gibi bazı uygulamaların sonsuza dek çalışması beklenir. O zaman bu sistemlerin programlanmasında while True: döngüleri kullanılabilir.

Proje 6.7. Sonsuz Döngü: Karesini Al

Girilen sayı sıfırdan farklı olduğu sürece o sayının karesini alan Proje 6.4'teki programı while True: ile yeniden kodlayalım.

Çözüm: Program sıfır "0" girilmediği sürece girilen sayıların karesini ekranda göstermektedir. Sıfır girildiğinde ise programı sonlandırmaktadır. Diğer bir ifade ile döngüden çıkış koşulu sıfır girilmesidir.

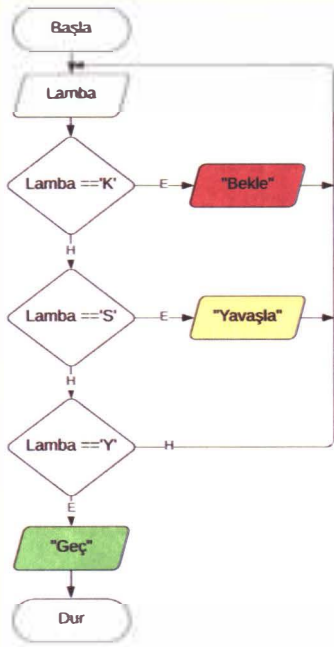
Algoritması	Python dili kodlaması
- Başla - Sonsuz döngü (while True:) - Gir bir sayı (S) - Hesapla ve Yaz karesini (S*S) - Eğer (S==0): break (Döngüden çık) - Dur	<pre>print("Çıkmak için 0 giriniz!") while True: S=int(input('Gir bir sayı.:')) print ("Karesi.: ", S*S) if (S==0): break #döngüden çık #Döngü sonu</pre>

Proje 6.8. Sonsuz Döngü: Trafik Sinyalizasyon Sistemi

Daha önce Proje 1.4'de algoritması ve akış şeması verilen trafik lambası programını while True: ile Python dilinde kodlayalım.

Çözüm: Trafik lambası uygulamasında döngüden çıkış koşulu, lambanın yeşil yani "Y" olmasıdır.

Algoritması



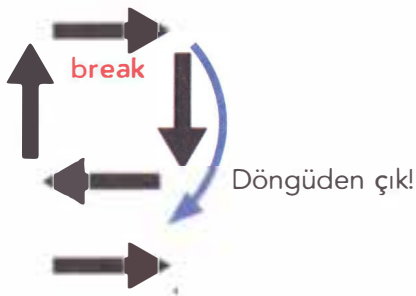
Python dili kodlaması

```
# Trafik Lambası Programı
while True:
    Lamba = input('Trafik Lambası rengi.:')
    if (Lamba=='K'):
        print ("Bekle")
    if (Lamba=='S'):
        print ("Yavaşla")
    if (Lamba=='Y'):
        print ("Geç")
        break #döngüden çık
#Döngü sonu
```

break Komutu

Bir koşula bağlı olarak while veya for döngülerinden çıkmak için yani döngüyü sonlandırmak için kullanılır. Program akışını döngünün dışındaki ilk komuta atlar.

Döngü:



```
1 for a in range(0,12,2):
2     if (a==6):
3         break
4     print(a, end=' ')
```

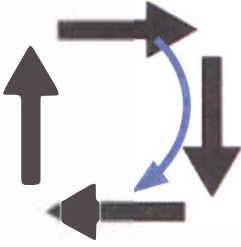
Console

```
<terminated> printKomut.py [C:\Users\bcl\
0 2 4 |
```

Açıklama: Programımız normalde 0'dan 10'a kadar olan çift sayıları ekranda göstermektedir. Fakat a=6 olduğunda 3. satırdaki break ifadesi döngüden çıkılmasına (kırılmasına) sebep olur ve ekran çıktısına 6'dan sonraki sayılar yazdırılmamış olur.

continue Komutu

continue komutu; döngü içerisinde o anki işlemin atlanmasını (göz ardı edilmesi) ve bir sonraki işlemde devam edilmesini sağlar.



```
1 for a in range(0,12,2):
2     if (a==6):
3         continue
4     print(a, end= ' ')
```

Console

<terminated> printKomut.py [C:\Users\bc\

0 2 4 8 10

Açıklama: Programımız normalde 0'dan 10'a kadar olan çift sayıları ekranda göstermektedir. Fakat a=6 olduğunda 3. satırdaki continue ifadesi programın bir satır atlamasına (yani 4. satırın göz ardı edilmesine) sebep olur ve ekran çıktısına 6 sayısı yazdırılmamış olur.

Proje 6.9. Sonsuz Döngü: Hesap Makinesi

Dört işlem (toplama, çıkarma, çarpma ve bölme) yapan basit bir hesap makinesi uygulamasını kullanıcının devam etmek isteyip/istememesine göre kodlayalım.

Python Dili Kodlaması

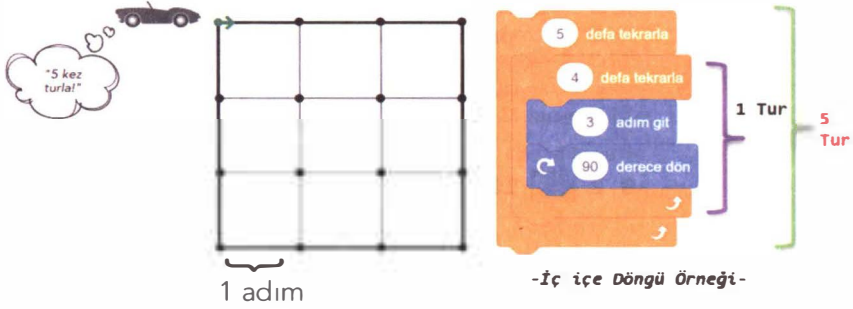
```
while True:
    s1 = int(input('sayı1.:'))
    s2 = int(input('sayı2.:'))
    op = input('işlem.:')
    if (op == '+'):
        S = s1 + s2
    elif (op == '-'):
        S = s1 - s2
    elif (op == '*'):
        S = s1 * s2
    elif (op == '/'):
        S = s1 / s2
    else:
        print("Hatalı seçim")
    print("Sonuç=",S)
    cvp=input("Başka işlem olacak mı[e/h]?")
    if cvp=='h':
        break
    else:
        continue
print("..Makine kapandı..")
```

Örnek Ekran Çıktısı

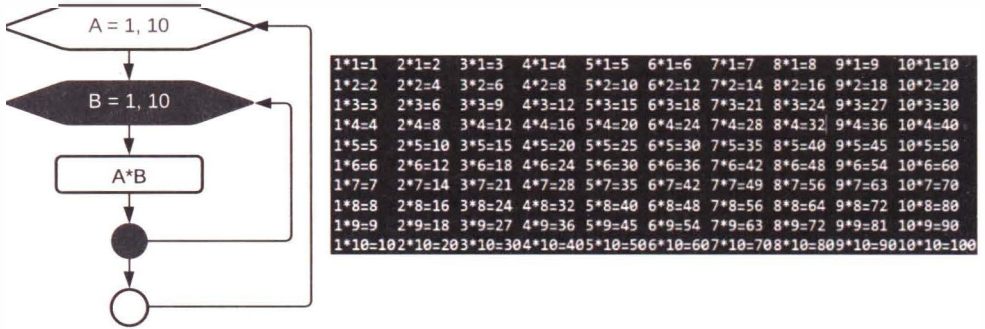
```
sayı1.:
sayı2.:
işlem.:
Sonuç= 15
Başka işlem olacak mı[e/h]?
sayı1.:
sayı2.:
işlem.:
Sonuç= 13
Başka işlem olacak mı[e/h]?
..Makine kapandı..
```

İç İçe Döngüler

Bir döngü yapısı içerisinde başka bir döngü kullanılarak iç içe döngüler oluşturulabilir. Örneğin bir yarış arabasını kare şeklindeki bir yarış pistinde 5 tur attıktan sonra yarışı tamamlayacak programı iç içe döngü bloklarını kullanarak ifade edebiliriz.



İç içe döngülere verilebilecek başka bir örnek çarpım tablosudur. Çarpım tablosunda içteki döngünün her tamamlanışından sonra, dıştaki döngü otomatik olarak bir artar. Dıştaki döngü değişkeni A=1 iken içteki döngü değişkeni B; 1'den 10'a kadar değer alır. B son değerine (B=10) ulaştıktan sonra A'nın değeri 2 olur ve bu işlem 10 kez tekrarlanır. Yani toplam 100 (10*10) kez çarpma işlemi gerçekleşir.



İç içe döngü yapılarında içteki döngü tamamen bitmeden dıştaki döngüye geçilmez.

Proje 6.10. Çarpım Tablosu

Ekran çıktısı yukarıda verilen çarpım tablosunu kodlayalım.

Python Dili Kodlaması

```
for A in range(1,11):
    for B in range(1,11):
        print (A,'*', B,'=', A * B)
    print("\n") #bir satir atla
    #B dongusu sonu
#A dongusu sonu
```

Açıklama

print("\n")
çarpım tablosunu alt alta
göstermek için kullanıldı.

Proje 6.11. İç İç Döngü: Dik Üçgen

Yandaki dik üçgen desenini ekrana basacak/yazacak programı iç içe döngü kullanarak kodlayalım.

Çözüm:

Çıktı desenini incelediğimizde ekrana alt alta farklı sayılarda “#” karakteri yazılmaktadır. Ekranın hem satır hem de sütununa yazıldığına göre iç içe döngü kullanmamız gerekir. Tekrar sayısı da belli olduğuna göre for döngüsü kullanmamız daha doğru olur.

Programda dıştaki döngümüz (m) satır sayısını, içteki döngümüz (n) sütun sayısını göstermektedir. Desen, 5 satır ve 5 sütundan oluşmaktadır. Satır sayısını gösteren m değişkeni sırası ile 1-5 arası değer alır. Her seferinde satır sayısı (m) kadar ‘#’ karakteri ekrana bastırıldığından içteki döngü değişkeni n, m kadar ilerler.

		n: Sütunlar					Python Dili Kodlaması
m: Satır		1	2	3	4	5	
	1	#					#dıştaki döngü: m for m in range (1,6): #içteki döngü: n for n in range(m): print("#", end = ' ') #içteki döngü sonu print() #bir satır atla #dıştaki döngü sonu
	2	#	#				
	3	#	#	#			
	4	#	#	#	#		
	5	#	#	#	#	#	

Sıra Sende Etkinliği-3

Sen de yandaki ters dik üçgen desenini ekranda gösterecek programı iç içe döngü kullanarak kodlayabilir misin?

#

Soru ve Cevaplarla Neler Anladık?

- 1 Ne zaman **while**, ne zaman **for** döngülerini kullanmak daha mantıklıdır?
Tekrar sayısının baştan belli olduğu döngülerde **for**, belli olmadığı koşul geçerli olduğu sürece işlemlerin tekrar edildiği döngülerde ise **while** kullanmak daha mantıklıdır.
- 2 **break** ve **continue** komutları döngülerde ek kontrol sağlar. Bu komutların birbirinden farkı nedir?
break komutu; bir koşula bağlı olarak döngü değişkeninin son değerine ulaşmadan döngüyü terk etmesini (döngüyü kırmasını) sağlar.
continue komutu ise döngü içerisinde o anki işlemi atlamamızı/göz ardı etmemizi sağlar fakat döngüden bir çıkış sağlamaz.
- 3 İç içe döngü başlığı altında gösterilen yarış arabası, 5 tur atarak kare şeklindeki yarış pistini tamamlayacaktır. Buna göre yarış arabası toplamda kaç adım (metre) ilerleyerek yarışı tamamlamıştır? (Sayfa 78'deki görsele bakarak cevaplayınız).
 $(3*4)*5=60$ adım = 60 metre
- 4 Aşağıdaki programın ekran çıktısı ne olur?

```
metin = "Python"
for a in range(6,0,-1):
    print (a, end=' ')
```

6 5 4 3 2 1
- 5 Aşağıdaki programın ekran çıktısı ne olur?

```
metin = "Python"
for a in range(6,0,-1):
    print (metin, end=' ')
```

Python Python Python Python Python Python
- 6 Aşağıdaki programın ekran çıktısı ne olur?

```
for a in [1,2,3]:
    print(a*a, end=' ')
```

1 4 9
- 7 Python dilinde sonsuz döngü oluşturma komutu hangisidir?
while True:

- 8 0'dan 20'ye kadarki (20 dahil değil) çift sayıları ekranda gösteren programı while döngüsü ile kodlayınız.

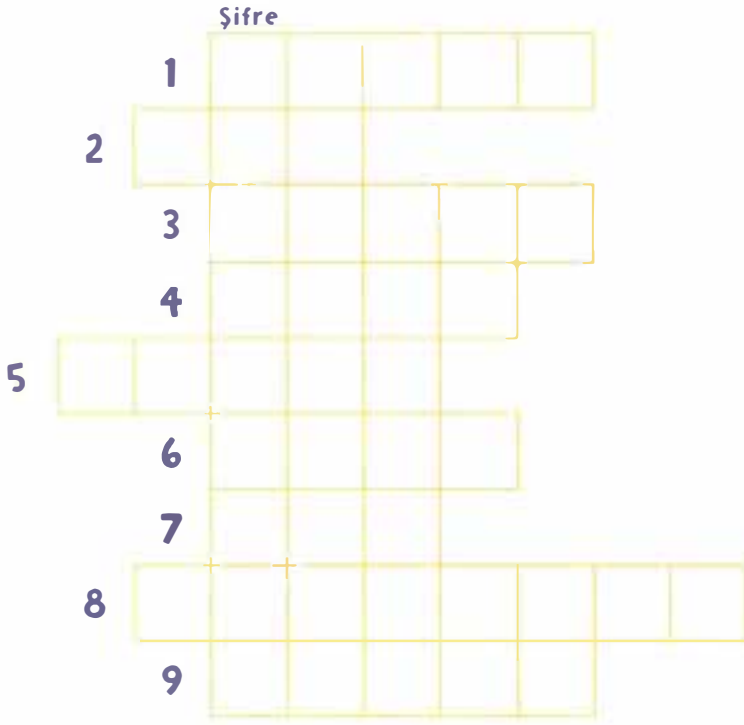
```
a=0
while (a<20):
    print (a)
    a+=2
```

- 9 a'nın içinde 8 olup olmadığını sorgulayan aşağıdaki programın ekran çıktısı ne olur?

```
a=range(0, 8, 2)
if (8 in a):
    print ("var")
else:
    print ("yok")
```

yok

BULMACA



1. Tekrar sayısının baştan belli olmadığı, koşul geçerli olduğu sürece işlemlerin tekrar edildiği döngü yapısı.
2. ASCII kod tablosundaki sayısal değerlerin karakter karşılıklarını veren komut.
3. (Tersi) "American Standard Code for Information Interchange" kısaltması.
4. "Döngü" İngilizcesi.
5. Bir koşula bağlı olarak döngüyü sonlandırır.
6. Sonsuz döngülerde `while` yanına yazılır.
7. (Tersi) Tekrar sayısının baştan belli olduğu döngü yapısı.
8. (Tersi) Döngüde o anki işlemin atlanmasını ve bir sonraki işlemde devam edilmesini sağlar.
9. (Tersi) İstenilen aralıkta sayı dizisi oluşturmak için kullanılan komut.

Aşağıya bulduğunuz şifreyi kodlayınız.

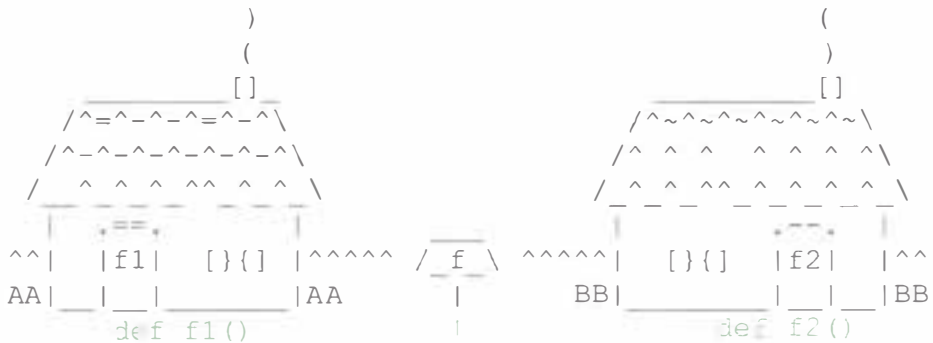
--	--	--	--	--	--	--	--	--	--

WHILETRUE

Bölüm

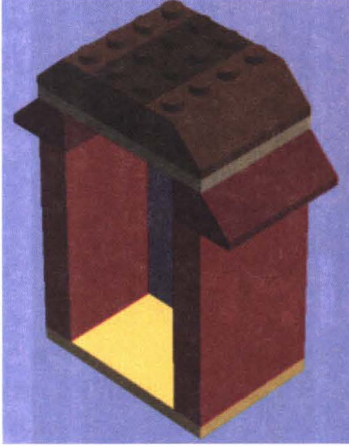
7

Fonksiyonlar

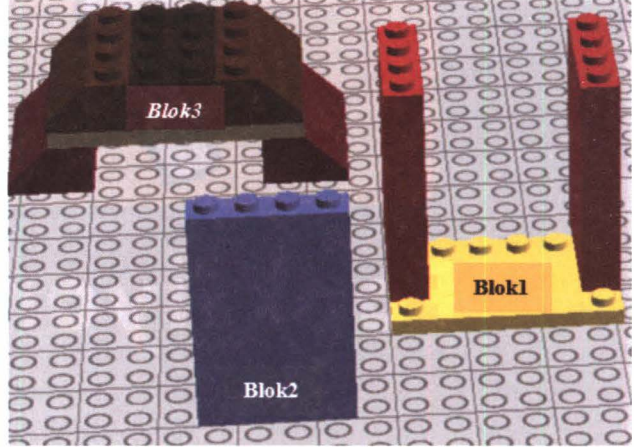


Fonksiyon Nedir?

Programların boyutu (kod satırı) büyüdükçe anlaşılabilirliği ve takibi zorlaşır. Bu zorluğu aşmak için programlar, küçük kod blokları (fonksiyonlar) halinde organize edilebilir.



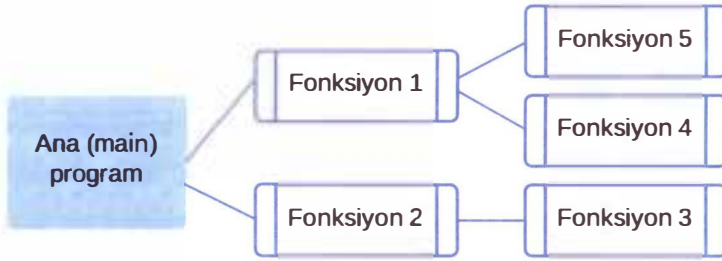
Ana lego



Ana legoyu oluşturan küçük alt bloklar

Program yazımı, LEGO bloklarını birleştirmeye benzer. LEGO örneğindeki gibi bir ana program, küçük kod bloklarının (fonksiyonların) birleşimi ile oluşur. Ve her bir kod bloğu, programın belli bir işlevini yerine getirir.

Programın belli bir işlevini (görevini) yerine getiren küçük program parçacıklarına (kod bloklarına) fonksiyon adı verilir.



Python dilinde genellikle bir ana (main) program, birden fazla fonksiyonun birleşiminden oluşur.

Not. Farklı programlama dillerinde **fonksiyon** ismi yerine "alt program, alt yordam, metot, yordam veya prosedür" gibi isimler de kullanılabilir.

Fonksiyonlar Niçin Kullanılır?



Bir evin tamamını tek bir kişi yapamaz. Duvarları örmek için duvarcı(), boyatmak için boyacı(), su tesisatları için tesisatçı(), elektrik tesisatları için elektrikçi(), kapı-pencereler için mobilyacı() gibi farklı ustalar (fonksiyonlar) çalıştırılır. Aynen ev örneğinde olduğu gibi büyük bir program da farklı işlevlere sahip birçok fonksiyondan oluşabilir.

Bir elektrikçi(), bir evin (hatta başka evlerin) farklı oda tesisatları için birçok kez çağrılabilir. Benzer şekilde bir fonksiyon da başka fonksiyonlar tarafından birçok kez çağrılabilir.

Fonksiyon kullanımı hem kod tekrarını önler hem de programı daha anlaşılır yapar. Programların anlaşılabilirliğini, hata takibini ve program üzerinde ileri-geri kod takibini kolaylaştırmak için programın fonksiyonlara ayrılması iyi olur.

Python Dilinde Fonksiyon Nasıl Tanımlanır?

Python dili print(), input() gibi belli bir işlevi yerine getiren yüzlerce yerleşik (İngilizcesi: built-in) fonksiyon içerir.

Eğer bir Python komutu yanında “()” aç-kapa parantezleri varsa, o bir fonksiyondur. Python dilinin yerleşik fonksiyonlarına benzer şekilde bizde kendi fonksiyonlarımızı tanımlayabiliriz.

Python dilinde fonksiyon tanımlamak için “def” komutu kullanılır. def; “fonksiyon tanımlama” anlamındaki İngilizce “definition function” kelimelerinden türetilmiştir. Kullanım şekli:

```
def fonksiyonAdı() :  
    fonksiyon gövdesi
```

```
def Selam():  
    print ("Merhaba, Sayın Çobanoğlu")
```

Tanımlanan bir fonksiyon, ana program veya başka bir fonksiyon içerisinde sadece ismi belirtilerek “Selam()” şeklinde çağrılabilir. Örneğin:

Python dili kodlaması

```
#Selam isimli fonksiyon  
def Selam():  
    print("Merhaba, Sayın Çobanoğlu")  
  
#Ana program  
Selam() #fonksiyon çağrıldı  
Selam() #fonksiyon tekrar çağrıldı
```

Ekran çıktısı

```
Merhaba, Sayın Çobanoğlu  
Merhaba, Sayın Çobanoğlu
```

Not. Bu örnekteki gibi bir fonksiyon bir defa oluşturulur. Fakat birçok kez çağrılabilir.

Bir Fonksiyonun Girdileri (Parametreleri)

Bir fonksiyon girdi olarak farklı değerler alabilir. Fonksiyon girdilerine "parametre" ya da "argüman" adı verilir. Fakat bu iki kelime arasında şöyle bir fark vardır:

Fonksiyon tanımlanırken belirtilen girdilere "parametre", çağrılırken belirtilen girdilere ise "argüman" denir.

Örneğin;

Python dili kodlaması	Ekran çıktısı
<pre>#Selam isimli parametrelili fonksiyon def Selam(Ad): #Ad bir parametredir. print("Merhaba, Sayın", Ad) #Ana program Selam("Bülent") #'Bülent' bir argümandır Selam("Ayşe") #'Ayşe' bir argümandır Selam("Eda") #'Eda' bir argümandır</pre>	<pre>Merhaba, Sayın Bülent Merhaba, Sayın Ayşe Merhaba, Sayın Eda</pre>

Varsayılan Parametre Değeri

Bir fonksiyona tanımlama aşamasında varsayılan (default) olarak bir değer atanabilir. Fakat istenirse bu değer daha sonra değiştirilebilir yani geçersiz kılınabilir (override edilebilir). Örnek;

Python dili kodlaması	Ekran çıktısı
<pre>#Varsayılan(default) ad:'Bülent' def Selam(Ad='Bülent'): print("Merhaba, Sayın", Ad) #Ana program Selam() Selam("Eda") #Varsayılan ad değiştirildi</pre>	<pre>Merhaba, Sayın Bülent Merhaba, Sayın Eda</pre>

Açıklama: selam() fonksiyonu tanımlanırken, varsayılan olarak 'Ad' değişkenine "Bülent" değeri atandı. Fakat 6. Satırda bu varsayılan değer "Eda" ile değiştirildi.

Proje 7.1. Fizz Buzz Oyunu:

1'den 100'e kadar olan sayıları, sırayla ekrana yazdıralım. Fakat program; eğer sayı 3 ve katları ise, sayı yerine "fizz"; 5 ve katları ise, sayı yerine "Buzz"; eğer sayı hem 3'ün hem de 5'in (15'in) katıysa, sayı yerine "fizzBuzz" yazmalıdır.

Python dili kodlaması	Ekran çıktısı
<pre>#fizzBuzz fonksiyonu def fizzBuzz(): for x in range (1,101): if x % 15 == 0: print ("fizzBuzz") if x % 3 == 0: print ("fizz") if x % 5 == 0: print ("Buzz") else: print(x, end=' ') #Ana program fizzBuzz() #fonksiyon çağrıldı</pre>	<pre>1 2 fizz 3 4 Buzz fizz 6 7 8 fizz 9 Buzz 11 fizz 12 13 14 fizzBuzz fizz Buzz 16 17 fizz 18 19 Buzz fizz 21 22 23 fizz 24 Buzz 26 fizz 27 28 29 fizzBuzz fizz Buzz 31 32 fizz 33 34 Buzz fizz 36 37 38 fizz 39 Buzz 41 fizz 42 43 44 fizzBuzz fizz Buzz 46 47 fizz 48 49 Buzz fizz 51 52 53 fizz 54 Buzz 56 fizz 57 58 59 fizzBuzz fizz Buzz 61 62 fizz 63 64 Buzz fizz 66 67 68 fizz 69 Buzz 71 fizz 72 73 74 fizzBuzz fizz Buzz 76 77 fizz 78 79 Buzz fizz 81 82 83 fizz 84 Buzz 86 fizz 87 88 89 fizzBuzz fizz Buzz 91 92 fizz 93 94 Buzz fizz 96 97 98 fizz 99 Buzz</pre>

Açıklama: Sayıları yan yana yazdırmak için her bir print içerisinde end=', ' ifadesini kullandık. Sayıları alt alta yazdırmak için bu ifadeyi silmeniz yeterlidir.

Bir Fonksiyonun Çıktıları (return Komutu)

Bir fonksiyonun çıktısı (geri dönüş değeri) return komutu ile alınabilir. return komutu; fonksiyonun geriye döndüreceği değeri üreten bir komuttur.

Eğer bir fonksiyon çıktı olarak bir değer üretmeyecekse None değerini döndürür. Farklı return kullanım örnekleri aşağıdaki tabloda verilmiştir.

Python return kullanım örnekleri

Komut örneği	Çıktı değeri	Açıklama
return 6	6	Bu örnekte fonksiyonun çıktısı (geri dönüş değeri) sabit bir sayıdır.
return a	a	Bu örnekte fonksiyonun çıktısı bir değişkendir.

return us(a,b)	us() fonksiyonunun sonucu	Bu örnekte fonksiyonun geri dönüş değeri, başka bir fonksiyonun "us()" çıktısıdır.
return a,b	a ve b	Python dilinde return ile aynı anda birden fazla değer geri döndürülebilir.
return	None	Bu örnekte fonksiyonun bir çıktısı yoktur. return, sadece fonksiyonu sonlandırır.

Proje 7.2. Dikdörtgenin Çevresini ve Alanını Hesaplama

Yandaki gibi dikdörtgen şeklindeki bir tarlanın çevresini ve alanını hesaplayan programı kodlayalım.



Bir dikdörtgensel bölgenin alanı Alan isimli fonksiyon ile aşağıdaki formülle hesaplanabilir:

$$\text{Alan} = \text{Uzunluk (u)} * \text{Genişlik (g)}$$

Bir dikdörtgensel bölgenin çevresi ise Çevre isimli fonksiyon ile aşağıdaki formülle hesaplanabilir: $\text{Çevre} = 2*g + 2*u$ (Bütün kenarların toplamı).

Python dili kodlaması	Ekran çıktısı
<pre>#Alanı hesaplayan fonksiyon def Alan(u,g): a = u * g return a #Çevreyi hesaplayan fonksiyon def Çevre(u,g): c = 2*u +2*g return c #Ana program u = int (input('Tarlanın uzunluğu:')) g = int(input('Tarlanın genişliği:')) print ("Tarlanın Alanı.:", Alan(u,g)) print ("Tarlanın Çevresi.:", Çevre(u,g))</pre>	<pre>Tarlanın uzunluğu:20 Tarlanın genişliği:30 Tarlanın Alanı.: 600 Tarlanın Çevresi.: 100</pre>

Proje 7.3. Fahrenheit Dereceye Çevirme

Ana programda (main fonksiyonunda) girilen Fahrenheit sıcaklığını (F°) çevir isimli fonksiyonla Dereceye (C°) çeviren ve çevrim sonucunu ana programda gösteren programı yazalım.

Fahrenheit sıcaklığını dereceye çevirmek için $C^{\circ} = (F^{\circ} - 32) / 1.8$ formülü kullanıldı. Programın örnek çıktısı:

Fahrenheit değeri:212
100.0 derecedir

Algoritması

1. Başla
2. Fahrenheit değerini gir (f)
3. **cevir(f)** isimli fonksiyona git ve dereceye çevir
 $C = (F - 32) / 1.8$
5. Ana programda işlem sonucunu yaz
6. Dur

Python dili kodlaması

```
#cevir fonksiyonu
def cevir(f):
    return ((f - 32) / 1.8)
#Ana program
f=float(input('Fahrenheit değeri:'))
C = cevir(f)
print(C , " derecedir")
```

Fonksiyonlarda Global ve Yerel (Lokal) Değişken Tanımlaması

Bir değişken, programın tamamında ya da sadece belli bir alanında faaliyet gösterebilir, yani çalışabilir. Tanımlandığı alan, dikkate alınarak değişkenler, genel (global) ve yerel (lokal) olmak üzere ikiye ayrılmaktadır:

Genel (global) değişkenler: Programın tamamında faaliyet gösterirler.

Yerel (lokal) değişkenler: Sadece belli bir fonksiyon ya da blok içerisinde faaliyet gösterirler.

Eğer bir değişken, fonksiyon içerisinde tanımlandı ise yerel, dışında (veya ana program içerisinde) tanımlandı ise global değişken olarak adlandırılır.

Normalde yerel değişkenler sadece tanımlandıkları fonksiyon içerisinde faaliyet gösterirler. Tanımlandıkları fonksiyon dışından erişilmeleri mümkün değildir. Ancak yerel bir değişken başına global kelimesi getirilerek ("global s" gibi) genel değişken yapılabilir. Örnek kullanımlar;

```
1 #carp() fonksiyonu
2-def carp():
3     a=2 #yerel degisken
4     b=3 #yerel degisken
5     print ("c:", c)
6     return a*b
7 #Ana program
8 c = 5 #global degisken
9 print("a*b=",carp())
10 print ("a:",a)
```

Açıklama: carp() fonksiyonu içinde tanımlanan yerel "a" değişkeni fonksiyon dışında (ana programda) faaliyet gösteremez. Programın 10. satırındaki hatanın kaynağı budur.

```
Console
<terminated> printKomut.py |C:\Users\bc\AppData\Local\Program
c: 5
a*b= 6
Traceback (most recent call last):
  File "C:\Users\bc\workspace\
    print ("a:",a)
NameError: name 'a' is not defined
```

```

1 #carp() fonksiyonu
2 def carp():
3     global a
4     b = 3 #yerel degisken
5     a = 2 #global degisken
6     print ("c:", c)
7     return a*b
8 #Ana program
9 c = 5 #global degisken
10 print("a*b=", carp())
11 print ("a:",a)

```

Console

<terminated> printKomut.py [C:\Users\bc\AppData\Local\P

```

c: 5
a*b= 6
a: 2

```

Açıklama: Soldaki programın 3. satırındaki yerel "a" değişkeninin başına global sözcüğü getirilerek genel değişken yapıldı. Bir global değişken, programın tamamında faaliyet gösterdiğinden hata mesajı almadık.

İçeriği Olmayan Fonksiyon Tanımlama

Python dilinde kod gövdesi (işlevi) olmayan sadece başlığı olan fonksiyon tanımı geçersizdir. Eğer henüz içeriğine karar veremediğiniz bir fonksiyonunuz var ve hata mesajı da almak istemiyorsanız fonksiyon içerisine pass veya return yazarak ilerleyebilirsiniz. Örnek kullanım;

pass kullanımı	Eşdeğeri	Ekran çıktısı
<pre>def ornek(): pass #ana program print(ornek())</pre>	<pre>def ornek(): return #ana program print(ornek())</pre>	None

Açıklama: None: "hiçbir şey (iş) yapma" anlamında bir komuttur. pass veya geri dönüş değeri belirtilmeyen return komutları sonuç olarak "None" değeri döndürürler.

Fonksiyon Yazımını Kısaltma: Lambda

Tek satırlık kısa fonksiyonlar tanımlamak için lambda komutu kullanılabilir. Kullanım şekli:

fonksiyon ismi= lambda değişkenler: komut satırı

Sanırım iki sayının toplamı programını aşağıdaki gibi hem def hem de lambda fonksiyonları ile yazdığımızda farkı daha iyi anlayacaksınız:

İki sayı toplamı: def fonksiyonu ile	İki sayı toplamı: Lambda fonksiyonu ile
<pre>def topla(a, b): return a+b</pre>	<pre>topla = lambda a, b: a+b</pre>

Proje 7.4. Lambda

Daha önce Proje 7.3'te verilen Fahrenheit sıcaklığını (F°) Dereceye (C°) çeviren programı lambda fonksiyonunu kullanarak tek satırda kodlayalım.

Python dili kodlaması	Ekran çıktısı
<pre>C = lambda f: ((f - 32) / 1.8) print (C(212))</pre>	100.0

Özyinelemeli (Rekürsif) Fonksiyonlar



Kendi kendini çağıran fonksiyonlara "özyinelemeli fonksiyonlar" adı verilir. Özyinelemeli fonksiyonlar algoritma tasarımını basitleştirir ancak fonksiyon çağırma sayısı arttığı için daha fazla bellek alanı kullanırlar.

Özyineleme, en genel anlamıyla bir yapının kendi kendini çağırmasıdır. Çam kozalaklarının oluşması, bir ağacın büyümesi sırasında büyüyen dal ve yaprakların bir önceki seneki yapıların üstünde yükselmesi özyinelemeli yapıya örnek verilebilir. Özyineleme, özellikle [matematiksel](#) (bileşke fonksiyon) işlemlerde sıklıkla kullanılır.



matruşka

Ayrıca Rus yapımı bir oyuncak bebek türü olan Matruşka bebeği de özyineleme yapısına örnek verebiliriz.

Dışarıdan bakıldığında tek bir bebek gibi gözüken matruşka bebekleri açıldığında içinden daha küçük boyutta ama birebir dıştaki ile aynı başka bir bebek çıkar, onu açtığınızda yine

daha küçük boyutta başka bir bebek çıkar ve bu işleme en içteki bebek çıkana kadar devam edilir.

Proje 7.5. Özyinelemeli Üs Alma

Aşağıda sol sütunda çalışma mantığı verilen 2 sayısının N. üssünü alan programı özyinelemeli fonksiyon yapısında kodlayalım.

N=5 için programın ekran çıktısı: 32 olacaktır.

2 sayısının N. üssünü alan programın çalışma mantığı: (N=5 için)

$2^5 = 2^4 * 2$ (şimdi 2^4 'i bulmalıyız)
 $2^4 = 2^3 * 2$ (şimdi 2^3 'i bulmalıyız)
 $2^3 = 2^2 * 2$ (şimdi 2^2 'i bulmalıyız)
 $2^2 = 2^1 * 2$ (şimdi 2^1 'i bulmalıyız)
 $2^1 = 2^0 * 2$ (şimdi 2^0 'i bulmalıyız)
 $2^0 = 1$ (şimdi program sonlandı)

```
"""
Us fonksiyonu: Eğer N=0 ise 1,
                Değilse Us(N-1)*2
sonucunu üretmektedir.
"""
```

Python dili kodlaması

```
#Özyinelemeli Us fonksiyonu
def Us(N):
    if N==0:
        return 1
    else:
        return 2*Us(N-1)
```

```
#Ana program
N = 5
print (Us(N))
```

Sıra Sende Etkinliği:

Yukarıdaki programı (Proje 7.5) 3 sayısının N. üssünü alacak şekilde yeniden düzenleyebilir misin? {N sayısı da kullanıcı tarafından klavyeden girilecektir.}

Proje 7.6. Öklid Algoritması

Öklid algoritmasına göre iki tam sayının OBEB'ini hesaplayan programı kodlayalım.

İki A ve B tam sayısının "Ortak Bölenlerinin En Büyüğü (OBEB)" (veya EBOB, En Büyük Ortak Bölen): A ve B'yi bölen en büyük tam sayıdır. Öklid algoritmasına göre OBEB fonksiyonu kendi içinde kendisini çağırdığı (kendisine atıf yaptığı) için özyinelemeli bir fonksiyondur.

Öklid'in OBEB algoritması

1. Başla
2. Eğer A=0 ise OBEB(0,B)=B olacağı için OBEB(A,B)=B olur.
3. Eğer B=0 ise, OBEB(A,0)=A olacağı için OBEB(A,B)=A olur.
4. Değilse (B>0 ise)
OBEB(A,B) =OBEB(B%A, A) olur.
5. Dur

Açıklama

```
Örneğin;
OBEB(27,9) = OBEB (9,27%9)
            = OBEB (9,0)
            = 9
```

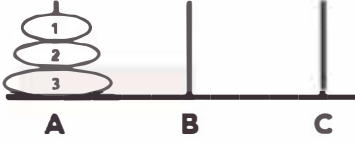
Python dili kodlaması

```
def OBEB(a, b):
    if (a == 0):
        return b
    else:
        return OBEB(b%a, a);
#Ana program
A = int(input("A sayısı:"))
B = int(input("B sayısı:"))
print ("OBEB= ", OBEB (A,B))
```

Ekran çıktısı

```
A sayısı.:27
B sayısı.:9
OBEB= 9
```

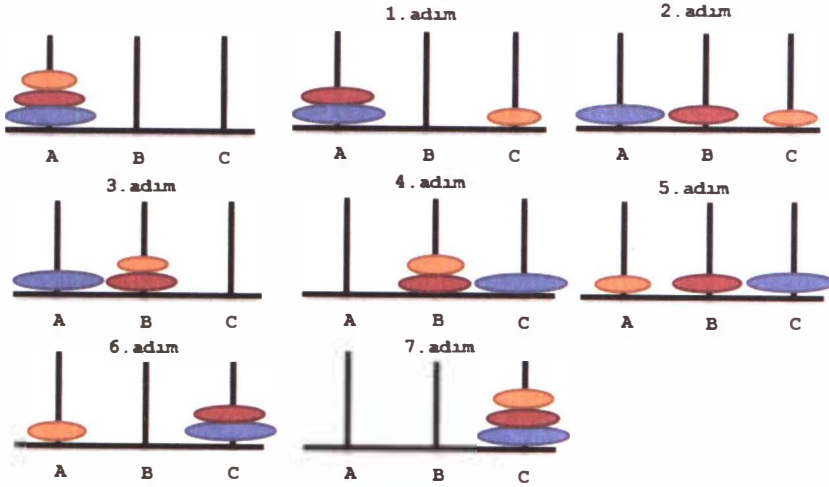
Proje 7.7. Hanoi kuleleri oyunu



Dik olarak konumlandırılmış üç çubuk (A, B, C) verilir. İlk çubukta en küçük disk en üstte olacak şekilde yerleştirilmiş farklı büyüklükte n adet disk vardır.

Oyunun amacı: A çubuğunda (sütununda) küçükten büyüğe sıralı haldeki diskleri, verilen kurallara uyarak C çubuğuna yine aynı şekilde küçükten büyüğe sıralı olacak şekilde taşımak ve bu işlemi minimum adımda (3 disk için 7 adımda) tamamlamaktır. Kurallar ise şöyledir:

- Aynı anda sadece tek bir disk taşınabilir.
- Bir disk boş bir çubuğa veya daha büyük bir diskın üzerine taşınabilir.



Şimdi bu kurallara göre A çubuğundaki diskleri C çubuğuna en az adımda taşıyacak programı kodlayalım.

Hanoi algoritması

1. Başla
2. Hanoi Fonksiyonu (disk sayısı(n), kaynak, hedef, yedek) tanımla
3. Eğer disk sayısı > 0 ise
 1. Hanoi(n-1, kaynak, yedek, hedef)
 2. Taşı disk, kaynaktan hedefe
 3. Hanoi(n-1, yedek, hedef, kaynak)
 4. Fonksiyonu sonlandır
4. Dur

Python dili kodlaması

```
def Hanoi(n, A, C, B):  
    global s #s global bir değişken  
    if n>0:  
        Hanoi(n-1, A, B, C)  
        print(s, ".adım:", A, "-->", C)  
        s=s+1  
        Hanoi(n-1, B, C, A)  
    return  
  
#ana program  
n = int(input('n degeri:'))  
s=1  
Hanoi(n, 'kaynak', 'hedef', 'yedek')  
Hanoi(n, 'A', 'C', 'B')
```

Açıklama

Programda n: disk sayısı olmak üzere
A: kaynak, C: hedeftir. B ise disklerin
geçici tutulduğu değişkendir.

Ana programda verilen 's' sayaç de-
ğişkeninin hanoi fonksiyonu içerisinde
de faaliyet gösterebilmesi için global
tanımlandığına dikkat ediniz.

Not. Hanoi fonksiyonu da kendi içinde
kendisini çağırdığı (kendisine atıf yap-
tığı) için özyinelemeli bir fonksiyondur

Örnek ekran çıktısı

```
n degeri.: 3
1 .adim: A --> C
2 .adim: A --> B
3 .adim: C --> B
4 .adim: A --> C
5 .adim: B --> A
6 .adim: B --> C
7 .adim: A --> C
```

Soru ve Cevaplarla Neler Anladık?

1 Python dilinde fonksiyon tanımlamak için hangi komut kullanılır?

```
def
```

2 Fonksiyondan geriye değer döndürmede hangi komut kullanılır?

```
return
```

Aşağıdaki karar isimli fonksiyonun x parametresinin değeri 5 ise bu fonk-
siyon hangi değeri döndürür?

```
3 def karar (x) :
    if (x%2) == 0:
        return 1
    else:
        return 0
```

0

4 Kendi kendini çağıran fonksiyonlara ne ad verilir?

Özyinelemeli (rikürsif) fonksiyon denir

5 Fonksiyonlar niçin kullanılır?

Fonksiyonlar hem kod tekrarını önler hem de programların anlaşılabilir-
liğini, hata ve kod takibini kolaylaştırır.

- 6 Aşağıdaki özyinelemeli **CimBom** fonksiyonunun n=5 için ekran çıktısı ne olur?

```
def CimBom (n):  
    if n== 0:  
        return  
    print ("Cim")  
    CimBom(n-1)  
    print ("Bom")  
#Ana program  
CimBom(5)
```

Ekrana 5 kez önce "Cim" sonra da "Bom" yazar.

- 7 Fonksiyon içinde tanımlanan değişkenlere yerel değişken denir ve bu değişkenlere tanımlandıkları fonksiyon dışından erişilmeleri mümkün değildir. Yerel bir değişken nasıl genel (global) değişken yapılır?

Yerel bir değişken başına **global** kelimesi getirilerek ("**global a**" gibi) genel değişken yapılır.

- 8 Aşağıda verilen program çıktı olarak ne üretir?

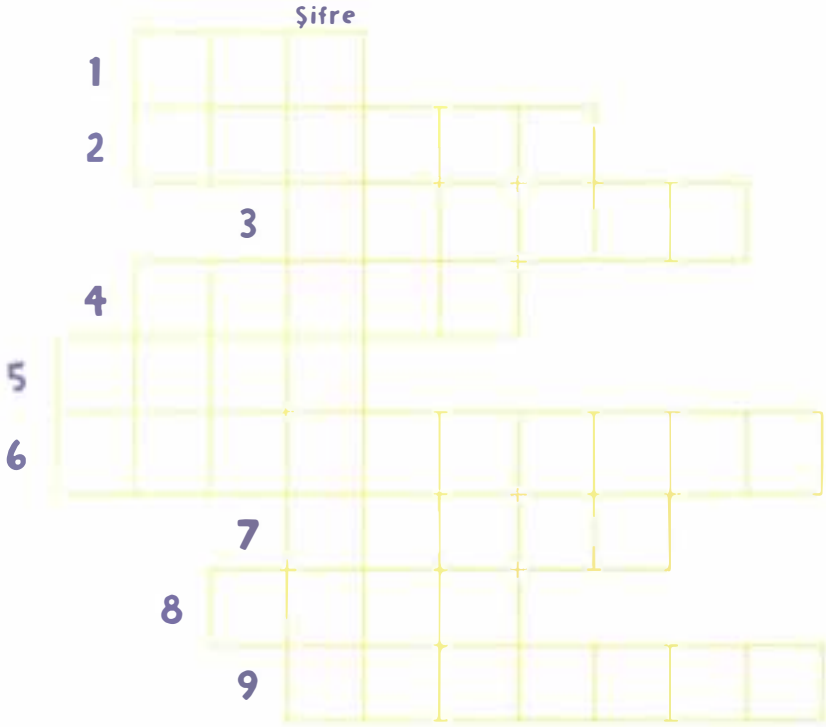
```
def Test(i):  
    print(i**2)  
#ana program  
Test(4)
```

16

- 9 Parametre ile argüman arasındaki farkı açıklayınız?

Fonksiyon tanımlanırken belirtilen girdilere "parametre", çağrılırken belirtilen girdilere ise "argüman" denir.

BULMACA



1. Fonksiyon tanımlamak için kullanılır.
2. Programın tamamında faaliyet gösteren değişken.
3. (Tersi) Fonksiyon değerini geriye döndürür.
4. "Yerel" kelimesinin eş anlamlısı.
5. Kod gövdesi olmayan (boş) fonksiyon tanımında kullanılır.
6. En genel anlamıyla bir yapının kendi kendini çağırması.
7. Genel olmayan.
8. "Hiçbir şey (iş) yapma" anlamında bir komut.
9. (Tersi) Fonksiyon çağrılırken kullanılan girdi değerleri.

Aşağıya bulduğunuz şifreyi kodlayınız.

--	--	--	--	--	--	--	--	--	--

Bölüm

8

String

(Karakter Dizisi) İşlemleri

PYTHON DİLİNDE

"Çift tırnak",
'tek tırnak',
veya
''' üç tırnak '''

içerisine yazılan her şey string (karakter dizisi)
kabul edilir

Giriş

Yazılım dünyasında karakter dizileri string olarak adlandırılır. Karakter dizileri, birden fazla karakterin 'tırnak içerisinde alınması' ile oluşur.

Python dili çift tırnak "...", tek tırnak '...' veya üç tırnak """ ...""" içerisinde yazılan herşeyi string (karakter dizisi) olarak algılar. String (karakter dizisi) değişkenleri str veri tipi ile ifade edilirler.

Tek bir karakterin veya karakter dizisinin (stringin) Python dilinde nasıl tanımlanacağı aşağıdaki tabloda örneklerle gösterilmektedir.

Python dili string tanımlama şekilleri	Veri tipi: str
1. <code>k='1'</code> 2. <code>k="""</code> 3. <code>k= 'PYTHON'</code> 4. <code>k= "PYTHON"</code> 5. <code>k= '''PYTHON'''</code>	""" tek veya çift tırnak arasına yazılan herşey string'tir. Örneğin yandaki kod satırları için <code>print(type(k))</code> komutu uygulandığında; <code><class 'str'></code> ekran çıktısı elde edilir. ...

İndis (index) Kavramı

Ad =	"	M	e	r	h	a	b	a		A			"
pozitif indis		0	1	2	3	4	5	6	7	8	9	10	
negatif indis		-11	-10	-9	-8	-7	-6	-5	-4	-3	-2	-1	

String işlemlerde veya listelerde/dizilerde değişken ismini takiben köşeli parantez [] içerisinde belirtilen numaraya indis adı verilir. Bir stringin (karakter dizisinin) veya listenin her bir elemanına indis numarası ile erişilir.

Not. Çoğu programlama dilinin aksine Python, negatif indis tanımlamasına izin vermektedir.

Python dilinde;

- pozitif indis numaraları sıfırdan başlayarak soldan sağa doğru verilirken, negatif indis numaraları birden başlayarak sağdan sola doğru verilir.
- Bir dizi veya listenin ilk elemanı; 0. indisli eleman, son elemanı ise karakter sayısının 1 eksiği veya -1. indisli elemandır.

Örneğin Ad= "Merhaba Ali" şeklinde tanımlanan bir string ifade esasen 11 elemanlı bir karakter dizisidir. Bu karakter dizisinin ilk elemanı Ad[0], son elemanı ise A[10] veya A[-1] dir.

Bir string ifadenin herhangi bir karakterine erişmek için Ad[indis numarası] şeklindeki gösterim kullanılır.

```
>>> Ad = "Merhaba Ali"
>>> Ad[0]
'M'
>>> Ad[-11]
'M'
>>> Ad[2]
'r'
>>> Ad[-1]
'i'
```

Örneğin yandaki programda Ad[0] veya Ad[-11] ifadesi; Ad karakter dizisinin 0. indisli ilk elemanı olup, bu örnek için değeri M'dir.

Soru: Karakter dizisinin herhangi bir elemanı Ad[2]= 'y' şeklinde doğrudan değiştirilebilir mi?

Cevap: Hayır, Python dilinde stringler doğrudan değiştirilemez (immutable) tipte verilerdir. Peki, ne yapılabilir? Python dili string işlemler için çok yetenekli fonksiyonlar/metotlar içerir. Bu metotlar veya bazı operatörler (+, *, :) yardımı ile bir string dilimlenebilir sonra da birleştirilebilir.

İki Stringi Birleştirme

İki string ifadeyi birleştirmek için "+" operatörü kullanılır.

Proje 8.1. String Birleştirme

İki stringi birleştirelim.

Python dili kodlaması

```
A = "Python"
B = "Programlama"
C = A + B
print(C)
```

Ekran çıktısı

```
PythonProgramlama
```

Stringi Dilimleme (Parçalara Ayırma)

Karakter dizilerini veya listeleri parçalara ayırmak (dilimlemek) için ":" operatörü kullanılır. Kullanım şekli;

stringAdı [ilkIndis: sonIndis: adımMiktari]

Bu kullanımda stringin ilkIndis değeri işleme dâhil edilirken, sonIndis değeri edilmez. Eğer ilk indis değeri belirtilmezse "0", son indis değeri belirtilmezse stringin toplam uzunluğu kabul edilir. Adım miktarı belirtilmez ise değeri "1" kabul edilir.

String dilimleme örnekleri;

: Kullanımı	Açıklama	Örnek kullanım	Ekran çıktısı
S[:]	Karakter dizisini olduğu gibi al.	S="Python?" print(S[:])	Python?
S[3:]	3. indisli (4.) elemandan dizi sonuna kadar al.	print(S[3:])	hon?
S[:3]	Başlangıçtan itibaren ilk 3 elemanı al.	print(S[:3])	Pyt
S[1:5]	1.indisli (2.sıradaki) elemandan başla ve 5.indisli elemana kadar al (5.indis dâhil değil).	print(S[1:5])	ytho
S[1::2]	1.indisli (2.sıradaki) elemandan başla ve ikişer atlayarak dizi sonuna git.	print(S[1::2])	yhn
S[1:6:3]	1.indisli (2.sıradaki) elemandan başla ve 6.indisli elemana kadar 3'er 3'er git.	print(S[1:6:3])	yo
S[::-1]	S stringini ters çevirir.	print(S[::-1])	?nohtyP

Proje 8.2. String Dilimleme

"Python" stringini "Pithon" olarak değiştirelim.

Bu işlemi dilimleme operatörünü ':' kullanarak; değiştirmek istediğimiz karakterden öncesi ve sonrasındaki karakterleri ayırır, araya değiştirilecek karakteri yerleştirir ve sonra da hepsini birleştirerek çözebiliriz. Fakat daha kolay bir şekilde replace() fonksiyonunu kullanarak da çözebiliriz.

Python dili kodlaması -1	Ekran çıktısı
<pre>S="Python" S2=S[:1] + 'i'+S[2:] print(S2)</pre>	Pithon

Python dili kodlaması -2	Ekran çıktısı
<pre>S="Python" S3=S.replace('y', 'i') # 'y'--> 'i' yapar print(S3)</pre>	Pithon

Bir String İfadeyi Güncelleme/Değiştirme

Bir stringin karakterlerini değiştirmek/güncellemek için bir önceki projede açıklandığı gibi hem dilimleme operatörü hem de replace() fonksiyonu kullanılabilir. Kullanım şekli:

stringAdı.replace ("eski", "yeni") #eski, yeni olur.

Proje 8.3. String Güncelleme

Bir stringin içeriğini (belli bir karakterden sonrasını değiştiren) güncelleyen programı kodlayalım.

Python dili kodlaması -1	Ekran çıktısı
<pre>ad = 'Refahiye-Hanzar' yeniAd= ad[:9] + 'Teknecik' print ("Güncellendi: ", yeniAd)</pre>	Güncellendi: Refahiye-Teknecik
Python dili kodlaması -2	Ekran çıktısı
<pre>ad = 'Refahiye-Hanzar' yeniAd= ad.replace('Hanzar', 'Teknecik') print ("Güncellendi: ", yeniAd)</pre>	Güncellendi: Refahiye-Teknecik

Bir Stringin Tamamını Veya Belli Bir Kısımını Silme

Python dilinde bir stringin (karakter dizisinin) elemanlarını doğrudan değiştiremediğimiz gibi silemeyiz de. Fakat dilimleme operatörü ile belli bir kısmını silebiliriz.

Proje 8.4. Karakterleri Sondan Başa Doğru Silme

Bir Stringin karakterlerini sondan başa doğru silen programı kodlayalım.

Python dili kodlaması	Ekran çıktısı
<pre>metin = "Python" print (metin) for a in range(1,6): print (metin[:-a])</pre>	Python Pytho Pyth Pyt Py P

Sıra Sende Etkinliği-1:

Yandaki ekran çıktısını elde etmek için yukardaki programda nasıl bir değişiklik yapmak gerekir?

P
Py
Pyt
Pyth
Pytho
Python

Bir Stringin Karakterlerine For Döngüsü İle Erişim

Herhangi bir String ifadenin karakterlerine aşağıdaki örnekteki gibi for-in döngüsü ile erişilebilir.

Proje 8.5. Karakterleri Alt Alta Yazdıralım

Bir string ifadenin karakterlerine döngü ile erişerek alt alta ekrana yazdıralım.

Python dili kodlaması	Ekran çıktısı
<pre>for k in "Dizi": print(k)</pre>	<pre>D i z i</pre>

Bir Stringi Ayırıştırarak Listeye Dönüştürme

Bir stringi olduğu gibi veya belli bir karaktere göre ayırıştırarak listeye dönüştürmek için split (ayraç) komutu kullanılır. Örneğin;

```
'1,2,3'.split(',')
```

Çıktısı ['1', '2', '3'] şeklinde bir listedir.

Proje 8.6. Metni Ayırıştırma

Bir string metni içerisindeki "," virgöl karakterlerinden ayırıştırarak listeye dönüştüren programı kodlayalım.

metin değişkeni str tipinde iken ayrıştırılmış hali olan liste değişkeni list tipindedir.

Python dili kodlaması	Ekran çıktısı
<pre>metin = "* ** *** ****" liste = metin.split(",") for x in liste: print (x)</pre>	<pre>* ** *** ****</pre>

Bir Stringin Karakter Sayısını Öğrenme

Bir string ifadenin kaç karakterden oluştuğunu len komutu ile öğrenebiliriz. String ifade içerisindeki belli bir karakterden kaç adet olduğu bilgisini ise count komutu ile öğrenebiliriz.

Proje 8.7. Bir Cümlelin Karakter Sayısını Bulalım

Bir stringin karakter sayısını (karakter uzunluğunu) veren programı kodlayalım.

Python dili kodlaması

```
cumle = "Merhaba"
print ("Karakter sayısı: ", len(cumle))
print (" 'a' harfi sayısı: ", cumle.count('a'))
```

Ekran çıktısı

```
Karakter sayısı: 7
'a' harfi sayısı: 2
```

İki String İfadeyi Karşılaştırma

İki stringi veya karakteri karşılaştırmak için eşit == veya eşit değil != operatörleri ile kullanılır. Örnek kullanım;

```
>>> 'Ali' == 'Ali' #eşdeğeri: 'Ali' is 'Ali'
True
```

Proje 8.8. Parola Sorgulama

Şifre/parola sorgulaması yapan bir programı kodlayalım.

Şifre'nin "elma" olduğu varsayılacak ve kullanıcıya şifreyi yanlış girmesi halinde 3 hak verilecektir. Programda kullanıcıya tanınan haklar, sayac değişkeninde tutulmaktadır. Eğer sayac>=3 ise ekrana "Hakkınız bitti" mesajı yazdırılacaktır.

Python dili kodlaması

```
sayac=0
while True:
    sifre=input("Parola.:")
    if sifre=="elma":
        print ("Şifre doğru")
        break
    sayac+=1
    if sayac>=3:
        print ("Hakkınız bitti")
        break
```

Ekran çıktısı

```
Parola.:123
Parola.:elma
Şifre doğru
```

String İfade İçerisine Değişken Değerini Yazdırma

Bir string ifade içerisine değişken değerini yazdırmak için `format()` komutu kullanılır. Örnek kullanım;

```
>>> Pi=3.14
>>> "Pi sayısının değeri {} dir".format(Pi)
'Pi sayısının değeri 3.14 dir'
>>> #Eşdeğeri: print(f"Pi sayısının değeri {Pi} dir")
```

Not. Değişken değerinin string içerisindeki süslü parantezler {içerisine} yazıldığına dikkat ediniz.

String İfadeyi Ters Çevirme

Bir stringi ters çevirmek için dilimleme yöntemi kullanılabileceği gibi `reversed()` ve `join` komutları birlikte kullanılabilir. Örnek kullanım:

```
>>> ''.join(reversed("ali"))
'ila'
>>> "ali"[::-1]
'ila'
```

Proje 8.9. Palindrom mu?

Girilen bir metnin Palindrom olup olmadığını sorgulayan programı kodlayalım.

Palindrom, tersten yazılışı da kendisi ile aynı olan metinlere denir. Örneğin "ada" sözcüğü palindrom iken "eda" sözcüğü palindrom değildir.

Python dili kodlaması

```
s=input("Bir metin gir.:")
if s == ''.join(reversed(s)): #s=s[::-1]:
    print(s, ":palindrom")
else:
    print(s, ":palindrom değil")
```

Ekran çıktısı

```
Bir metin gir.:adanada
adanada :palindrom
```

Not.

`if s == ''.join(reversed(s)):` yerine `if (s==s[::-1]):` de yazabilirdik.

Bir Stringin Harflerini (Büyük/ Küçük Harf) Dönüştürme

Bir string ifadenin tamamını veya belli bir karakterini küçük veya büyük harfe dönüştürmek için aşağıdaki komutlar kullanılır;

Dönüştürme komutu	Açıklama	Örnek kullanım
<code>s.lower()</code>	s'yi küçük harfe dönüştürür.	<code>print("HANZAR".lower())</code> # hanzar
<code>s.upper()</code>	s yi büyük harfe dönüştürür.	<code>print("hanzar".upper())</code> # HANZAR
<code>s.swapcase()</code>	s'nin küçük harflerini büyük, büyük harflerini küçük harfe dönüştürür.	<code>print("HanzaR".swapcase())</code> # hANZAr
<code>s.capitalize()</code>	s' nin ilk karakterini büyük harfe dönüştürür.	<code>print("hanzar".capitalize())</code> # Hanzar

Proje 8.10. Sesli ve Sessiz Harfleri Sayalım

Bir metindeki sesli ve sessiz harf sayısını hesaplayan programı kodlayalım.

Python dili kodlaması	Ekran çıktısı
<pre> cumle="Devletin dini adalettir." sesli = "aeiouü" sessiz = "bcçdfgğhijklmnprsstvyz" sayac1=sayac2=0 for harf in cumle.lower(): if harf in sesli: sayac1 = sayac1 + 1 if harf in sessiz: sayac2 = sayac2 + 1 print ("Sesli harf sayısı: ", sayac1) print ("Sessiz harf sayısı: ", sayac2) </pre>	<pre> Sesli harf sayısı: 9 Sessiz harf sayısı: 12 </pre>

Bir String İçerisinde Başka Bir Stringi Arama

Bir string ifade içerisinde başka bir string ifadenin olup olmadığı in veya not in komutları ile sorgulanabilir. Kullanım şekli:

```

>>> "S" in "GS"      #'S', 'GS' içerisinde var mı?
True
>>> "s" not in "fb"  #'s', 'fb' içerisinde yok mu?
True

```

Proje 8.11. Kelime Arama

Bir metin (cümle) içerisinde aranan kelimenin olup olmadığını sorgulayan bir programı kodlayalım.

Metinde aranacak kelime ara değişkenine atandı. Bu kelime değiştirilerek program test edilebilir.

Python dili kodlaması

```
cumle = "Devletin dini adalettir."  
ara = "adalet"  
if ara in cumle:  
    print ("Evet,", ara, "vardır.")  
else:  
    print ("Hayır,", ara, "yoktur.")
```

Ekran çıktısı

Evet, adalet vardır.

Sıra Sende Etkinliği-2

Bir metindeki kelime ve cümle sayılarını ekranda gösteren programı kodlayabilir misin?

İpucu: Her kelime boşluk ' ' karakteri ile cümle ise nokta '.' karakteri ile sonlanmaktadır.

Soru ve Cevaplarla Neler Anladık?

1 Karakter dizilerine ne denir?

String

2 Karakter dizilerinin veri tipi nedir?

str

3 Python dilinde string bir ifade nasıl tanımlanır?

Çift tırnak "...", tek tırnak '...' veya üç tırnak "..."" içerisinde yazılan her şey string bir ifadedir.

4 Indis nedir?

String işlemlerde veya listelerde/dizilerde değişken ismini takiben köşeli parantez [] içerisinde belirtilen numaraya indis adı verilir.

5 Python dilinde indis numaralandırması nasıl yapılır?

Python dilinde indis numarası verilirken ya solda sağa (pozitif sayılar) ya da sağdan sola (negatif sayılar) doğru numaralandırma yapılabilir. [0]. indis ilk karakteri [-1]. Indis ise son karakteri verir.

- 6 Aşağıdaki programın çalıştırılmasından sonra 'ad' değişkeni en son hangi değeri alır?

```
ad = "Ali"  
ad = ad + "Veli"  
ad = ad[::-1]
```

"ileViLA"

- 7 Aşağıdaki programın çalıştırılmasından sonra 'j'nin değeri ne olur?

```
i = "9"  
j = i + i
```

99

- 8 Aşağıda verilen Python kodu çıktı olarak ne üretir?

```
print("ada".replace("a","e"))
```

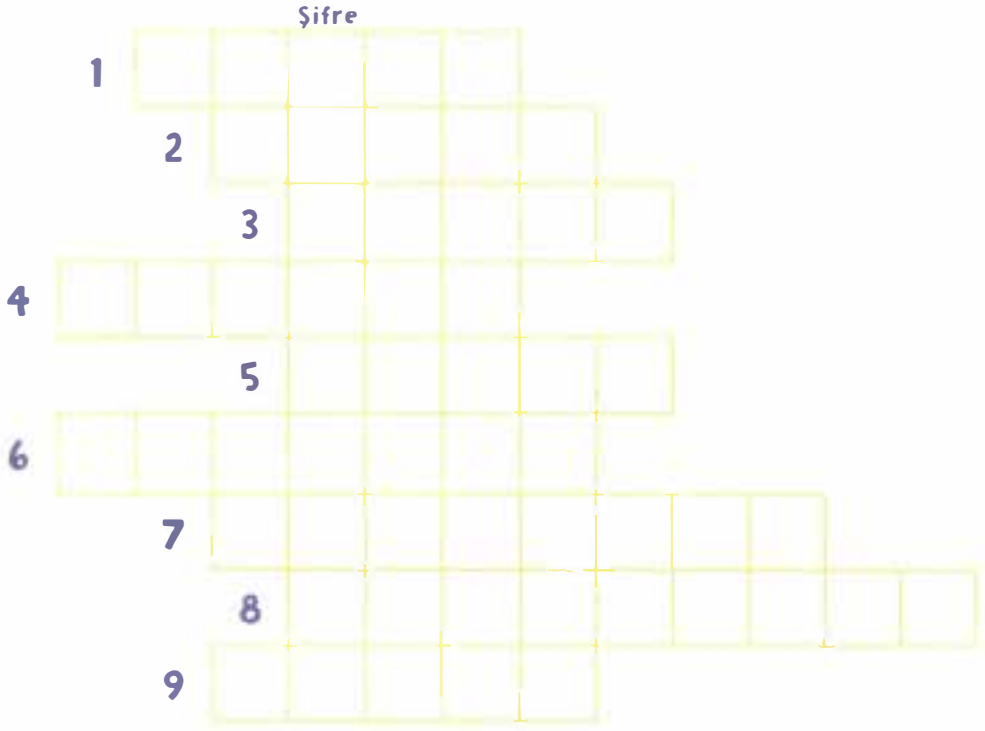
ede

- 9 Aşağıda verilen Python kodu çıktı olarak ne üretir?

```
s="levent ekrem"  
print(s[:-5])
```

Levent

BULMACA



1. Bir string veya listenin her bir elemanına erişmek için kullanılır.
2. (Tersi) Türkçe "ayraç" anlamında bir komut.
3. Küçük harfe dönüştürür.
4. Karakter dizisi.
5. Aşağıdaki programın çıktısı;

```
s="MARMARA"  
print(s[:-2])
```
6. Eskiye, yenisi ile değiştirme.
7. Bir stringi ters çevirmede kullanılan bir metot.
8. (Tersi) Tersten yazılışı da kendisi ile aynı olan metin.
9. `print("madam".replace("a","e"))` işlemi.

Aşağıya bulduğunuz şifreyi kodlayınız.

--	--	--	--	--	--	--	--	--	--

DİLİMLEME

Bölüm

9

Diziler ve Listeler



[Listeler, bir sıralı elemanlar dizisidir.]

Dizi Nedir?

Artarda sıralanmış aynı türden elemanlardan oluşan bir grup objeye dizi (array) adı verilir. Dizi örnekleri:

Elma_Dizisi: 

Sayı_Dizisi:

5	7	9	11	13
---	---	---	----	----

Karakter_Dizisi:

'S'	'E'	'L'	'A'	'M'
-----	-----	-----	-----	-----

İndis

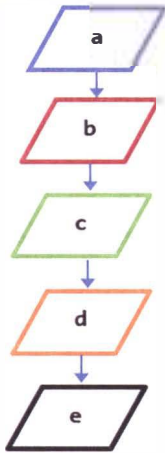
0	1	2	3	4
---	---	---	---	---

Bir dizinin her bir elemanı ayrı bir değişken (veya nesne) gibi ele alınır. Bir dizi elemanına `DiziAdı[indis]` şeklinde erişilir. Örneğin yukarıdaki sayı dizisinin 3. indisli elemanı, `Sayı_Dizisi[3]` şeklinde ifade edilir ve değeri (11)'dir.

Diziler Niçin Kullanılır?

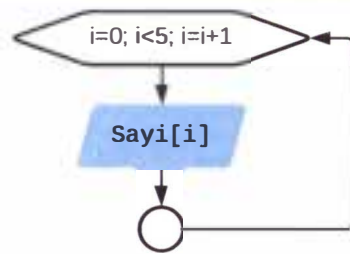
Klavyeden girilen 5 farklı sayıyı almak ve bilgisayarda saklamak istiyorsunuz diyelim. Ne yapmanız gerekir?

Dizi olmasaydı?



5 adet değişken tanımlamamız ve tek tek değer atamamız gerekirdi.

Dizi olsaydı?



Tek bir dizi değişkeni tanımlar ve bir döngü içerisinde otomatik değer ataması yapabiliriz.

Not. 5 değil de 555 farklı sayı girişi için hangi yöntemi kullanmak sizce daha pratik olacaktır?

Listeler

Python dilinde diziler, listeler şeklinde ifade edilir. Listeler, bir sıralı elemanlar dizisidir. [] köşeli parantezler arasına yazılan sıralı elemanlar dizisi, birer listedir. Listelerin veri tipi <list> dir. Dolayısıyla list sarmalına alınan her veri yapısı birer listedir.

Soru: Diziler ile Listeler arasında bir fark var mıdır?

Cevap: Temel fark, diziler aynı tip (cins) elemanlardan oluşurken listeler farklı tip elemanlardan oluşabilir. Örnek vermek gerekirse bir önceki sayfada ki sadece elmalardan oluşan Elma_Dizisi aynı zamanda bir listedir fakat aşağıdaki gibi farklı meyvelerden oluşan bir Meyve_Listesi bir listedir ama dizi değildir.

Meyve_Listesi : [     ]

Python dili liste tanımlama şekilleri	Açıklama
1. L = ['P' , 'Y' , 'T' , 'H' , 'O' , 'N']	Karakter Listesi
2. L = ["Bol" , "Han" , "Zar" , "Muz"]	String Listesi
3. L = [1,3,5,7,9,11,13,15]	Tam sayı Listesi
4. L = []	Boş Liste
5. L = ['A' , 1 , [1,2,3] , "Han" , 3.14]	Bir liste farklı tipteki verilerden hatta iç içe başka listelerden oluşabilir.

Şimdi listeler üzerinde yapabileceğimiz işlemleri ve bu işlemler için kullanılan fonksiyonları alt başlıklar altında açıklayalım.

Liste elemanlarına erişim

Liste ismini takiben parantez içerisinde belirtilen numaraya indis adı verilir. Bir listenin elemanlarına, aynı stringlerde olduğu gibi indis numarası ile doğrudan

erişebilir ve değiştirebilirsiniz. Listeler **değiştirilebilir** (mutable) özellikte veri yapılarıdır.

Aşağıdaki tablodan da görüleceği üzere listelerin indis numarası sol baştan 0 ile sağ baştan ise -1 ile başlar. Dolayısıyla bir L listesinin ilk elemanına L[0], son elemanlarına ise L[-1] ifadeleri ile erişilir.

	1 eleman	2 eleman	3 eleman	4 eleman	5 eleman	7 eleman	8 eleman
L listesinin elemanları	1	3	5	7	9	11	13
Liste elemanlarına erişim	L[0]	L[1]	L[2]	L[3]	L[4]	L[5]	L[6]
'ListedeL[indis no]'	L[-7]	L[-6]	L[-5]	L[-4]	L[-3]	L[-2]	L[-1]

Listedeki Bir Elemanın İndis Numarasını Öğrenme

Listedeki bir elemanın indis numarasını öğrenmek için `index()` fonksiyonu kullanılır. Örnek kullanım ve açıklama aşağıda verilmiştir;

```
L.index(X)  X elemanının indisini döndürür.  L = [1, 3, 5, 7]
print (L.index(5))  #Çıktı: 2
```

Liste Dilimleme

Aynı karakter dizilerinde olduğu gibi listeleri de parçalara ayırmak (dilimlemek) için ":" operatörü kullanılır. Örnek kullanım;

```
>>> L = [1,2,3,4,5,6,7,8,9,10]
>>> L[:2]  #2 şer atlayarak dilimler
#Çıktı: [1, 3, 5, 7, 9]
```

Listeye Eleman Ekleme

Bir listenin sonuna "`append()`" fonksiyonu ile yeni elemanlar eklenebilir. Kullanım şekli;

```
liste.append(9)  #listenin sonuna 9 sayısını ekler.
```

Listenin sonuna değil de liste elemanları arasına yeni bir eleman eklemek için "`insert()`" fonksiyonu kullanılır. Kullanım şekli;

```
liste.insert(2, 9)  #listenin 2. indisini 9 yapar ve listeyi kaydırır.
```

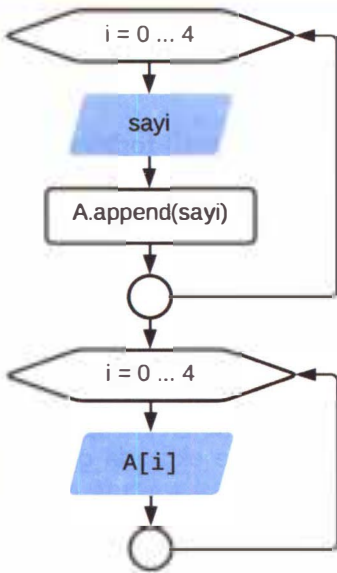
L.append(x)	Liste sonuna x elemanını ekler. Eşdeğeri: L[len(L):] = [x] dir.	L = [1, 3, 5, 7] L.append(9) print (L) #Çıktı: [1,3,5,7,9]
L.insert(i, X)	X elemanını L listesinin i. indisli elemanı yapar.	L = [1, 3, 5, 7] L.insert(2,9) print (L) #Çıktı:[1,3,9,5,7]

Proje 9.1. Listeye Eleman Ekleme

Kullanıcı tarafından girilen 5 adet sayıyı A listesine aktarıp, şekildeki gibi ekranda gösteren programı kodlayalım.

```
Sayı.:5
Sayı.:6
Sayı.:7
Sayı.:8
Sayı.:9
A[ 0 ]= 5
A[ 1 ]= 6
A[ 2 ]= 7
A[ 3 ]= 8
A[ 4 ]= 9
```

Akış Şeması



Python dili kodlaması

```
A = [] #boş liste
for i in range(0,5):
    sayı = int(input('Sayı.:'))
    A.append(sayı) #girilen sayıyı
                  #listeye ekle

#döngü sonu

#Liste elemanlarını alt alta yazdıralım
for i in range(0,5):
    print ("A[" , i , "]= ",A[i])
#döngü sonu
```

Not. Dizi elemanlarını istenen formda yazdırmak için ayrı bir for döngüsü kullanılır.

Listenin Eleman Sayısını Öğrenme

Bir listenin eleman sayısını öğrenmek için len() fonksiyonu kullanılır. Bir listedeki herhangi bir elemanın adetini öğrenmek için ise count() fonksiyonu kullanılır.

len(L)	Listedeki toplam eleman sayısını verir.	L = [1, 3, 5, 7] print (L.count(5)) #Çıktı: 4
L.count(X)	Listedeki 'X' elemanlarının sayısını/adedini döndürür.	L = [1, 3, 5, 7] print (L.count(5)) #Çıktı: 1

İki Listeyi Birleştirme

İki listeyi birleştirmek için "+" operatörü ya da extend() fonksiyonu kullanılır.

L1.extend(L2)	Mevcut L1 listenin sonuna L2 listesini ekler. Eşdeğeri: L1 = L1 + L2 dir.	L1 = [1, 3, 5, 7] L2 = [9, 11, 13] L1.extend(L2) print (L1) #Çıktı: [1,3,5,7,9,11,13]
----------------------	--	---

Bir Listeyi Ters Çevirme

Bir listenin elemanlarını tamamen ters çevirmek için reverse() fonksiyonu kullanılır.

L.reverse()	Liste elemanlarını ters sırada oluşturur.	L = [1, 3, 5, 7] L.reverse() print(L) #Çıktı: [7,5,3,1]
--------------------	---	--

Bir Listenin En Küçük ve En Büyük Elemanını Öğrenme

Bir listenin en küçük elemanı min(), en büyük elemanı ise max() fonksiyonu ile öğrenilebilir. Örnek;

```
L = [3,5,7,2,9,6]
print ( min(L) , max(L) ) #Çıktı: 2 9
```

Proje 9.2. A Dizisini B Dizisine Ters Sırada Kopyalama

A dizisinin elemanlarını, ters sırada B dizisine kopyalayan ve ekranda gösteren programı kodlayalım.

Python dili kodlaması	Ekran çıktısı
A = ['a','b','c','d','e','f'] A.reverse() #listeyi ters çevirir B = A #A listesini B'ye aktarır print (B) #B listesini ekranda gösterir	['f', 'e', 'd', 'c', 'b', 'a']

Eğer bir listenin elemanlarını ekranda gösterirken köşeli parantezlerin gözükmesi isteniyorsa liste adının önüne aşağıdaki gibi "*" operatörü getirilir.

Python dili kodlaması: devamı

```
print (*B)
```

Ekran çıktısı

```
f e d c b a
```

Bir Stringi (Karakter Dizisini) Listeye Dönüştürme

Bir string ifadeyi liste haline dönüştürmek için list() fonksiyonu kullanılır.

Kullanım şekli;

```
L = [] #boş liste
L = list("Python") #listeye dönüşür. L=['P','y','t','h','o','n'] olur.
```

Bir Listedden Eleman Silme Veya Listeyi Tamamen Boşaltma

Bir listeden;

- Belli bir elemanı silmek için remove(eleman),
- İndis numarası belirtilen (örneğin 3. indisli) bir elemanı silmek için L.pop(3),
- Bir L listesini tamamen boşaltmak için L.clear(),

fonksiyonları kullanılır.

L.remove(x)	Listedeki ilk 'x' elemanını siler.	L = [1, 3, 5, 7] L.remove(3) #3'ü siler print (L) #Çıktı:[1,5,7]
L.pop(i)	Listeden i indis numarası verilen elemanı siler.	L = [1, 3, 5, 7] L.pop(3) #7'yi siler print(L) #Çıktı:[1,3,5]
L.clear()	Listedeki tüm elemanları siler. Eşdeğeri: del L[:] dir.	L = [1, 3, 5, 7] L.clear() print (L) #Çıktı:[]

Proje 9.3. Cümleden Sesli Harfleri Kaldırma ve Listeye Dönüştürme

Kullanıcının girdiği bir cümleyi listeye dönüştüren ve bu listeden sesli harfleri çıkaran (sessiz harflerden bir liste oluşturan) programı kodlayalım.

Python dili kodlaması

```
L = [] #boş liste
cumle=input('Bir Cümle Giriniz.:')
L=list(cumle) #listeye dönüştür
print ("Sessiz Harfler Listesi:")
for k in L:
    if k in 'aeioöü':
        L.remove(k) #sesli harfi
                        listeden sil
#döngü sonu
print (L) #listeyi yaz
```

Ekran çıktısı

```
Bir Cümle Giriniz.:Denemekten-
Korkma
Sessiz Harfler Listesi.:
['D', 'n', 'm', 'k', 't', 'n',
'K', 'r', 'k', 'm']
```

Listelerde Arama

Sıralı ya da sırasız bir listedeki herhangi bir elemanın yerinin bulunması işlemine arama denir. Bir liste içerisindeki elemanlar belirli bir anahtar (key) ifadeye göre aranır.

Python dilinde arama işleminin en basit çözümü;

"x in L" şeklinde x elemanını L listesinde aramaktır.

Daha öncede açıklandığı üzere in operatörü, "eğer x, L içerisinde varsa True, yoksa False" sonucunu üretir.

Proje 9.4. Listedeki Yeri Bulma

Bir A listesinde aranan sayı varsa sayının listedeki yerini ve sırasını veren programı kodlayalım.

Python dili kodlaması

```
A = [2, 3, 5, 8, 9, 12, 34, 6, 7]
Ara=int(input("Aranan Sayı: "))
if Ara in A:      #True ise
    i=A.index(Ara) #Arananın indisi
    print("Aranan", i+1, ". sırada bulundu")
else:            #False ise
    print ("Aranan bulunamadı.")
```

Ekran çıktısı

```
Aranan Sayı: 8
Aranan 4 . sırada bulundu
```

Liste Elemanlarını Sıralama

Bir listenin elemanlarını küçükten büyüğe sıralamak için sort() fonksiyonu kullanılır.

L.sort()

Aynı tip elemanlardan oluşan listenin içeriğini küçükten büyüğe doğru sıralar.

```
L = [3, 7, 1, 9]
```

```
L.sort()
print(L)  #Çıktı: [1,3,7,9]
```

Sıralama işlemini hazır sort() fonksiyonu yerine kendi yazacağınız bir sıralama algoritması ile de yapabilirsiniz. Bilgisayar mühendisleri tarafından yazılan birçok farklı (kabarcık, araya eklemeli, seçmeli, kabuk, hızlı, ...) sıralama algoritması vardır.

Proje 9.5. Liste Elemanlarını Sıralama

Bir string ifadeyi listeye dönüştürüp liste elemanlarını küçükten büyüğe doğru sort() fonksiyonu ile sıralayan programı kodlayalım.

Python dili kodlaması

```
S = "6539421" #karakter dizisi (string)
A = list(S) #listeye dönüştü
A.sort() #A listesi sıralandı
print (A) #sıralı hali ekrana yazdırıldı
```

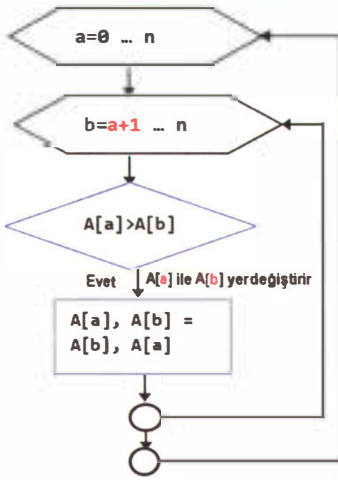
Ekran çıktısı

```
['1', '2', '3', '4', '5', '6', '9']
```

Proje 9.6. Kabarcık (Bubble) Sıralama Algoritması

Bir string ifadeyi listeye dönüştürüp liste elemanlarını küçükten büyüğe doğru kabarcık sıralama (bubblesort) algoritmasına göre sıralayan programı kodlayalım.

Kabarcık sıralama algoritması



Python dili kodlaması

```
#kabarcık sıralama algoritması
def benimSort(A):
    n = len(A) #A listesinin eleman sayısı
    for a in range(0,n):
        for b in range(a+1,n):
            if A[a]>A[b]:
                #yer değiştir
                A[a], A[b] = A[b], A[a]

#Ana program
S = "6539421"
A = list(S)
benimSort(A) #A listesini sırala
print (A) #sıralı hali ekrana yaz
```

Bir üstteki program ile aynı ekran çıktısı elde edilir.

Sıra Sende Etkinliği:

Sen de Proje 9.6'daki benimSort() fonksiyonunu farklı bir sıralama algoritması (seçmeli ya da araya eklemeli) ile yeniden kodlayabilir misin?

enumerate() Fonksiyonu ile Liste Elemanlarını Numaralandırma

enumerate() fonksiyonu, kelime anlamına uygun olarak verileri numaralandırmak amacıyla kullanılır. Bir listedeki sıralı elemanların hem eleman değerini hem de indis numaralarını (sırasını) öğrenmek için kullanılan bir fonksiyondur. Genellikle for döngüsü ile kullanılır. Kullanım şekli:

```
for indisi, eleman_değeri in enumerate(liste):
```

Proje 9.7. Haftanın Günleri

Haftanın günlerinin rakamsal karşılığını veren programı kodlayalım.

Python dili kodlaması	Ekran çıktısı
<pre>gun= ['Pazartesi', 'Salı', 'Çarşamba', 'Perşembe', 'Cuma', 'Cumartesi', 'Pazar'] #enumerate() ile indisini ve değerini aldık for i, deger in enumerate(gun): print (str(i+1) + ".gün: " + deger)</pre>	<pre>1.gün: Pazartesi 2.gün: Salı 3.gün: Çarşamba 4.gün: Perşembe 5.gün: Cuma 6.gün: Cumartesi 7.gün: Pazar</pre>

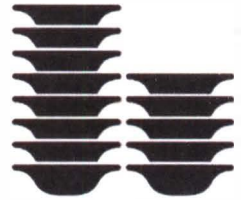
Açıklama: Yukarıdaki for döngüsünde, enumerate() fonksiyonunun geriye döndürdüğü elemanın indis numarasını "i" değişkeni ile değerini ise "deger" değişkeni ile alarak ayrı ayrı ekrana yazdırdık. Biliyorsunuz, indis numaraları 0'dan başlar. Biz de ekrana listedeki ilk günü, "1.gün" olarak yazdırmak için indis değerini i+1 şeklinde 1 artırdık.

Listelerden Yığın (Stack) Oluşturma

Yığın (stack), eleman ekleme ve çıkarma işlemlerinin listenin en son elemanı üzerinden gerçekleştiği özel bir yapıdır. Yığın, Son giren - İlk çıkar (Last In – First Out (LIFO)) mantığı ile çalışır.

Yığının çalışma mantığını anlatan günlük hayatta karşılaştığımız birkaç örnek;

- Mutfak rafındaki üst üste yerleştirilmiş tabaklar
- Bir masa üzerine üst üste yerleştirilmiş kitaplar



Mutfaktaki rafta üst üste yerleştirilmiş tabaklardan biri alınmak istendiğinde doğal olarak en üstteki tabak alınır veya bir tabak eklenmek istendiğinde, o tabak en üstteki tabağın üzerine yerleşir.

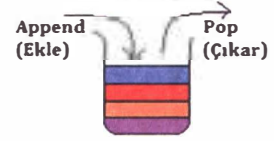
Yığına atılan ilk eleman en altta kalır. Dolayısıyla sadece en son eklenen (en üstteki) elemana erişilebilir. Bu özelliği itibari ile yığınlar, üst tarafı açık alt tarafı kapalı sıralı listelere benzetilebilir.

Yığın yapılarında iki temel işlem söz konusudur;

- Yığına eleman ekleme: append()
- Yığından en son eklenen elemanı çıkarma: pop() işlemleridir.

üst	6
	5
	4
alt	3

Python dilinde yığın (stack) yapısına yeni bir eleman eklemek istenildiğinde, bu elemanı listenin en sonuna eklemek için `append()`, çıkarmak için ise `pop()` fonksiyonları kullanılır. Yığın (stack) işlem fonksiyonları;



Yığın listesine eleman ekleme	<code>L.append()</code>
Yığından eleman çıkarma	<code>L.pop()</code>
Yığının tepesindeki (yani listenin en son) elemanı öğrenme	<code>L[-1]</code>

Proje 9.8. Yığına Eleman Ekleme ve Çıkarma

Yığına eleman ekleme ve çıkarma işlemlerini liste fonksiyonlarını kullanarak gerçekleştirilelim.

Python dili kodlaması	Ekran çıktısı
<pre> L = [3, 4] #Liste L.append(5) #5 eklendi L.append(6) #6 eklendi #Mevcut yığın yapısı print (L) #Yığının tepesindeki eleman çıkarıldı ust = L.pop() print ("Yığından çıkarılan: ", ust) #Yığının mevcut durumu print (L) #Yığının en üstündeki eleman print ("Şimdi en tepedeki eleman:", L[-1]) </pre>	<pre> [3, 4, 5, 6] Yığından çıkarılan: 6 [3, 4, 5] Şimdi en tepedeki eleman: 5 </pre>

Listelerden Kuyruk (Queue) Oluşturma

Kuyruk (queue), eleman ekleme işlemlerinin listenin sonundan, çıkarma işlemlerinin ise listenin başından yapıldığı özel bir veri yapısıdır. Kuyruk, İlk giren - İlk çıkar (First-In-First-Out (FIFO)) mantığı ile çalışır.

Kuyruk çalışma mantığını anlatan günlük hayatta karşılaştığımız birkaç örnek;

- Bir ATM önünde para çekmek için bekleyen insanlar,
- Bir sinema gişesinden bilet almak için bekleyen insanlar,
- Bir yemekhane kuyruğunda bekleyen öğrenciler.





Başka bir örnek, yazıcı (printer) kuyruk yapısıyla işlemleri gerçekleştirmektedir. Yazdır komutu ile verilen birden fazla belgenin ilki yazdırılmakta, diğerleri ise kuyrukta bekletilmekte, sırası geldiğinde yazdırılmaktadır.

Kuyruk yapılarında yığın yapılarının tersine kuyruğa ilk gelen ilk işlem görür. Kuyruk yapısında da iki temel işlem söz konusudur;

- Kuyruğa eleman ekleme `append()` ve
- Kuyruktan eleman çıkarma `pop(0)` işlemleridir.

Kuyruğa eleman eklendiğinde eleman en sona eklenir. Kuyruktan eleman çıkarıldığında ise en baştaki eleman çıkarılır. Kuyruktan çıkarılan eleman aynı zamanda silinmektedir.

Kuyruk (queue) işlem fonksiyonları;

Kuyruğun sonuna eleman ekleme	<code>L.append()</code>
Kuyruktan (listenin en başından) eleman çıkarma	<code>L.pop(0)</code>
En baştaki elemanı öğrenme	<code>L[0]</code>

Proje 9.9. Kuyruğa Eleman Ekleme ve Çıkarma

Kuyruğa eleman ekleme ve çıkarma işlemlerini liste fonksiyonlarını kullanarak gerçekleştirelim.

Python dili kodlaması	Ekran çıktısı
<pre>L = [3, 4] # Liste L.append(5) # 5 eklendi L.append(6) # 6 eklendi print(L) # Kuyruk listesi bas = L.pop(0) # 3 ayrıldı print("Kuyruktan ayrılan: ", bas) #Kuyruğun mevcut durumu print(L) #Kuyruğun en başındaki eleman print("Şimdi en baştaki eleman:", L[0])</pre>	<pre>[3, 4, 5, 6] Kuyruktan ayrılan: 3 [4, 5, 6] Şimdi en baştaki eleman: 4</pre>

Soru ve Cevaplarla Neler Anladık?

- 1 Dizi (array) ile Liste (list) yapıları arasında bir fark var mıdır?

Diziler aynı tip elemanlardan oluşurken listeler farklı tip elemanlardan oluşabilir. Python dilinde liste, dizi yerine kullanılır ve çok daha yeteneklidir.

2 Listelerin veri tipi nedir?

list

3 Python dilinde listeler nasıl tanımlanır?

[] köşeli parantezler arasına yazılan sıralı elemanlar dizisi, birer listedir.

L = [1,2,3] gibi

4 Aşağıdaki programın ekran çıktısı ne olur?

```
ad = "veli"
ad = list(ad)
ad.sort()
ad.reverse()
print (ad)

['v', 'l', 'i', 'e']
```

5 Listelerin indis numaralandırması nasıl yapılır?

Python dilinde indis numarası verilirken ya solda sağa (pozitif sayılar) ya da sağdan sola (negatif sayılar) doğru numaralandırma yapılabilir. [0]. indis ilk elemanı [-1]. indis ise son elemanı verir.

6 Aşağıdaki programın çalıştırılmasından sonra 'ad' değişkeninin içeriği ne olur?

```
ad = "Ali"
ad = list(ad)
ad.append('m')

['A', 'l', 'i', 'm']
```

7 A = ['1','2','3']
for a in A:
 print(3*a)

Yukarıdaki programın ekran çıktısı nedir?

111
222
333

8 Listedeki bir elemanın indis numarasını öğrenmek için hangi fonksiyonu kullanılır?

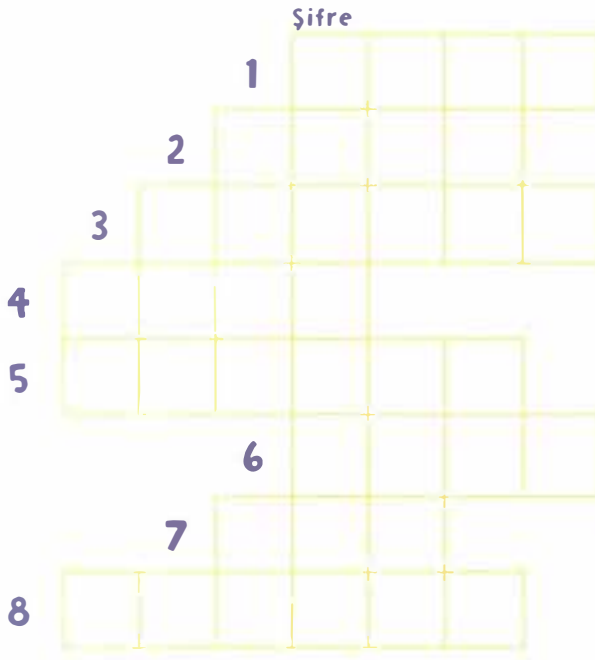
index() fonksiyonu kullanılır. Örnek: ad.index('e')

9 Aşağıdaki programın çalışmasından sonra A listesinin ilk ve son elemanını değeri ne olur?

```
A=[-1,1,-1,1,-1]
for i in range (1,3):
    A.append(i+1)
```

İlk elemanı -1, son elemanı 3 olur.

BULMACA



1. Liste veri tipi.
2. Ardarda sıralanmış aynı türden elemanlardan oluşan bir grup obje.
3. Liste aralarına eleman ekler.
4. Listeyi sıralar.
5. Liste sonuna eleman ekler.
6. Last In First Out.
7. Listenin toplam eleman sayısını verir.
8. FIFO mantığı ile çalışan veri yapısı.

Aşağıya bulduğunuz şifreyi kodlayınız.

--	--	--	--	--	--	--	--	--	--

Bölüm

10

Sözlükler

```
    .---|---|   .-.-   |---|   .-.-  
.-.-|===|Ch|---|_|---.---|$$|---|:::|---.  
|%%|Eng|Oo|===key==|%%|Öö|--|Tr| | | | |
| | |Ii| | | | |İi| |:::|==|  
| |Eng|Uu| | | | |Üü| |Tr| | |  
|~~|===|---|--value-|%%|~~|--|:::|==|  
{dict^---'^--^--:--^--^--^--'^dict}
```

Sözlükler; birbirlerinden virgüllerle ayrılmış {key:value} şeklinde eşleşen veri yapılarıdır.

Giriş

Gerçek hayattaki sözlükleri elektronik ortamda oluşturmak için sözlük yapıları kullanılır. Sözlükler; birbirlerinden virgüllerle ayrılmış {anahtar(key) : değeri(value)} şeklinde eşleşen veri yapılarıdır.

Sözlük yapılarında çiftler süslü parantezler arasına yazılır:

Sözlük Adı	Anahtar	: Değeri
Mevsimler = {	'Kış'	1,
	'İlkBahar'	2,
	'Yaz'	3,
	'SonBahar'	4 }

Sözlük (İngilizcesi: dictionary) veri yapısının tipi "dict"dir. Dolayısıyla dict sarmalına alınan her veri yapısı birer sözlüktür.

Sözlüklerde indis kullanılmaz. Sözlük elemanlarına erişim "anahtar" değeri üzerinden yapılır. Aşağıdaki tabloda sözlük tanımlama şekillerine ve örneklerine yer verilmiştir.

Python dili sözlük (dict) tanımlama şekilleri:

```
b = {'bir': 1, 'iki': 2, 'uc': 3}
d = dict([('iki', 2), ('bir', 1), ('uc', 3)])
s = {} #boş sözlük tanımlaması
Sözlüğe eleman ekleme: S[anahtar]=[değeri]
S[1]= 'one'; S[2]= 'two'; # S={1: 'one', 2: 'two'}
```

Sözlük yapısının özellikleri;

- Sözlük elemanları değiştirilebilir (mutable) özelliktedir.
- Birbirlerinden virgüllerle ayrılmış {‘anahtar: değeri’, ...} şeklinde eşleşen çift yapılardır.
- Listelerde [...] köşeli parantezler, sözlüklerde ise {...} süslü parantezler kullanılır.
- String ve liste elemanlarına erişim için tamsayı indisler kullanılırken, sözlüklerde anahtar kullanılır.
- Her bir anahtar, tekil (benzersiz) olmalıdır.

Sözlük (Dict) Fonksiyonları

Aşağıda verilen sözlük fonksiyonlarını kullanarak sözlük üzerinde farklı işlemler gerçekleştirilebilir. Sık kullanılan bazı sözlük fonksiyonları:

Sözlük fonksiyonları	Açıklaması	Örnek kullanım
<code>S.get(key [,mesaj])</code>	Eğer anahtar (key) değeri sözlükte varsa değerini yoksa mesajı verir.	<pre>A = 1 S = {1:'bir',2:'iki'} print(S.get(A,"yoktur")) # Ekrana 'bir' yazar.</pre>
<code>S.pop(key)</code>	Sözlükten anahtar değeri (key) verilen elemanı siler.	<pre>S = { 1:'bir',2:'iki'} S.pop(1) #1.elemanı sil print (S) #Ekrana {2:'iki'} yazar</pre>
<code>S.keys()</code>	Sözlük çiftlerinin sadece anahtarlarını verir.	<pre>S = { 1:'bir',2:'iki'} print (S.keys()) #Ekrana [1, 2] yazar</pre>
<code>S.values()</code>	Sözlük çiftlerinin değerlerini (anahtar karşılıklarını) verir.	<pre>S = { 1:'bir',2:'iki'} print (S.values()) #Ekrana ['bir','iki'] yazar</pre>
<code>S[anahtar]</code>	Anahtarı belirtilen sözlüğün değer karşılığını verir.	<pre>S = { 1:'bir',2:'iki'} print (S[2]) #Ekrana 'iki' yazar</pre>
<code>S.items()</code>	Sözlük elemanlarını (anahtar, 'değeri') çifti şeklinde verir.	<pre>S = {1:'bir', 2:'iki'} print (list(S.items())) #Çıktı: [(1, 'bir'), (2, 'iki')]</pre>

Proje 10.1. Rakamın Harf Karşılığını Bulalım

Sayıları harf karşılıkları ile eşleştiren bir sözlük yapısı oluşturalım. Sonra da klavyeden girilen rakamın harf karşılığını ekrana yazdıran programı kodlayalım.

Bu sözlük yapısı rakamlar anahtar, harf karşılıkları da değeri şeklinde düzenlenebilir.

Python dili kodlaması

```
rakam=int(input("1-9 arası bir sayı gir.:"))

Sozluk = {1:"Bir", 2:"İki", 3:"Üç", 4:"Dört", 5:"Beş",
          6: "Altı", 7:"Yedi", 8:"Sekiz", 9:"Dokuz"}

#Rakam karşılığı sözlükte yoksa uyarı mesajı verelim
print(Sozluk.get(rakam, "Sözlükte yoktur!"))
```

Ekran çıktısı

```
1-9 arası bir sayı gir.: 4
Dört

1-9 arası bir sayı gir.: 10
Sözlükte yoktur!
```

Proje 10.2. Türkçe-İngilizce Sözlük

Girilen Türkçe günün İngilizce karşılığını veren Türkçe – İngilizce bir sözlük programı kodlayalım.

Bu sözlük yapısı Türkçe gün isimleri anahtar, İngilizce karşılıkları da değeri şeklinde düzenlenebilir.

Python dili kodlaması

```
ara = input("Türkçe gün adı: ")
trEn = {"pazartesi": "monday", "salı": "tuesday",
        "çarşamba": "wednesday", "perşembe": "thursday",
        "cuma": "friday", "cumartesi": "saturday", "pazar": "sunday"}

print("İngilizcesi:", end=" ")
print(trEn.get(ara, "Bu kelime sözlükte yoktur!"))
```

Ekran çıktısı

```
Türkçe gün adı: salı
İngilizcesi: tuesday
```

Sıra Sende Etkinliği-1:

Sözlüğü girilen İngilizce günün Türkçe karşılığını verecek şekilde yeniden kodlayabilir misin?

İpucu: {anahtar : değeri} şeklindeki eşleştirmeyi değiştirmek yeterli olabilir.

Proje 10.3. İller ve Plakaları

İl plakalarını tutan bir sözlük yapısı oluşturarak sözlüğe yeni eleman ekleme, silme ve sorgulama işlemlerinin nasıl yapılacağını gösteren bir program yazalım.

Python dili kodlaması

```
plaka = {} #boş plaka sözlüğü tanımlandı
#plaka sözlüğüne elemanlar ekleniyor
plaka['01'] = 'Adana'
plaka['06'] = 'Ankara'
plaka['16'] = 'Bursa'
plaka['24'] = 'Erzincan'

#plaka sorgulaması yapıldı
print("01 plakalı il.:", plaka.get('01'))
#plaka silindi
plaka.pop('01') #01 plakalı il silindi
#Tekrar plaka sorgulaması yapıldı
print("01 plaka silindi.:", plaka.get('01'))
```

Ekran çıktısı

```
01 plakalı il.: Adana
01 plaka silindi.: None
```

Program açıklaması kod içerisinde yorum '#' satırları ile verilmiştir.

Sıra Sende Etkinliği-2:

Bu programdaki tüm sözlük eşleştirmelerini `plaka['Ankara']='06'` olacak şekilde değiştirebilir misin?

Proje 10.4. Roma Rakamına Dönüştürme

Verilen sayıyı roma (romen) rakamına dönüştüren programı kodlayalım.

Sayı (anahtar)	1000	900	500	400	100	90	50	40	10	9	5	4	1
Roma (değeri)	M	CM	D	CD	C	XC	L	XL	X	IX	V	IV	I

Programda onluk sayı sistemi ve roma rakamı eşdeğerlerinin tutulduğu bu iki ayrı listenin birleşimi bir sözlük yapısı şeklinde ifade edilmiştir.

Python dili kodlaması

```
#sayıyı roma rakamına dönüştüren fonksiyon
def romaCevir(sayi):
    cikti = ""
    #sozluk çiftini anahtar ve değeri şeklinde al
    for anahtar, deger in sozluk.items():
        while sayi >= anahtar:
            #sayi>=anahtar olduğu sürece
            cikti = cikti + deger #değeri çıktıya ekle
            sayi = sayi - anahtar #sayıyı anahtardan çıkar
    return cikti

#Sayıların roma rakam eşdeğerleri:
sozluk = {1000:"M", 900:"CM", 500:"D", 400:"CD", 100:"C", 90:"XC",
          50:"L", 40:"XL", 10:"X", 9:"IX", 5:"V", 4:"IV", 1:"I" }

sayi = int (input("Sayı:..")) #sayı gir
print (romaCevir(sayi))      #sayıyı çevir ve ekrana yazdır.
```

Ekran çıktısı

```
Sayı:..124
CXXIV
```

Proje 10.5. Tic-Tac-Toe (XOX) Oyunu

Tic-Tac-Toe (XOX) Oyununu sözlük yapısını kullanarak kodlayalım.

Tic-Tac-Toe oyununun algoritması;

1. Tic-tac-toe tahta yerleşim şeklini ayarla:

- Paralel iki dikey çizgi ve bunları kesen iki yatay çizgi çizilir.
- Çizgileri çektiğinizde oyun tahtanız # yorum simgesi şeklinde gözükür.

2. Simgeleri yerleştir:

- Sırası ile X veya O simgelerinin tahtada nereye konumlanacağını gir.

3. Yerleştirilenleri kontrol et:

- Oyunun amacı kendi simgelerini (alt alta veya yan yana veya çapraz) yerleştirerek oyunu kazanmaktır. Tüm hücreler dolduğunda simgeler arka arkaya sıralı değilse oyun berabere biter.

Python dili kodlaması

```
#1.Tic-tac-toe tahta yerleşim şeklini çizdirelim:
print ('1|2|3\n-+-\n4|5|6\n-+-\n7|8|9')

#Tahta sözlük yapısının anahtar ve değerlerine atama yapılıyor
tahta = {'1': ' ', '2': ' ', '3': ' ',
         '4': ' ', '5': ' ', '6': ' ',
         '7': ' ', '8': ' ', '9': ' '}

def yazTahta(tahta):
    print(tahta['1'] + '|' + tahta['2'] + '|' + tahta['3'])
    print('-+-')
    print(tahta['4'] + '|' + tahta['5'] + '|' + tahta['6'])
    print('-+-')
    print(tahta['7'] + '|' + tahta['8'] + '|' + tahta['9'])

#3. Oyunu kim kazandı? Olasılıkları yazalım
def kimKazandi(tahta, simge):
    #yan yana kazanma olasılıkları
    if tahta['1'] == tahta['2'] == tahta['3'] == simge:
        return True
    elif tahta['4'] == tahta['5'] == tahta['6'] == simge:
        return True
    elif tahta['7'] == tahta['8'] == tahta['9'] == simge:
        return True
    #alt alta kazanma olasılıkları
    elif tahta['1'] == tahta['4'] == tahta['7'] == simge:
        return True
    elif tahta['2'] == tahta['5'] == tahta['8'] == simge:
        return True
    elif tahta['3'] == tahta['6'] == tahta['9'] == simge:
        return True
    #çapraz kazanma olasılıkları
    elif tahta['1'] == tahta['5'] == tahta['9'] == simge:
        return True
    elif tahta['3'] == tahta['5'] == tahta['7'] == simge:
        return True
    else:
        return False #beraberlik durumu
```

Ekran çıktısı

```
1|2|3
-+-+
4|5|6
-+-+
7|8|9

X, nereye
yerleştirilsin?
5
| |
-+-+
|X|
-+-+
|X|

O, nereye
yerleştirilsin?
6
| |
-+-+
|X|O
-+-+
|X|

X, nereye
yerleştirilsin?
9
| |
-+-+
|X|O
-+-+
| |X
```

```
#2.Simgeleri yerleştir: X ve O ları yerleştir
simge = 'X' #ilk olarak oyuna X başlasın
for i in range(9):
    print(simge + ', nereye yerleştirilsin?')
    gir = input() #simge gir
    tahta [gir] = simge
    if simge == 'X': #simgeyi değiştir
        simge = 'O'
    else:
        simge = 'X'
    yazTahta(tahta) #simgeyi tahtaya yerleştir
    #kimKazandi() fonksiyonundan gelen sonuca göre ekrana
    mesaj yaz
    if kimKazandi(tahta, 'X'):
        print("Oyunu X Kazandı")
        break
    if kimKazandi(tahta, 'O'):
        print("Oyunu O Kazandı")
        break
#beraberlik durumunda ki mesajı yazalım
if kimKazandi(tahta, 'O') == False and kimKazandi(tahta, 'X')
==False:
    print("Berabere..")
```

```
O, nereye
yerleştirilsin?
4
| |
-+-+
O|X|O
-+-+
| |X

X, nereye
yerleştirilsin?
1
X| |
-+-+
O|X|O
-+-+
| |X

Oyunu X
Kazandı
```

Küme (set) ve Demet (tuple) Veri Yapıları

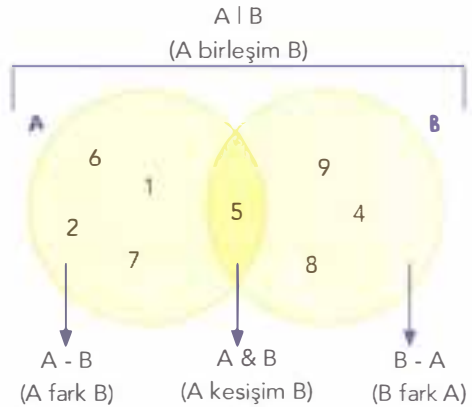
Python dilinde liste ve sözlük yapıları kadar sık kullanılsa da iki farklı veri yapısını da burada kısaca açıklamakta fayda var. Bunlar;

- **Küme (set)**; sırasız, tekil elemanlar topluluğudur. Küme elemanları süslü parantezler {...} arasına yazılır. Veri tipi ise <set>'dir. Örnek kullanım;

```
k = set('abracadabra')
#Çıktısı: k = {a,b,d,c,r} kümesi olur
```

Python küme yapısı matematiksel küme işlemlerini {!:birleşim, -:fark, &: kesişim gibi} destekler.

Yanda Venn şeması verilen A ve B kümeleri üzerinde;
birleşim işlemi $A \cup B$,
kesişim işlemi $A \cap B$,
A fark B işlemi $A - B$ şeklinde ifade edilir.



$A = \{1, 2, 5, 6, 7\}$ #A kümesi
 $B = \{4, 5, 8, 9\}$ #B kümesi

- **Demet (tuple):** sıralı, değişmez nesneler dizisidir. Demet elemanları aç-ka-pa parantezleri (...) arasına yazılır. Veri tipi ise <tuple>'dır. Değişmez yapısı nedeni ile tuple elemanları dict (sözlük) veri yapısında anahtar (key) olarak kullanılabilir iken listeler kullanılamaz. Listelere (list) göre daha hızlı çalışırlar. Örnek kullanım şekilleri:

<code>D = ('P', 'Y', 'T', 'H', 'O', 'N')</code>	Karakter demeti
<code>S = "hanzar"; D = tuple(S)</code>	String demet yapısına dönüştü. Çıktısı: <code>#('h', 'a', 'n', 'z', 'a', 'r')</code>

Soru ve Cevaplarla Neler Anladık?

- 1 Listelerde [...] köşeli parantezler kullanılırken sözlüklerde hangi parantezler kullanılır?
{...} süslü parantezler kullanılır.
- 2 Sözlük yapısının veri tipi nedir?
dict
- 3 Python dilinde sözlükler nasıl tanımlanır?
Sözlükler; birbirlerinden virgüllerle ayrılmış {anahtar : değeri} şeklinde eşleşen çiftlerdir. Örnek; S = {1:'bir',2:'iki'} gibi
- 4 Boş bir sözlük tanımlaması nasıl yapılır?
`S = { }`
- 5 Aşağıdaki programın ekran çıktısı ne olur?

```
S = { 'bir':1, 'iki':2}
for i in S.values():
    print(i)
```


1
2
- 6 Aşağıdaki programın ekran çıktısı ne olur?

```
S = { 'bir':1, 'iki':2}
for i in S.keys():
    print(i)
```


bir
iki

7 Aşağıdaki programın ekran çıktısı ne olur?

```
il = {'Adana':7, 'Ankara': 6, 'Tokat':60}  
print(il.get('Ankara'))  
  
6
```

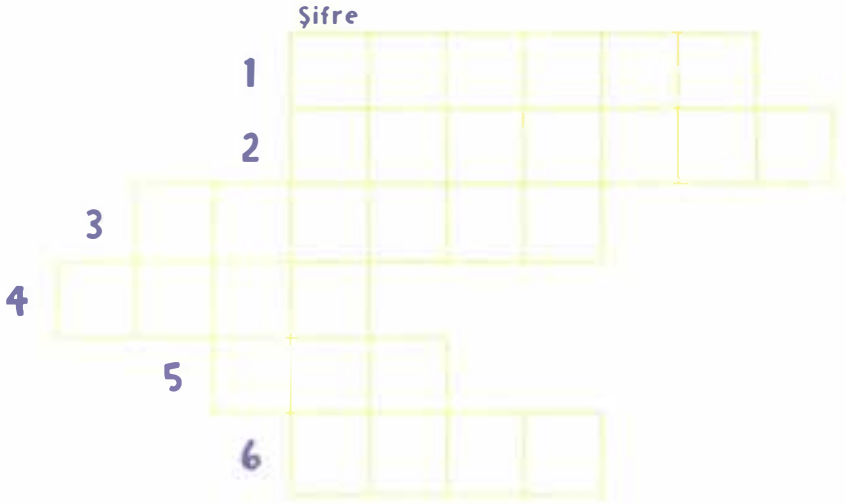
8 Aşağıdaki programın ekran çıktısı ne olur?

```
il = {'Adana':7, 'Ankara': 6, 'Tokat':60}  
print(il.get(6))  
  
None
```

9 Aşağıdaki programın ekran çıktısı ne olur?

```
il = {'Adana':7, 'Ankara': 6, 'Tokat':60}  
il.pop('Ankara')  
print (il)  
  
{ 'Adana': 7, 'Tokat': 60}
```

BULMACA



1. Sözlük çiftlerinin değerlerini (anahtar karşılıklarını) verir.
2. Sözlük elemanlarına erişim onun üzerinden yapılır.
3. (Tersi) Birbirlerinden virgüllerle ayrılmış {anahtar(key): değeri(value)} şeklinde eşleşen veri yapısı.
4. Aşağıdaki programın çıktısı;

```
zit = {"Evet": "Evet", "Hayır": "Hayır", "Evet": "Hayır", "Hayır": "Evet"}  
print (zit["Hayır"])
```
5. Eğer anahtar (key), sözlükte varsa değerini yoksa mesajı verir.
6. (Tersi) Sözlük çiftlerinin sadece anahtarlarını veren metot.

Aşağıya bulduğunuz şifreyi kodlayınız.

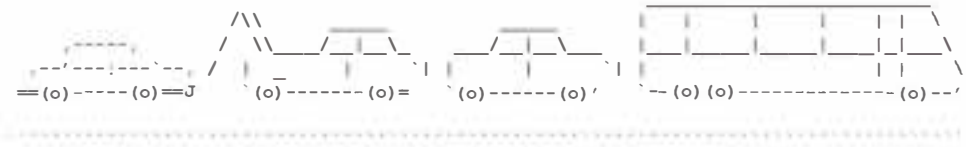
--	--	--	--	--	--	--

Bölüm

11

Nesne ve Sınıf Kavramları

Araba sınıfına ait nesneler;



Bütün nesneler, bir sınıf temel alınarak oluşturulur,
yani türetilir.

Giriş

Python dilinde her şey (değişken, liste, fonksiyon, sınıf vb.) bir nesne (İngilizcesi: object) olarak algılanır. Ve bütün nesneler (tamsayılar, karakter dizileri, listeler, fonksiyonlar, sınıflar, modüller vb.) eşit statüye sahiptir. Dolayısıyla bütün nesneler bir değişkene atanabilir.

Sınıf (İngilizcesi: class), ortak özellikleri ve davranışları olan nesnelerin, bu ortak özellik ve davranışlarını tanımlayan soyut bir kavramdır.

Sınıf, kendisinden nesneler türetilen bir yapıdır. Aşağıda bazı sınıf yapıları ve bu sınıf yapılarından türetililecek nesne örneklerine yer verilmiştir;

Sınıf (class)	Sınıftan türetililebilen nesneler
Çokgen ()	Üçgen Dörtgen Beşgen
Araba ()	Kamyon Taksi Otobüs
Hayvan ()	Kedi Köpek Tavşan
Kurabiye_kalıbı () 	Kurabiyeler 

Not. Örneğin Kurabiye_kalıbı() sınıfı bir kalıptır (şablondur). Bu kalıp kullanılarak onlarca kurabiye nesnesi türetililebilir. Dolayısıyla sınıflar kendilerinden nesneler türetilen yapılardır.

Sınıf (Class) Tanımlama

Python dilinde sınıf tanımlama işlemi fonksiyon tanımlamaya benzer. Fonksiyon tanımlarken def, sınıf tanımlarken ise class deyimleri kullanılır. Aynı fonksiyonlarda olduğu gibi sınıf tanımının başlangıç ve bitişini göstermek için girintiler kullanılır.

Bir yazım kuralı olarak da class sözcüğünün tamamı küçük harflerden oluşmalıdır.

Sınıf yapısı	Örnek kullanım
<pre>class SınıfAdı(): <işlemler-1> <işlemler-N></pre>	<pre>class Araba(marka): #Araba sınıfı model = marka #özellikleri def metot(self): #metotları self.marka= 'BMW'</pre>
Not 1: class tanımlarken SınıfAdı yanındaki () parantezlerinin kullanımı zorunlu değildir. Not 2: Sınıf içerisinde tanımlanan tüm metotlar (fonksiyonlar) ilk parametre olarak nesnenin kendisini işaret eden self işaretçisini (parametresini) alırlar.	

self ve pass Deyimleri

pass deyimi; gövdesiz içi boş fonksiyon veya sınıf tanımlamak için kullanılır.

self deyimi; sınıf içerisinde erişilecek elemanlar için kullanılır. Nesnenin kendisini referans etmesini sağlar. Başka bir ifade ile sözü edilen nesneyi doğrudan göstermek için kullanılır.

Nesne (Object) Tanımlama

Bütün nesneler bir sınıfı temel alarak oluşturulurlar, yani türetilirler. Bir sınıftan birden fazla nesne türetilir.

Sınıftan nesne üretme	Örnek sınıf ve nesne tanımlamaları
<pre>Nesne_adi = SınıfAdı (parametre) #parametre girişi seçimlidir.</pre>	<pre>class Araba(): pass Kamyon = Araba() #Kamyon nesnesi Taksi = Araba() #Araba nesnesi #Araba sınıfından Kamyon ve Taksi nesneleri oluşturuldu</pre>

Her nesne, bir takım özellik ve metotlara sahiptir.

Örneğin; Bir "Araba" sınıfından "Kamyon", "Taksi" nesneleri türetilir. Bir sınıftan türetilen nesneler o sınıfa ait özelliklere sahip olur. Her iki nesne de "Araba" sınıfında yer alan özellik (renk, model, vs.) ve varsa metotlara (git(), dur() gibi yeteneklere) sahip olacaktır.

Nesne Özellikleri

Bir nesnenin özellikleri sınıfa ait değişkenlerdir. Bir nesnenin özellik veya metotlarına erişim için nokta "." simgesi kullanılır.

Araba sınıfının marka, model gibi değişkenleri, Araba sınıfına ait nesnelerin birer özelliğidir:

```
marka = "Honda Jazz"  
model = 2020 gibi
```

Bir nesne, ait olduğu sınıfa ait özellikleri;

```
nesne.özellik= "yeni değeri" şeklinde değiştirebilir.
```

Örneğin Taksi nesnesinin modeli yükseltmek istendiğinde aşağıdaki gibi değiştirilebilir;

```
Taksi.model = 2021
```

Nesne Metotları

Nesnenin metotları sınıf içerisinde tanımlanan fonksiyonlardır. Yani Python dilinde bir fonksiyon, bir sınıf (class) içerisinde tanımlandı ise "metot" ismini alır.

Nesnelerin metotları aşağıdaki gibi çağrılır:

```
nesneAdı.metotAdı()  
#Taksi.aracBilgisi() gibi
```

Not. Bir nesnenin özellik veya metotlarına erişim için nokta "." simgesi kullanılır.

Sınıf Metotlarının Normal Fonksiyonlardan Bir Farkı Var Mıdır?

Hayır yoktur. Sadece sunum açısından şu farklar vardır:

1. Normal fonksiyon doğrudan çağrılırken, metotlar nesne.metot() şeklinde çağrılır.
2. Sınıf içerisindeki metot tanımlamalarında self parametresi kullanılır. Fakat metot çağrılırken self parametresine değer atamasına gerek yoktur. Çünkü self deyimi zaten nesnenin kendini işaret etmektedir.

Şimdi bir Araba sınıfından Kamyon, Taksi gibi nesneler üretip, özellik ve metot tanımlamalarını nasıl yapabileceğimizi görelim.

Proje 11.1. Araba Sınıfından Yeni Nesneler Üretme

Araba sınıfından “Kamyon, Taksi” gibi nesneler oluşturup, bu nesnelerin özelliklerini ekrana yazdıran uygulamayı kodlayalım.

Araba sınıfından aynı anda çok sayıda (kamyon, taksi, ...) nesne türetilabilir.

Python dili kodlaması

```
class Araba():
    # Sınıf özelliklerine ilk değerleri atandı
    def __init__(self, marka, model, renk):
        self.marka = marka
        self.model = model
        self.renk = renk
    # Sınıf fonksiyonu
    def aracBilgisi(self):
        print("markası: ", self.marka)
        print("model yılı:", self.model)
        print("rengi:", self.renk)

#Ana program
#Taksi nesnesi oluşturuldu
Taksi = Araba("Honda", 2020, "Gri")
#Kamyonet nesnesi
Kamyonet = Araba("Ford", 2010, "Beyaz")
#Bu iki nesnenin araç bilgilerini alalım
print ("Taksinın özellikleri..")
Taksi.aracBilgisi()
print ("Kamyonetin özellikleri..")
Kamyonet.aracBilgisi()
```

Ekran çıktısı

```
Taksinın özellikleri.:
markası: Honda
model yılı : 2020
rengi: Gri

Kamyonetin özellikleri.:
markası: Ford
model yılı: 2010
rengi: Beyaz
```

__init__() Metodu

Python dilinde sınıf tanımlarken sınıf değişkenlerine (özelliklerine) başlangıç değerleri __init__() isimli metot (çift alt tire) ile atanabilir.

__init__() metodu, bir sınıftan bir nesne türetildiğinde otomatik olarak ilk çağrılan ve çalıştırılan metottur.

__init__() metot işlevi

Sınıftan bir nesne oluşturulduğunda otomatik çağrılan bir metottur. Nesnenin başlangıç değerlerinin atandığı metottur. Bu metot return fonksiyonu almaz.

Örnek kullanım

```
class Araba:
    def __init__(self, ma, mo):
        self.marka = ma
        self.model = mo
x=Araba("Ford", 2020)
print (x.marka, x.model)
```

Not. Java ve Python gibi diller, çöp toplayıcı mekanizmasına sahiptir. Yani görevi sona eren nesneler, otomatik olarak bellekten silinir.

Bir Nesnenin Metot ve Özellikleri Nasıl Öğrenilebilir?

Bir nesnenin tüm metot ve özellikleri `dir()` fonksiyonu ile öğrenilebilir.

Örnek kullanım

```
class Araba:
    def __init__(self, ma, mo):
        self.marka = ma
        self.model = mo
kamyon = Araba("Ford", 2020)

print (dir (kamyon))
```

Ekran çıktısı

```
['__class__', '__delattr__', '__dict__', '__dir__', '__doc__', '__eq__', '__format__', '__ge__', '__gt__', '__hash__', '__init__', '__le__', '__lt__', '__module__', '__ne__', '__new__', '__reduce__', '__reduce_ex__', '__repr__', '__setattr__', '__sizeof__', '__str__', '__subclasshook__', 'marka', 'model']
```

Yukarıdaki programda, kamyon nesnesinin metot ve özellikleri `dir (kamyon)` komutu ile öğrenildi. Dikkat edilirse ekran çıktısında bazı metotların başında ve sonunda `"_"` alt tireler varken, bazılarında yoktur.

Python dili, kendi standart / yerleşik metotlarını `"_"` alt tireler ile ayırır. Doğal olarak bizim tanımladığımız metotlar ve özellikler (marka, model) çıktıda alt tiresiz gösterilenlerdir.

Sınıf Yapıları ile Yeni Veri Tipleri Tanımlama

Python dilinde tüm veri tipleri bir sınıf (class) içerisinde tanımlanmıştır. Bu nedenle bir değişkenin veya nesnenin tipi `type` deyimi ile öğrenilmek istenildiğinde o verinin tipi, `<class '...'>` anahtar kelimesi ile birlikte gösterilir. Örneğin;

```
>>> a = 5 ; b = []
>>> print (type(a))
#Çıktısı:<class 'int'>
>>> print (type(b))
#Çıktısı: <class 'list'>
```

Proje 11.1'deki "Araba" sınıfında olduğu gibi her sınıf (class) tanımlaması aynı zamanda yeni bir tip tanımlamasıdır. Örneğin Araba sınıfından bir taksi nesnesi türetildiğinde taksinin veri tipi, türetildiği Araba sınıfı olacaktır. Örnekteki Taksi nesnesinin veri tipi `type` komutu ile aşağıdaki gibi görülebilir:

```
>>> print (type(Taksi))
#Taksi nesnesinin tipi: Araba sınıfıdır: <class '__main__.Araba'>
```

Kalıtım

Kalıtım (İngilizcesi: inheritance), üst (türetilen) sınıfa ait özelliklerin, alt (türeyen) sınıflara miras olarak aktarılma işlemidir. Böylece kalıtım alt sınıf, ait olduğu üst sınıfın özelliklerini taşır.

Kalıtım ile bir sınıfta tanımlanmış özellik ve metotlar, yeniden tanımlanmaya ihtiyaç duyulmadan alttaki sınıflara aktarılır.

Bir çocuğun genetik olarak anne babasından aldığı özelliklere ek olarak farklı özelliklere sahip olması gibi kalıtımla türeyen (alt) sınıf, türediği (üst) sınıfa ait yeteneklere (özellik ve davranışlara) doğal olarak sahip olurken bazı yetenekleri de kendine göre özelleştirebilir.

Bir üst sınıftan alt sınıflar türetme işlemi Python dilinde aşağıdaki gibi tanımlanabilir:

Python dili kalıtım tanımlaması	Örnek kullanım
<pre>class ustSınıf:</pre>	<pre>#Araba bir (türetilen) üst sınıftır class anneBaba(): pass</pre>
<pre>class altSınıf(ustSınıf):</pre>	<pre>#kizCocuk ve erkekCocuk (türeyen) alt sınıflardır class kızCocuk(anneBaba): def __init__(self, soyad): self.soyad = soyad # class erkekCocuk(anneBaba): def __init__(self, soyad): self.soyad = soyad #</pre>

Örneğimizde; anneBaba üst sınıfının kalıtım ile oluşmuş alt sınıfları; erkekCocuk ve kızCocuk'dur. Bu alt sınıflara, anneBaba sınıfının üyeleri kalıtım ilkesine göre miras kalır. Böylece, tekrar yazmaya gerek kalmadan her iki alt sınıf da anneBaba sınıfında yer alan özelliklere (göz rengi, soyadı gibi) ve metotlara sahip olacaktır.

Kalıtımın bu özelliği sayesinde yazılım geliştirme aşamasındaki olası kod tekrarı azaltılmış olur.

Proje 11.2. Kalıtım

Kalıtım konusunu daha iyi anlamak için örneğimizi kodlayalım.

Python dili kodlaması

```
#üst sınıf
class anneBaba():
    #başlatıcı metot ve parametreleri
    def __init__(self, renk):
        self.renk = renk
        print ("Göz rengim:" + self.renk)

#anneBaba sınıfından kalıtımla Evlat alt sınıfı
türetildi
class Evlat(anneBaba):
    #başlatıcı metot ve parametreleri
    def __init__(self, renk, alinTerim):
        #anneBaba sınıfından göz rengimi aldım
        anneBaba.__init__(self, renk)
        #kendi alın terim ise diplomam
        self.alinTerim = alinTerim
        print ("Alın terim:" + self.alinTerim)

# Ana program
# Evlat alt sınıfından 'ali' nesnesi türetildi
print ("---Ben Ali---")
Ali = Evlat("Ela", "Diploma")
```

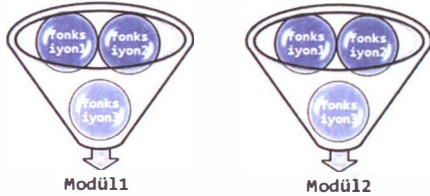
Ekran çıktısı

```
---Ben Ali---
Göz rengim.:Ela
Alın terim:Diploma
```

Kalıtım; bir veya daha fazla üst sınıfa ait özelliklerin miras alan alt sınıfa kalıtsal olarak aktarılması işlemidir. Örneğin bir 'Evlat' alt sınıfı; kendi alın terine (diploma) ek olarak 'anneBaba' üst sınıfından göz rengini (renk) miras alabilir.

Modül Kavramı

Bir Python programı fonksiyonlardan oluşur. Ve her Python programı (.py uzantılı dosya) da bir modüldür.



Farklı import Şekilleri

Bir modül dosyasını programa eklemek için import deyimi kullanılır. import; Türkçe "içeri aktarmak" anlamına gelen bir Python komutudur. Python standart paketinde

olmayan bir fonksiyonu program içerisine almak (eklemek) için import kullanılır. import komutunun farklı kullanım şekilleri aşağıdaki tabloda gösterilmiştir.

Kullanım şekli	Açıklama
<code>import modülAdı</code>	Klasik kullanım şeklidir. Modül içerisindeki komutlara modül ismi ile erişilir (math.pi gibi). Örnek; <pre>import math math.sqrt(9)</pre>
<code>import modul1, modul2, ...</code>	Tek bir komut satırında birden fazla modül "," virgüller ile belirtilerek tanımlanabilir. Örnek; <pre>import math, random</pre>
<code>from modül import fonksiyon</code>	Bir modül içerisindeki sadece belli bir fonksiyonu programa ekler ve fonksiyona modül ismi belirtmeden doğrudan erişim sağlar. Örnek; <pre>from math import sqrt sqrt(9) #math yazılmasına gerek yoktur.</pre>
<code>from modül import *</code>	Bir modül içerisindeki tüm fonksiyonları programa ekler ve fonksiyonlara modül ismi belirtmeden doğrudan erişim sağlar. Örnek; <pre>from math import * print (sqrt(9)) #math yazılmasına gerek yoktur</pre>
<code>import modülAdı as yeniAdı</code>	Program içerisinde uzun modül adlarını daha kısa bir isim ile değiştirir. Örnek; <pre>import math as m m.sqrt(9) #math yerine m yazılır</pre>

__name__ Özelliği

Python dilinde her modül `__name__` isimli bir özelliğe sahiptir. Bir modül, ana program olarak (doğrudan) çalıştırıldığında `__name__` özelliğinin değeri `__main__` olarak ayarlanır.

Bazen programların sonunda şöyle bir if sorgusu görülebilirsiniz:

```
if __name__ == '__main__':
    #fonksiyonAdı()
```

Bu sorgunun iki anlamı vardır:

1. Eğer bu modül ya da program, ana program ise altta yazdığım kod satırlarını yani `fonksiyonAdı` çalıştır değilse çalıştırma.

2. Eğer bu program başka bir program tarafından **import** edilirse bu sorgu altında belirtilen fonksiyonu çalıştırma (kullanma).

PIP ile Yeni Modüller Ekleme

Python programını bilgisayarınıza kurduğunuzda standart modülleri (random, math, sys, os, gibi) ve bu modüllere ait olan fonksiyonları programımızda **import** ederek kullanabiliyorduk.

Fakat Python kurulumu ile birlikte gelmeyen 3. şahıslarca hazırlanan Python programlarını (NumPy, Playsound, Pandas, Pillow gibi) programımıza **import** ettiğimizde bu modülün kurulu olmadığını belirten bir hata mesajı alırız.

Kurulum ile birlikte gelmeyen Python modüllerini bilgisayara kurmanın en kolay yolu **pip** (Python Paket Kurucusu) programını kullanmaktır. Bunun için şu adımlar izlenir;

1. Bilgisayarın komut istemine (terminal ekranına) geçilir. (Windows'ta cmd yazılarak, Linux'ta ise Ctrl + Alt + T tuşları ile geçilebilir).
2. Örnekteki gibi "pip install modül_ismi" komut satırı ile modül kurulur.

Seç Komut İstemi

```
Microsoft Windows [Version 10.0.18363.1256]
(c) 2019 Microsoft Corporation. Tüm hakları saklıdır.

C:\Users\bc>pip install playsound
Collecting playsound
  Using cached playsound-1.2.2-py2.py3-none-any.whl (6.0 kB)
Installing collected packages: playsound
Successfully installed playsound-1.2.2
```

Not. Python kurulumu ile gelmeyen farklı kurum veya şahıslar tarafından geliştirilen (pip komutu ile kurulabilecek) projeler, <https://pypi.org/> adresinde yer almaktadır. Herhangi bir konuda programınıza **import** edebileceğiniz bir projeyi aramak için öncelikle [www.pypi.org](https://pypi.org/) adresine bakabilirsiniz.

Soru ve Cevaplarla Neler Anladık?

1 Her Python programı (.py uzantılı kaynak dosya) aynı zamanda nedir?
Bir modüldür.

2 Bir modül dosyasını programınıza eklemek için hangi deyim kullanılır?
`import`

3 Bir modül içerisindeki tüm fonksiyonları programa eklemek ve fonksiyonlara modül ismi belirtmeden doğrudan erişim sağlamak için import komutu nasıl tanımlanmalıdır?
`from modulAdı import * (from random import * gibi)`

4 Sınıf ile nesne arasındaki fark nedir?
"Araba" ile kamyon veya taksi arasındaki fark ne ise sınıf ile nesne arasındaki fark da odur.

5 Python'da, bir sınıftan bir örnek (nesne) oluşturulduğunda ilk olarak otomatik yapılandırıcı metod çağrılır. Bu metodun ismi nedir?
`__init__`

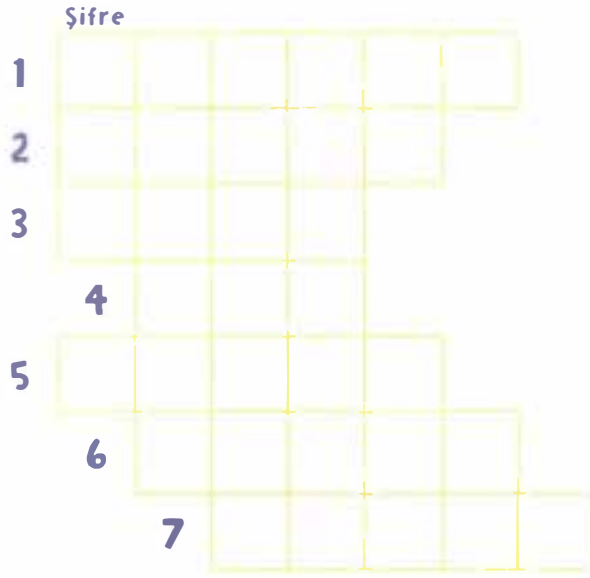
6 `self` deyiminin işlevini açıklayınız?
Sınıf içerisinde erişilecek üyeler için kullanılır. Nesnenin kendisini referans etmesini sağlar. Başka bir ifade ile sözü edilen nesneyi doğrudan göstermek için kullanılır.

7 Bir sınıf veya nesnenin özelliklerine erişim için hangi simge(işaret) kullanılır?
Nokta "." simgesi kullanılır; `k.yas` gibi.

8 Python dilinde bir sınıf nasıl tanımlanır?
Python dilinde sınıf sınıf tanımlarken ise 'class' deyimini kullanılır. Örnek kullanım;
`class Hayvan:
 pass`

9 Python dilinde bir nesne nasıl tanımlanır?
Bütün nesneler bir sınıf, temel alınarak oluşturulur, yani türetilir. Örnek kullanım;
`class Hayvan:
 pass
kedi = Hayvan() #kedi bir nesnedir.`

BULMACA



1. Üçgen, dörtgen, beşgen gibi nesnelerin türetildiği üst sınıf.
2. Sınıf tanımlama deyimi.
3. Sınıf içerisinde erişilecek elemanlar için kullanılır. Nesnenin kendisini referans etmesini sağlar.
4. Bir nesnenin tüm metot ve özellikleri listeler.
5. Sınıf içerisinde tanımlanan fonksiyon.
6. Kendisinden nesneler türetilen soyut yapı.
7. import deyimi ile bir programa eklenen.

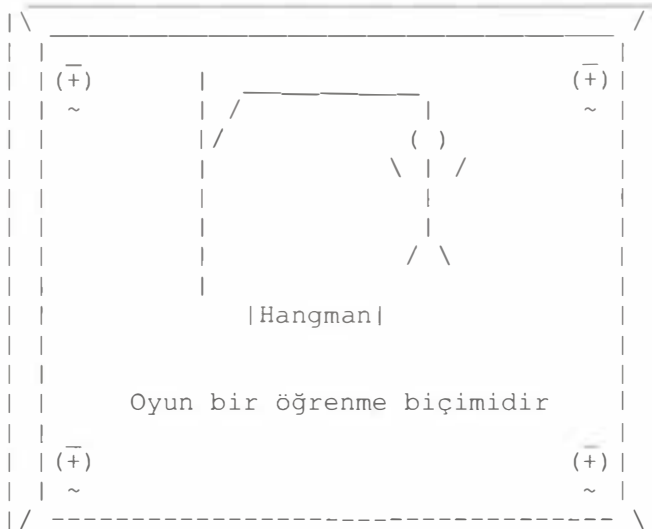
Aşağıya bulduğunuz şifreyi kodlayınız.

--	--	--	--	--	--	--	--

Bölüm

12

Rastgele Sayı Üretimi ve Matematiksel Oyunlar



Matematiksel İşlem Fonksiyonları

Python dilinde sayısal işlemleri kolaylaştırmak için birçok yerleşik (built in) fonksiyon tanımlanmıştır. Bu fonksiyonlar (print()) gibi doğrudan ismi ile çağrılabilir. Fakat bazı matematiksel fonksiyonlar ise `math` modülü içerisinde yer alır ve bu fonksiyonları kullanabilmek için `math` kütüphanesini program başında "import math" şeklinde eklemek gerekir.

Matematiksel işlem fonksiyonları:

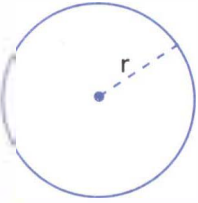
Fonksiyon	Açıklama	Örnek kullanım	Çıktı
Standart matematiksel fonksiyonlar (built-in functions)			
<code>abs(x)</code>	x sayısının mutlak değerini verir.	<code>abs(-23)</code>	23
<code>round(x)</code>	x sayısını en yakın tam sayıya tamamlar.	<code>round(2.5)</code>	2
<code>max(x, y)</code>	x ve y'den büyük olanını verir.	<code>max(2, 4)</code>	4
<code>min(x, y)</code>	x ve y'den küçük olanını verir.	<code>min(2, 4)</code>	2
<code>pow(x, y)</code>	x in y'inci kuvvetini verir.	<code>pow(2, 3)</code>	8
<code>bin(x)</code>	x'i ikili (binary) sayı sistemine çevirir.	<code>bin(25)</code>	11001
<code>hex(x)</code>	x'i onaltılı (hexadecimal) sayı sistemine çevirir.	<code>hex(25)</code>	19
<code>oct(x)</code>	x'i sekizli (octal) sayı sistemine çevirir.	<code>oct(25)</code>	31
<code>chr(x)</code>	x sayısının ASCII kod karakter karşılığı verir.	<code>chr(97)</code>	a
<code>ord(x)</code>	chr(x)'in tersi işleve sahiptir.	<code>ord('a')</code>	97
<code>sum([x,y,z])</code>	Bir listedeki elemanların toplamını verir.	<code>sum([1,2,3,4,5])</code>	15
math kütüphanesinde yer alan bazı matematiksel fonksiyonlar			
<code>sqrt(a)</code>	a sayısının kara kökünü verir.	<code>math.sqrt(9)</code>	3.0
<code>pi</code>	Pi (3.14) sayısını verir.	<code>math.pi</code>	3.14159
<code>gcd(a, b)</code>	a ve b'nin ortak bölenlerinin en büyüğü (obeb'ini) alır.	<code>math.gcd(12, 8)</code>	4
<code>ceil(x)</code>	x sayısını bir üst tam sayıya dönüştürür.	<code>math.ceil(2.5)</code>	3
<code>floor(x)</code>	x sayısını bir alt tam sayıya dönüştürür.	<code>math.floor(-2.5)</code>	-3
<code>cos(x)</code>	x in radyan cinsinden trigonometrik kosinüsünü hesaplar.	<code>math.cos(0.0)</code>	1.0
<code>sin(x)</code>	x in radyan cinsinden trigonometrik sinüsünü hesaplar.	<code>math.sin(0.0)</code>	0.0

Not. `math` kütüphanesinde yer alan bu komutları kullanabilmek için program başında `"import math"` komut satırını eklemek gerekir.

Python dili `math` modülü içerisinde yer alan fonksiyonların güncel ve tam bir listesine <https://docs.python.org/3/library/math.html> adresinden erişebilirsiniz.

Proje 12.1. Dairenin Alanını ve Çevresini Hesaplama

Yarıçap (r) uzunluğu klavyeden girilen bir dairenin alanını ve çevresini hesaplayan programı kodlayalım.

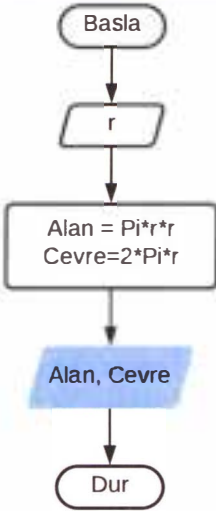


Yarıçapı r olan bir dairenin alanı πr^2 , çevresi ise $2\pi r$ formülleri ile hesaplanabilir.

$$\text{Alan} = \pi r^2$$

$$\text{Çevre} = 2\pi r$$

Akış Şeması



Python dili kodlaması

```
import math #pi sayısı için gerekli  
r = int(input("Yarıçapı gir.:"))  
Alan = math.pi * pow(r,2)  
Cevre = 2 * math.pi * r  
print ("Dairenin Alanı=", Alan)  
print ("Dairenin Çevresi=", Cevre)
```

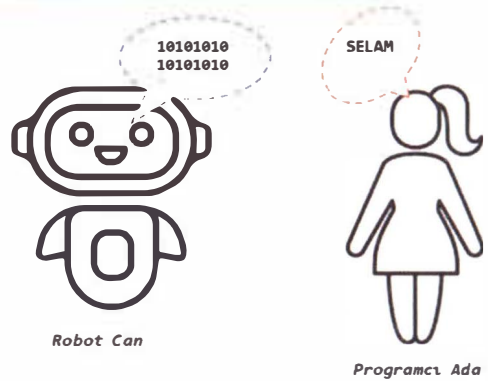
Not. Python dilinde π sayısı; `math.pi` komutu ile elde edilir.

Not. Python dilinde `r*r` işlemi ile `pow(r,2)` işlemi aynıdır/eşdeğerdir.

Proje 12.2. Çevirmen: Konuşma Dilini Makine Diline Çevirme

Haydi bir çevirici yapalım!

Robot Can ile programcı Ada nasıl anlaşacak? Bir çevirmen programı kodlayalım.



Projemizin amacı;

- ikili (binary) sayı sistemi 101010 şeklinde makine dilinde konuşan Robot Can'ın konuşmasını Türkçeye,
- Türkçe dilinde konuşan Ada'nın konuşmalarını Robot Can'ın anladığı makine diline (ikili sayı sistemine) çevirmek.

Böylelikle Ada ile Robot Can birbirlerinin konuşmalarını anlayabilecektir. Bu amaçla iki adet program yazmamız gerektiği açıktır. Robot Can'ın makine dilini Türkçeye, Ada'nın konuşmasını makine diline dönüştürecek ayrı ayrı iki programı aşağıdaki gibi yazabiliriz.

Ada'ya yüklenecek program kodu: Cevirmen1 programı	Robot Can'a yüklenecek program kodu: Cevirmen2 programı
<pre>print ("Ada Konuştu.:") S1 = bin (ord("S")) [2:] E1 = bin (ord("E")) [2:] L1 = bin (ord("L")) [2:] A1 = bin (ord("A")) [2:] M1 = bin (ord("M")) [2:] print (S1,E1,L1,A1,M1)</pre>	<pre>#1010011 1000101 1001100 1000001 1001101 # S E L A M print ("Robot Can Konuştu.:") S2 = chr(int('1010011', 2)) E2 = chr(int('1000101', 2)) L2 = chr(int('1001100', 2)) A2 = chr(int('1000001', 2)) M2 = chr(int('1001101', 2)) print (S2, E2, L2, A2, M2)</pre>
Ekran çıktısı:	Ekran çıktısı:
<pre>Ada Konuştu.: 1010011 1000101 1001100 1000001 1001101</pre>	<pre>Robot Can Konuştu.: S E L A M</pre>
Açıklama	Açıklama
Cevirmen1 programını açıklayalım: • ord("S"): S'nin ASCII kod değerini verir. Bu değer 83'tür.	Cevirmen2 programını açıklayalım: • int('1010011', 2): 2 tabanındaki 1010011 sayısını tam sayı tipinde ondalık sayıya dönüştürür. Bu değer 83'tür.

- `bin (ord("S"))`: Elde edilen değerin (83'ün) ikili (binary) sayı karşılığını verir. Bu değer `0b1010011`'dir.
- `0b` ikili sayı sisteminde olduğunu belirten bir ön ektir. Bu ön eki kaldırmak için kod satırına `[2:]` eklenir.
`[2:]` string ifadenin soldan ilk 2 karakterini siler.
- Böylece `bin (ord("S"))[2:]` kod satırı ile `S`'nin makine dili eşdeğeri: `1010011` elde edildi.

Bu program Ada'nın konuşmasını Robot Can'ın anlayacağı dile çevirmektedir.

- `chr(int('1010011', 2))`: `chr(83)`'ün ASCII kod tablosundaki karakter karşılığını yani `S`'yi verir.
- `ord("S")` ile `chr(83)` birbirinin tersi işlem yapar. Biri `83`'ü diğeri `S`'yi döndürür.
- Diğer karakterler de benzer şekilde elde edilir.

Bu program Cevirmen1'in tersini yapmaktadır. Robot Can'ın konuşmasını Ada'nın anlayacağı dile çevirmektedir.

Proje 12.3. Çevirmen 2

Bir önceki projedeki iki ayrı programı (fonksiyonu) tek bir modül (dosya) içerisinde birleştirip, çalıştıralım.

Python dili kodlaması

```
def Cevirmen1():
    print ("Ada Konuştu.:")
    S1 = bin (ord("S"))[2:]
    E1 = bin (ord("E"))[2:]
    L1 = bin (ord("L"))[2:]
    A1 = bin (ord("A"))[2:]
    M1 = bin (ord("M"))[2:]
    print (S1,E1,L1,A1,M1)

def Cevirmen2():
    print ("Robot Can Konuştu.:")
    S2 = chr(int('1010011', 2))
    E2 = chr(int('1000101', 2))
    L2 = chr(int('1001100', 2))
    A2 = chr(int('1000001', 2))
    M2 = chr(int('1001101', 2))
    print (S2, E2, L2, A2, M2)

#Ana program
Cevirmen1() #Önce Cevirmen1 çağrıldı
Cevirmen2() #Sonra Cevirmen2 çağrıldı
```

Ekran çıktısı

```
Ada Konuştu.:
1010011 1000101 1001100
1000001 1001101
Robot Can Konuştu.:
S E L A M
```

Rastgele Sayı Üretimi ve Oyun Programlama

Python'da rastgele sayı üretimi için `random` modülüne ait `random()`, `randrange()`, `randint()`, `uniform()` gibi fonksiyonlar kullanılır. `Random` modülüne ait bu fonksiyonları kullanabilmek için program başında `import random` komut satırını eklemek gerekir. Aşağıdaki tabloda `random` modülüne ait bazı fonksiyonların açıklaması verilmiştir.

`random` modülüne ait fonksiyonlar:

Fonksiyon	Açıklama
<code>random.random()</code>	0 - 1 arasında float tipinde rastgele bir sayı üretir.
<code>random.randint(s1,s2)</code>	s1 - s2 arasında int tipinde rastgele bir sayı üretir.
<code>random.randrange(s1,s2,adım)</code>	s1 - s2 değerleri arasında adım miktarına dikkat edilerek float tipinde rastgele bir sayı üretir, adım belirtilmezse 1 kabul edilir.
<code>random.uniform(s1,s2)</code>	s1 - s2 arasında float tipinde rastgele bir sayı üretir.
<code>random.choice([Liste])</code>	Listeden rastgele bir eleman seçer.
<code>random.shuffle([Liste])</code>	Listeyi rastgele sıralar.
<code>random.sample([Liste], a)</code>	Listeden rastgele a adet eleman seçer.

Python dili `random` modülünde yer alan fonksiyonların güncel ve tam bir listesine <https://docs.python.org/3.10/library/random.html> adresinden erişebilirsiniz.

Proje 12.4. Rastgele Sayı Üretme

1 ile 100 arasında 5 adet rastgele tam sayı üreten programı kodlayalım.

1-100 arasında rastgele tam sayılar üretmek için gerekli komut satırı:

`randint(1,100)`

Python dili kodlaması	Ekran çıktısı
<pre>import random #5 adet rasgele sayı üret for i in range (5): #0-100 arasında sayı üretir print (random.randint(1,100))</pre>	<pre>62 13 89 100 2 #Bu liste her çalıştırmada değişir</pre>

Proje 12.5. Sayısal Loto Çekilişi

Bilgisayarın rastgele ürettiği 1-49 arasındaki 6 adet sayıyı sayısal loto çekiliş sonucu olarak ekrana yazan programı kodlayalım.

Python dili kodlaması	Ekran çıktısı
<pre>#Sayısal Loto oyunu import random print("Sayısal Loto Çekilişi\n") #6 adet random sayı üret for i in range (6): #1-50 arası sayı üret sL=random.randint(1,49) #sayıları yan yana yaz print (sL, end=' ')</pre>	<pre>Sayısal Loto Çekilişi 27 47 38 18 31 33</pre>

Sıra Sende Etkinliği

Sayısal Loto Çekiliş programını, aynı sayıları üretmeyecek şekilde yeniden kodlayabilir misin?

Proje 12.6. Twitter Çekilişi

Bir yazar, tarihli bir sosyal medya (Twitter) takipçisine son çıkan kitabını hediye edecektir. Bu yazarın takipçi listesinden rastgele birini seçen programı kodlayalım.



#çekiliş var. Bu tweeti sektiren ve bu hesabı takip eden bir takipçime 'herkes için #python' kitabı #hediye edilecektir.

Çekiliş tarihi: 20.01.2020

Programlama - Bülent Çobanoğlu - Herkes İçin Python

Python dili kodlaması	Ekran çıktısı
<pre>import random liste = ["@enes", "@can", "@ali", "@ada", "@eko", "@berk", "@ela", "@alp", "@cefa", "@vefa"] #Listeden tarihli seçiliyor tarihli = random.choice(liste) print("Python kitabı çekiliş tarihli:") print(tarihli)</pre>	<pre>Python kitabı çekiliş tarihli: @alp</pre>

Proje 12.7. Sayı Tahmin Oyunu

Bilgisayarın tuttuğu (0-100 arasındaki) sayıya göre kullanıcıyı "daha küçük veya daha büyük sayı giriniz" gibi yönlendiren ve kaç tahminde sayıyı bulduğunu ve puanını söyleyen bir sayı tahmin oyununu kodlayalım.

Python dili kodlaması

```
import random
#Bilgisayar 0-100 arası bir sayı tutar
sayi = random.randint(1,100)
sayac=0
puan=100
print("İlk Tahmininiz.:");
while True:
    tahmin =int(input())
    sayac = sayac + 1
    if tahmin == sayi:
        print("Bravo!",sayac,".de bildiniz")
        break #döngüden çık
    elif tahmin < sayi:
        print("Daha büyük bir sayı gir!")
    else:
        print("Daha küçük bir sayı gir!")
        puan = puan - 10
print("Puanınız.: ",puan)
```

Ekran çıktısı

```
İlk Tahmininiz.:
11
Daha küçük bir sayı gir!
11
Daha küçük bir sayı gir!
22
Daha büyük bir sayı gir!
11
Daha küçük bir sayı gir!
28
Daha büyük bir sayı gir!
10
Daha büyük bir sayı gir!
12
Bravo! 7 .de bildiniz
Puanınız.: 40
```

Açıklama:

Bilgisayarın 1-100 arasında rastgele ürettiği tamsayı **sayi** isimli değişkende tutulmaktadır. Kullanıcının klavyeden girdiği tahminler ise **tahmin** isimli değişkende tutulmaktadır. Bu değişken değerine göre kullanıcının tahmin ettiği sayı ile bilgisayarın tuttuğu sayıyı karşılaştırıp ona göre kullanıcıyı yönlendirilmektedir. Kullanıcının kaç defada bilgisayarın tuttuğu sayıyı bildiği bilgisini **sayac** değişkeninde, puanını da **puan** isimli değişkende tutuyoruz. Her tahmin edileni bilememe durumunda sayacı 1 (**sayac+=1**) artırıyor, puanı ise 10 (**puan-=10**) puan azaltıyoruz. Sonuçları **print** komutları ile ekrana yazdırıyoruz.

Proje 12.8. Taş, Kağıt, Makas Oyunu

Bilgisayar ile kullanıcı arasında taş-kâğıt-makas oyunu oynayan, kimin ne seçtiğini ve kazananı ekranda gösteren bir programı kodlayalım.

Oyun kuralı:

- Taş durumu için el yumruk yapılır.
- Kâğıt durumu için el tamamen açılır.
- Makas durumu için sadece işaret ve orta parmak V şeklinde açılır.

Kazanma durumu:

- Taş, makası kırarak yener.
- Kâğıt, taşı sararak yener.
- Makas, kâğıdı keserek yener.



Python dili kodlaması

```
import random
Liste = ["Taş", "Kağıt", "Makas"]
#PC için listeden rastgele bir seçim yap
pc = random.choice(Liste)
#Oyuncunun seçiminin ilk harfini capitalize() ile büyük harfe dönüştürdük.
sen = input('[Taş-Kağıt-Makas]?').capitalize()

#Bilgisayarın seçimi:
print("Bilgisayar:", pc, "seçti")
#Oyuncunun seçimi:
print("Sen:", sen, "seçtin")
```

```
#9 ihtimal tek tek değerlendiriliyor
if pc == sen:
    print("İkinizde aynı şeyi seçtiniz, Berabere!")
if pc == "Kağıt" and sen == "Makas":
    print("Makas, kâğıdı keser (Sen kazandın!)")
if pc == "Makas" and sen == "Taş":
    print("Taş, makası kırar (Sen kazandın!)")
if pc == "Taş" and sen == "Kağıt":
    print("Kağıt, taşı sarar (Sen kazandın!)")
if pc == "Makas" and sen == "Kağıt":
    print("Makas, kâğıdı keser (Bilgisayar kazandı!)")
if pc == "Taş" and sen == "Makas":
    print("Taş, makası kırar (Bilgisayar kazandı!)")
if pc == "Kağıt" and sen == "Taş":
    print("Kağıt, taşı sarar (Bilgisayar kazandı!)")
```

Ekran çıktısı

```
[Taş-Kağıt-Makas]?Taş
Bilgisayar: Taş seçti
Sen: Kağıt seçtin
Kağıt, taşı sarar (Sen kazandın!)
```

Sıra Sende Etkinliği

Taş-Kağıt-Makas oyununu, yanda verilen çıktıdaki gibi kullanıcı devam etmek istediği sürece devam edecek şekilde yeniden kodlayabilir misin?

```
[Taş-Kağıt-Makas]?Taş
Bilgisayar: Taş seçti
Sen: Taş seçtin
İkinizde aynı şeyi seçtiniz, Berabere!
Devam etmek istiyormusunuz? [e/h]e
[Taş-Kağıt-Makas]?Makas
Bilgisayar: Kağıt seçti
Sen: Makas seçtin
Makas, kâğıdı keser (Sen kazandın!)
Devam etmek istiyormusunuz? [e/h]h
Program sonlandırıldı!
```

Proje 12.9. Çarpım Tablosu Ezberletme Oyunu

Haydi çarpım tablosunu ezberleme oyunu kodlayalım.

Programın açıklaması kod satırları arasında verilmiştir.

Python dili kodlaması

```
#Çarpım tablosu oyunu(1-10)
import random
print ("*Çarpım oyunu*\nÇıkmak için sıfır giriniz!!!»)
basarili = 0 #başarı durum sayacı
basarisiz = 0 #başarısız durum sayacı
#Oyun sonsuz döngüsü
while True:
    a = random.randint(1,10) #1-10 arası rasgele üretilen 1.sayı
    b = random.randint(1,10) #1-10 arası rasgele üretilen 2.sayı
    Soru = "{} * {} = ".format(a,b) # a * b = ? formatında veri alır

    dCvp = a * b #doğru cevap

    cvp = input(Soru)
    #oyundan çıkma durumu:
    if int(cvp) == 0: #Eğer cevap sıfır girilirse
        break #oyunu sonlandır

    #cevaba göre verilen mesaj
    if int(cvp) == dCvp:
        basarili +=1
        print("Cevap doğru!")
    else:
        basarisiz +=1
        print("Cevap hatalı")

#Test durumu
if basarili > basarisiz:
    print ("Tebrikler, çarpman iyi!")
else:
    print ("Üzgünüm, ezber yapman gerek.")
```

Ekran Çıktısı

```
*Çarpım oyunu*
Çıkmak için sıfır giriniz!!!
10 * 7 = 
Cevap doğru!
3 * 6 = 
Cevap doğru!
5 * 10 = 
Cevap hatalı
7 * 9 = 
Tebrikler, çarpman iyi..
```

Proje 12.10. Adam Asmaca (Hangman) Oyunu

Programlama dillerinin tutulduğu bir liste oluşturalım. Bu listeden rastgele bir programlama dilini bilgisayar seçsin ve oyuncu da bilgisayarın seçtiği kelimeyi (programlama dilini) 10 tahminde bulmaya çalışsın. Şimdi adam asmaca (Hangman) oyununu kodlayalım.

Programın açıklaması kod satırları arasında verilmiştir.

Python dili kodlaması

```
import random #random modülü import ediliyor
#Kelime (programlama dilleri) listesi
liste = ["python","basic", "swift", "php", "java", "javasc-
ript", "scratch", "ruby", "delphi"]
#listeden rastgele birini seç
kelime = random.choice(liste)
#her yanlış harfte ekranda gösterilecek 7 elemanlı hangman
listesi
hangman = ['']
```

birden fazla satırdan oluşan bir **string** ifade 3 tırnak arasında gösterilir.

The figure displays 12 diagrams arranged in a 4x3 grid, illustrating the evolution of a quantum state over time. Each diagram features a vertical line with various symbols (plus, minus, slash, circle) and horizontal bars. The diagrams are labeled with time steps: $t=0$, $t=1$, $t=2$, $t=3$, $t=4$, $t=5$, $t=6$, $t=7$, $t=8$, $t=9$, $t=10$, and $t=11$. The symbols and bars change from one diagram to the next, representing the state of the system at different times.

Ekran çıktısı

Kelime: +---+
Doğru
Kelime: s

e
Yanlış
Kelime: s
o

a
Doğru
Kelime: as
o

b
Doğru
Kelime: bas
o

4
Doğru
Kelime: basi
o

```

dogruHarf = [] #doğru harf listesi
yanlisHarf = [] #yanlış harf (hatalı durum) listesi
hak = len(hangman) #hangman eleman sayısı kadar hak
                    #verilir

while hak>0: #hak sıfırdan büyük olduğu sürece
    out="" #çıktı değişkeni
    for h in kelime: #kelimenin harfleri üzerinde dolaşalım
        if h in dogruHarf: #doğru harf ise
            out = out + h #çıktıya o harfi ekle
        else: #yanlış harf ise
            out = out + " " #çıktıya _ ekle
    if out == kelime: #çıktı, seçilen kelimeye eşitse
        break #döngüden çık
    print ( "Kelime: ", out) #Mevcut çıktıyı yaz
    print (hangman[hak-1]) #listedeki görseli yaz
    girHarf = input() #harf gir ve girilen harfi karşılaştıır
    if girHarf in dogruHarf or girHarf in yanlisHarf:
        print ( "zaten ", girHarf, " girilmişti!")
    elif girHarf in kelime:
        print ( "Doğru")
        dogruHarf.append(girHarf) #dogruHarf listesine ekle
    else: #girilen harf kelimeye yoksa
        print ( "Yanlış")
        hak = hak -1 #hakkı 1 azalt
        yanlisHarf.append(girHarf) #yanlisHarf listesine ekle
if hak !=0: #hak bitmeden çıkıldı ise Tebrikler
    print ( "Tebrikler, Evet", kelime)
else:
    print ( "Maalesef kelimemiz:", kelime)

```

c
Doğru
Tebrikler,
Evet: basic

Soru ve Cevaplarla Neler Anladık?

- 1** `L = ['a', 't', 'a', 'n', 'i']`
Yukarıdaki listedeki elemanları rasgele listeleyen ve ekranda gösteren programı kodlayınız.

```
import random
L = ['a', 't', 'a', 'n', 'i']
random.shuffle(L)
print (L)
```
- 2** `print (bin(3))`
Komut satırı çıktı olarak ne üretir?

0b11
- 3** 0b ikili sayı sisteminde olduğunu belirten bir ön ektir. Ekran çıktısında bu ön eki kaldırmak için ne yapılmalıdır.

Bu ön eki kaldırmak için kod satırına `[2:]` eklenir. `[2:]` string ifadenin soldan ilk 2 karakterini siler. Örnek:

```
print (bin(3)[2:]) #Çıktı: 11
```
- 4** Aşağıdaki program çıktı olarak ne üretir?

```
import math
print(math.pi)
```

Pi sayısını (3.141592653589793) verir.
- 5** Python dilinde rastgele sayı üretimi için hangi modül kullanılır?

random
- 6** 7-77 arasında rastgele bir tamsayı nasıl üretilir?

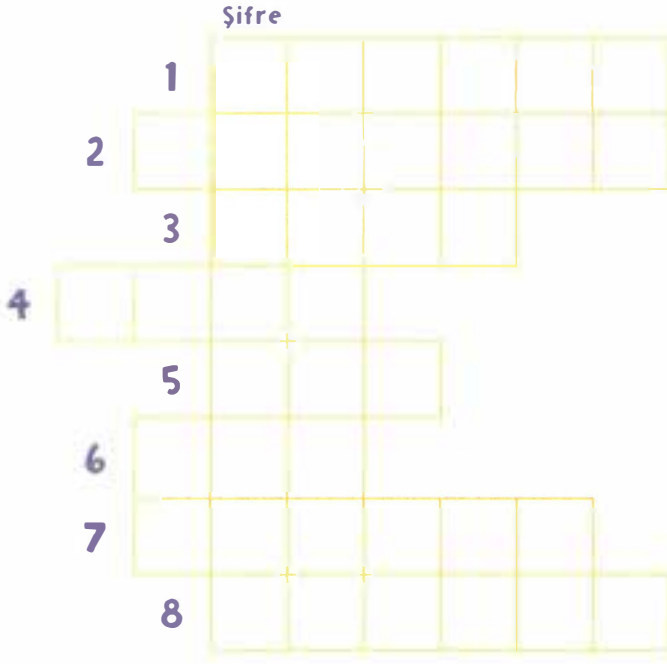
```
random.randint(7,77)
```
- 7** `pow(3, 4)` işlemi daha farklı nasıl kodlanabilir?

`3**4`
- 8** `9**(1/2)` işlemi daha farklı nasıl kodlanabilir?

```
import math
math.sqrt(9)
```
- 9** Bir cümleden/kelimeden rastgele bir harf seçen programı kodlayınız?

```
from random import *
print (choice("adaletyöneticilerdeolursadahaiyidir"))
```

BULMACA



1. Rastgele sayı üretimi için kullanılan modül.
2. Belli bir aralıkta rastgele tamsayı üreten fonksiyon.
3. Sayının karekökünü verir.
4. Matematiksel fonksiyonların yer aldığı kütüphane.
5. "OBEB" işlemini gerçekleştiren fonksiyon.
6. On altılı (hexadecimal) sayı sistemine çevirir.
7. (Tersi) Listedenden belirtilen sayıda elemanı (örneği) seçer.
8. (Tersi) Listedenden bir adet eleman seçer.

Aşağıya bulduğunuz şifreyi kodlayınız.

--	--	--	--	--	--	--	--

Giriş

Kaplumbağa (turtle) çizim grafikleri; turtle isimli modül aracılığı ile programa (from turtle import *) eklenir (import edilir). Turtle modülü ile her türlü çizim işlemi gerçekleştirilebilir. İki boyutlu (2D) veya üç boyutlu (3D) çizimler yapılabilir.

Turtle çizimleri; bir turtle nesnesinin hareketine bağlı olarak gerçekleştiği için aynı zamanda birer robotik hareket (çizgi izleyen veya labirentten çıkan robot gibi) uygulaması olarak da kabul edilebilir. Dolayısıyla robotik kodlama ile ilgilenenlerin bu temel çizim kavramlarının mantığını anlaması gerekir.

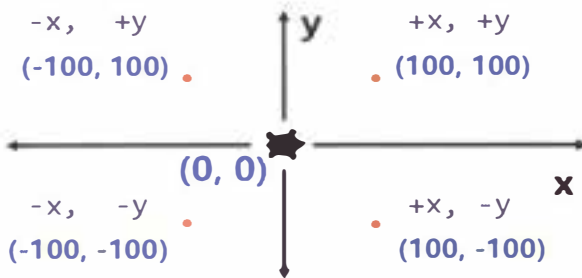
Örneğin robot bir fareyi, peynire ulaştıran yolun (her bir karenin 100 adımlık yol olduğu kabulü ile) algoritmasını ve Python dilinde kodlamasını aşağıdaki gibi yapabiliriz;

Fareyi Peynire ulaştırın yol:	Algoritması:	Python dili kodlaması:	Programı çalıştırdıktan sonra
100 adım=1 kare 	• Başla 1. 100 adım İleri Git; 2. 100 adım İleri Git; 3. 90 derece Sağa Dön; 4. 100 adım İleri Git; 5. 100 adım İleri Git; 6. 100 adım İleri Git; • Dur	<pre>forward (100) forward (100) right (90) forward (100) forward (100) forward (100)</pre>	

Grafik Koordinat Sistemi

Kaplumbağa (Turtle) çizim ekranı koordinatları bildiğimiz matematiksel (kartezyen) koordinat sistemi ile aynıdır.

Turtle grafik ekranının tam orta noktası, koordinat sisteminin merkezidir. Çizim ekranının merkez koordinatları 0, 0 noktasıdır.



Koordinat sisteminde

- Merkezden sağa doğru uzanan yatay çizgi pozitif x
- Merkezden sola doğru uzanan yatay çizgi negatif x

- Merkezden yukarı doğru uzanan dikey çizgi pozitif y
- Merkezden aşağı doğru uzanan dikey çizgi ise negatif y değerlerini içerir.

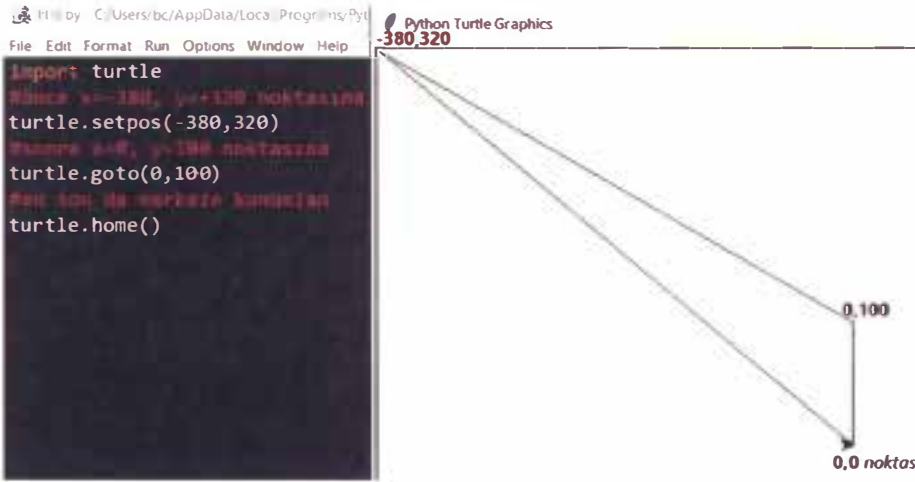
Python koordinat sisteminin ölçü birimi olarak piksel (İngilizcesi: pixel) kullanılır. Piksel bir grafiksel görüntüyü oluşturan noktalardan her biridir.

Kaplumbağa (Turtle) grafik ekranın çizim başlangıç noktası 0, 0 noktasıdır. Ekranın herhangi bir noktasında bulunan pozisyonu/koordinatı öğrenmek için `position()` veya `pos()` komutları kullanılır. Örneğin;

```
>>> import turtle
>>> turtle.position()
(0.00, 0.00)
```

Ekranın herhangi bir noktasına konumlanmak için ise `goto(x,y)`, `setposition(x,y)` veya `setpos(x,y)` komutlarından biri kullanılabilir. Tekrar grafik ekranın başlangıç noktasına (0, 0) konumlanmak için ise `reset()` veya `home()` komutları kullanılabilir.

Örneğin önce ekranın sol üst köşesine sonra (0,100) noktasına en sonunda da merkeze konumlanmak için aşağıdaki kod satırlarını yazmak gerekir;



Sadece yataydaki x ekseninin konumunu değiştirmek için `setx(x)`, dikeydeki y ekseninin konumunu değiştirmek için ise `sety(y)` komutları kullanılabilir.

Doğru Çizimi veya Doğrusal Hareket

Doğru çizimi veya doğrusal hareket ileri veya geri hareketler ile sağlanır. İleri yönde hareket için `forward(mesafe)` veya kısaca `fd(mesafe)`, geri yönde hareket için `backward(mesafe)` veya kısaca `bk(mesafe)` fonksiyonları kullanılır.

Çizim veya hareket yönünü değiştirmek için sola ya da sağa dönüş yapılır. Sola dönüş için left (derece) veya kısaca lt(derece), sağa dönüş için ise right (derece) veya kısaca rt(derece) fonksiyonları kullanılır.

Doğru çizerken yatay ve dikey eksenlerin başlangıç ve bitiş noktalarına dikkat edilmelidir. Örneğin robotik bir kaplumbağanın (turtle) başlangıç olarak koordinat sisteminin merkezinde (0,0) olduğunu farz edelim. import turtle komut satırından sonra, robota turtle.forward(100) komutu verildiğinde, robotumuz 100 piksel ileri yönde hareket eder, yani 100 piksel yatayda bir çizgi çizer. Sonrasında ise turtle.right(90) komutu verildiğinde robot, 90 derece saat yönünün tersi istikametine dönüş yapar.

Proje 13.1. Kare Çizimi

Ekrana kare çizen veya turtle robota karesel hareket yaptıran uygulamayı kodlayalım.

Bir kareyi 4 adet doğru ile çizdirebiliriz.

Algoritması	Python dili kodlaması	Ekran çıktısı
1. Başla 2. 100 piksel İleri Git, 3. 90 derece Sola Dön, 4. 100 piksel İleri Git, 5. 90 derece Sola Dön, 6. 100 piksel İleri Git, 7. 90 derece Sola Dön, 8. 100 piksel İleri Git, 9. Dur	<pre>File Edit Format Run Option #Kare çizimi from turtle import * forward (100) #- left (90) forward (100) # left (90) forward (100) #- left (90) forward (100) # </pre>	

Not. Bu örneğimizdeki gibi import turtle komutu from turtle import * şeklinde yazıldığında alt satırlardaki komutlarda "turtle" ön eki kullanılmaz.

Sıra Sende Etkinliği-1

Ekrana kare çizen Proje 13.1 örneğini for döngüsü ile yeniden kodlayabilir misin?

Farklı Turtle Simgeleri

Ekrana kaplumbağa simgesini çizdiren komut satırı shape('turtle')'dir. Shape fonksiyonu ile arrow, blank, circle, classic, square, triangle, turtle simgeleri çizdirilebilir.

Varsayılan shape değeri classic'dir. Örneğin;

- Kaplumbağa simgesi için shape("turtle"),
- Daire simgesi için shape("circle"),
- Kare simgesi için shape("square"),
- Üçgen simgesi için shape("triangle"),
- Ok simgesi için shape("arrow") komut satırları kullanılır.

Güncel kaplumbağa (turtle) şekillerinin isimlerini liste halinde öğrenmek için getshapes() komutu kullanılabilir;

```
>>> import turtle
>>> turtle.getshapes()

['arrow', 'blank', 'circle', 'classic', 'square', 'triangle', 'turtle']
```

Ayrıca hideturtle() veya shape('blank') ile mevcut Turtle simgesi gizlenebilir. showturtle() ile tekrar gizlenen simge ekrana getirilebilir.

Proje 13.2. İç İçe Kare Çizimi

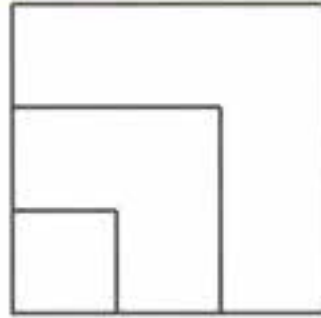
Ekrana farklı boyutlarda iç içe 3 adet kare çizen programı kodlayalım.

Python dili kodlaması

```
from turtle import *
#kare çizme fonksiyonu
def kareCiz(mesafe):
    for x in range(1, 5):
        forward(mesafe)
        left(90)

#Ana program
#Kaplumbağa simgesi yok
shape('blank')
#kalem kalınlığı 2 birim
pensize(2)
#iç içe 3 kare çiz
kareCiz(50)
kareCiz(100)
kareCiz(150)
```

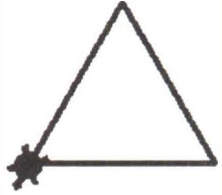
Ekran çıktısı



Not. Çizgi kalınlığını ayarlamak için pensize(kalınlık) komutu kullanılır.

Proje 13.3. Üçgen Çizimi

Bir kaplumbağa robota üçgensel hareket yaptıran (ekrana üçgen çizen) uygulamayı kodlayalım.

Algoritması	Python dili kodlaması	Ekran çıktısı
1. Başla 2. Kaplumbağa simgesi çiz (<code>shape('turtle')</code>) 3. İleri doğru 100 birim git 4. Sola doğru 120 derece dön 5. İleri doğru 100 birim git 6. Sola doğru 120 derece dön 7. İleri doğru 100 birim git 8. Dur	<pre> from turtle import * #kaplumbağa simgesi seçildi shape('turtle') forward(100) left(120) forward(100) left(120) forward(100) </pre>	

Proje 13.4. Merdiven Çizimi

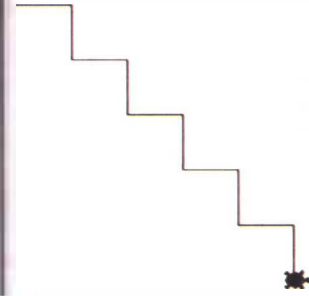
Bir kaplumbağa robotu, merdivenden 5 basamak aşağı indiren (veya 5 basamaklı bir merdiven şeklini ekrana çizen) uygulamayı kodlayalım.

(0,0) başlangıç noktasından; bir döngü içerisinde 50 birim yatayda hareket ve 90 derece sağa dönüşten sonra tekrar 50 birim yatay hareket ve 90 derece sola dönüş yapılarak merdiven basamakları oluşturulabilir.

```

#merdiven5.py - C:/Users/bc/AppData/...
File Edit Format Run Options Window Help
#5 basamaklı merdiven çizimi
from turtle import *
shape('turtle') #kaplumbağa çizer
for i in range(5):
    forward(50)
    right(90)
    forward (50) #
    left (90)

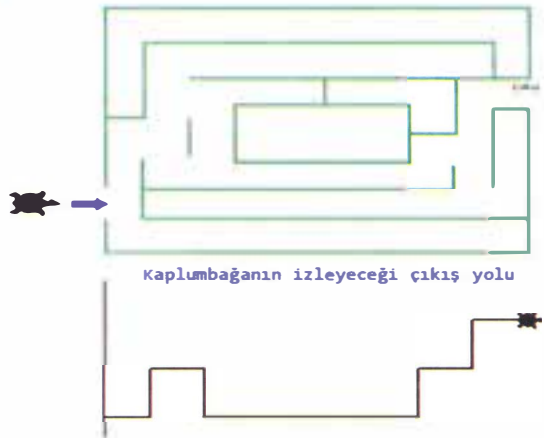
```



Proje 13.5. Labirentten Çıkış

Kaplumbağa robotu yandaki labirentten çıkaracak programı kodlayalım.

Amacımız kaplumbağanın izleyeceği çıkış yolunu çizdirmek. Dolayısıyla programımız yandaki çıkış yolunu çizecektir.



Algoritması	Python dili kodlaması
1. Başla 2. (-390, 0) noktasına konumlan 3. Ekranı temizle (clear) 4. Kaplumbağayı çiz 5. İleri(sağa) doğru 50 birim git 6. Sola doğru 90 derece dön 7. İleri doğru 50 birim git 8. Sağa doğru 90 derece dön 9. İleri doğru 50 birim git 10. Sağa doğru 90 derece dön 11. İleri doğru 50 birim git 12. Sola doğru 90 derece dön 13. İleri doğru 200 birim git 14. Sola doğru 90 derece dön 15. İleri doğru 50 birim git 16. Sağa doğru 90 derece dön 17. İleri doğru 50 birim git 18. Sola doğru 90 derece dön 19. İleri doğru 50 birim git 20. Sağa doğru 90 derece dön 21. İleri doğru 50 birim git 22. Dur	<pre> from turtle import * goto (-390,0) #Başlangıç noktası clear() #ekranı temizle shape('turtle') #İlmeğe başlarken forward(50) left(90) forward(50) right(90) forward(50) right(90) forward(50) left(90) forward(200) left(90) forward(50) right(90) forward(50) left(90) forward(50) right(90) forward(50) </pre>

Renk Komutları

Çizimleri ve çizgileri renklendirmek için color() fonksiyonu kullanılır. Kullanım şekli;

```
color ("kalem rengi", "dolgu rengi")
```

color() fonksiyonun ilk parametresi çizgi kalem (pen) rengi, ikinci parametresi ise dolgu (fill) rengidir. Bu iki parametre için ayrı ayrı fonksiyonlarda kullanılabilir:

Kalem rengi için pencolor(),

Dolgu rengi için fillcolor() fonksiyonları kullanılabilir.

Ayrıca çizim ekranı (screen) arka plan rengini ayarlamak için bgcolor() fonksiyonu kullanılır.

Belirtilen renk komutundan sonra çizdirilecek şeklin içini de boyamak için çizim komutları begin_fill() ve end_fill() fonksiyonları arasına yazılır.

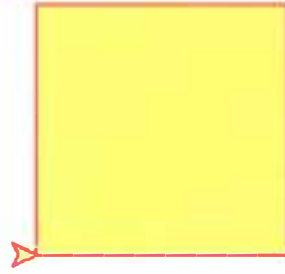
Proje 13.6. Sarı Kırmızı Kare

Çizgi rengi kırmızı, iç rengi sarı olacak şekilde bir kare çizen programı kodlayalım.

Algoritması

```
from turtle import *
#kare çizen fonksiyon
def kareCiz():
    for x in range(4):
        forward(100)
        left(90)
#çizgi rengi kırmızı, içi sarı
color ("red", "yellow")
#şeklin içini de boy
begin_fill()
kareCiz()
end_fill()
```

Python dili kodlaması



Açıklama: Yukarıdaki programda kare çizgilerini kırmızı, iç alanını da sarı renge boyamak için kare çizen kareCiz() fonksiyonu begin_fill() ve end_fill() komutları arasına alınmıştır.

Renk Tablosu

Renk komutları ile kullanılabilecek renk değerleri aşağıdaki tabloda verilmiştir. Dikkat edilirse renk fonksiyonları içerisine renklerin İngilizce karşılıkları doğrudan yazılabilmektedir. Renklerin İngilizce karşılıkları yerine RGB sistemindeki sayı karşılıkları da yazılabilir.

Grafik uygulamalarındaki tüm renkler, 3 ana rengin **RGB - Red** (Kırmızı), **Green** (Yeşil), **Blue** (Mavi) karışımından elde edilir. Bu üç ana rengin her biri 0 ile 255 arasında bir değere sahiptir. Buna göre de $256 \times 256 \times 256 = 16.777.216$ farklı renk birleşimi ortaya çıkmaktadır. RGB fonksiyonu kullanılmadan önce colormode(255) komutu ile RGB modu aktif hale getirilmelidir. Örneğin mavi renk için color("blue") yerine; colormode(255); color(0,0,255) komut satırları kullanılabilir.

Turtle renk değer listesi:

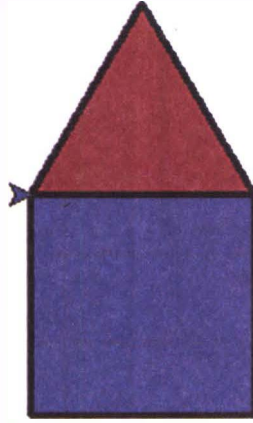
Renk	Renk ismi	RGB değeri (r, g, b)
Siyah	black	(0, 0, 0)
Beyaz	white	(255, 255, 255)
Mavi	blue	(0, 0, 255)
Camgöbeği	cyan	(0, 255, 255)
Kırmızı	red	(255, 0, 0)
Pembe	pink	(255, 192, 203)
Portakal	orange	(255, 200, 0)
Kahverengi	brown	(165, 42, 42)

Sarı	yellow	(255, 255, 0)
Altın	gold	(255, 215, 0)
Gri	gray	(128, 128, 128)
Koyu Gri	dark gray	(169, 169, 169)
Eflatun	magenta	(255, 0, 255)
Yeşil	green	(0, 255, 0)
Mor	purple	(128, 0, 128)
Menekşe rengi	violet	(238, 130, 238)

Not. Daha fazla renk kodu için <https://trinket.io/docs/colors> adresine bakabilirsiniz.

Proje 13.7. Ev Çizimi

Şekildeki gibi kahverengi üçgen çatılı ve mavi kareli bir ev çizen programı kodlayalım.

Python dili kodlaması	Ekran çıktısı
<pre> from turtle import * #kare çizme fonksiyonu def kareCiz(boyut): for x in range(1, 5): forward(boyut) right(90) #üçgen çizme fonksiyonu def ucgenCiz(boyut): for x in range(1, 4): forward(boyut) left(120) #ana program pensize(3) #çizgi kalınlığı begin_fill() #üçgeni boya fillcolor("brown") #içi kahve ucgenCiz(100) end_fill() #boyamayı kaldır begin_fill() #kareyi boya fillcolor("blue") #içi mavi kareCiz(100) end_fill() #boyamayı kaldır </pre>	

Proje 13.8. Yıldız Çizimi

Ekranı çizgi rengi kırmızı, dolgu rengi sarı bir yıldız çizen programı kodlayalım.

Python dili kodlaması	Ekran çıktısı
<pre>#Yıldız çizimi from turtle import * shape('blank') #ok kullanma pensize(3) #çizgi kalınlığı pencolor("red") #çizgi rengi fillcolor("yellow") #dolgu rengi begin_fill() #boyama başla for i in range(5): forward(100) right(144) end_fill() #boyamayı bitir</pre>	

Daire Çizimi

Daire (çember), yay ve çokgen (poligon) çizimi için

circle(yarıçap, açı, adım) fonksiyonu kullanılır.

Bu parametrelerden yarıçap değeri zorunlu, diğerleri zorunlu değildir. Açı belirtilmezse 360 derece (tam bir daire) kabul edilir.

Örnek daire tanımlamaları ve açıklaması;

Örnek kullanım	Açıklaması
<code>circle(-100)</code>	100 birim yarıçaplı saat yönünde hareket ederek tam bir daire çizer.
<code>circle(50)</code>	50 birim yarıçaplı saat yönü tersinde hareket ederek tam bir daire çizer.
<code>circle(100,180)</code>	100 birim yarıçaplı yarım daire (360/2) (yay) çizer.
<code>circle (-100, 110)</code>	100 birim yarıçaplı saat yönünde 110 derecelik bir yay çizer.
<code>for i in range(3): circle (100, 120, 2)</code>	Altıgen (100 birim yarıçaplı, 120 derecelik açığa sahip 2 şer adımlı) çizer.

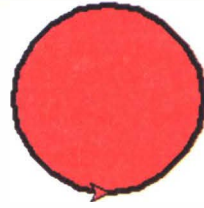
Proje 13.9. Daire Çizimi

Saat yönünün tersinde hareket ederek yarıçapı 50 birim ve dolgu/iç rengi kırmızı olan bir daire çizelim.

Python dili kodlaması

```
from turtle import *
pensize(3) #kalın uç
#kalem: siyah, iç dolgu: kırmızı renkli
color("black", "red")
begin_fill() #boyama başla
circle(50) #yarıçapı:50
end_fill() #boyamayı bitir
```

Ekran çıktısı



Proje 13.10. Japon Bayrağı Çizimi

30 birim yarıçaplı, 150*100 çerçeve içerisinde bir Japon bayrağı çizelim.

Japon bayrağı; bir beyaz çerçeve içerisinde kırmızı daire olan bayraktır. Buna göre önce dikdörtgen çerçeveyi çizip sonra içi dolu kırmızı renkte bir daire çizeceğiz.

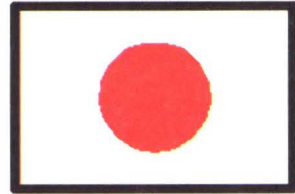
Python dili kodlaması

```
#japon bayrağı
from turtle import *
shape('blank')
pensize(5) #kalın çerçeve
#dikdörtgen çerçeveyi çiz
for i in range(2):
    forward(150)
    right(90)
    forward(100)
    right(90)

pencolor("red") #kalem rengi

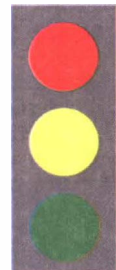
penup() #kalemi kaldır
goto(75,-80) #daire çizimi için konumlan
pendown() #kalemi koy
fillcolor("red") #içini kırmızıya boya
begin_fill() #boyama başla
circle(30) #yarıçapı:30 olan daireyi çiz
end_fill() #boyamayı bitir
```

Ekran çıktısı



Sıra Sende Etkinliği-2

Şekildeki gibi bir trafik sinyalizasyon (dikdörtgen içerisinde 3 farklı renkte daire) şekli çizebilir misin?



Çokgen (Poligon) Çizimi

Çokgen (Poligon), üç ya da daha fazla düz çizgi (kenar) ile birleştirilen üç ya da daha fazla noktanın (köşenin) oluşturduğu kapalı şekillerdir.

N (çok) tane noktanın birleştirilmesiyle oluşturulan çokgenler N-gen olarak adlandırılır; üçgen, dörtgen, beşgen gibi.

Çokgen çizimi, hem `forward()/backward()` hem de `circle()` fonksiyonları kullanılarak yapılabilir.

Bir çokgenin (poligonun) köşe noktalarını saptamak için `begin_poly()` ve `end_poly()` fonksiyonları kullanılır. `begin_poly()` ile ilk köşesi, `end_poly()` ile son köşesi saptanır.

Proje 13.11. Çokgen Çizimi

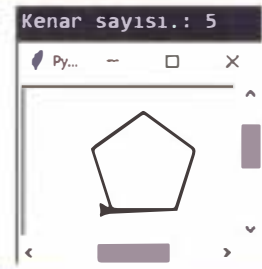
Kenar sayısının dışardan girildiği çokgen çizimi yapan programı kodlayalım.

Projemizde bir kenar sayısı (N) girildikten sonra N-gen (beşgen, altıgen gibi) çokgen çizdirmek için dönüş açısı " $360 / N$ " formülü ile hesaplanır ve `forward()` ya da `circle()` fonksiyonları ile çizim yapılır;

Python dili kodlaması

```
from turtle import *
N= int(input("Kenar sayısı.: "))
aci = 360/N #Açı hesaplandı
pensize(2) #Kalem kalınlığı seçildi
# N-gen çizimi döngü içerisinde yapıldı
for i in range(N):
    forward(50) #50 birim ileri git
    left(aci) #Açı derecesi kadar sola dön
```

Ekran çıktısı



Açıklama: Çokgen çizimini `circle()` ile de yapabilirdik.

N-gen (Çokgen) çizen kod

```
for i in range(N):
    forward(50) #50 birim ileri git
    left(aci) #Açı derecesi kadar sola dön
```

Eşdeğeri

```
for i in range(N):
    circle (50, aci, 1)
```

Proje 13.12. Stamp ile Altıgen Çizimi

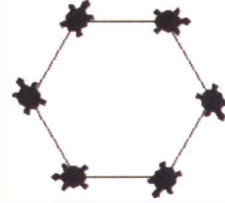
stamp (damga/kaşe) ile ekrana altıgen çizdiren bir programı kodlayalım.

Bu uygulamadaki amacımız kalem (penup()) kullanmadan stamp (damga) ile çizimin nasıl yapıldığını göstermektir.

Python dili kodlaması

```
from turtle import *
shape("turtle") #damga şekli
#360/6 = 60 derece ile 6-gen çizelim
for i in range(6):
    stamp() #her adımda mevcut damgayı bas
    right(60) #sağa dön
    forward(60) #30 birim ileri git
```

Ekran çıktısı



Sıra Sende Etkinliği-3

Sen de stamp (damga/kaşe) ile ekrana aşağıdaki gibi altı çizdiren bir programı kodlamak için programdaki boşluklara ne gelmelidir?

Python dili kodlaması

```
from turtle import *
shape("turtle") #damga şekli
#47 (-1 ... 45) adet damga bas
for i in range(45, -1, -1):
    _____ #mevcut damgayı bas
    _____ #sağa dön
    forward(30) #30 birim ileri git
```

Ekran çıktısı



Veri Girişi için İletişim (Diyalog) Kutularını Kullanma

Bir önceki örnekte olduğu gibi klavyeden(konsoldan) veri girişi için Python kabuk (shell) ekranını kullandık. Turtle grafik ekranında çalışırken klavyeden veri girişi için textinput() ve numinput() diyalog kutularını kullanabiliriz. Kullanım şekli;

```
textinput("Başlık", "Mesaj.:")
#string tipte veri alır. Tamsayı değer girişi için 'int'
sarmalına almak gerekir.
```

```
numinput("Başlık", "Mesaj.:", default, min, max)
```

#float tipte veri alır. Tamsayı değer girişi için 'int' sarılmalına almak gerekir. Giriş ekranında default(varsayılan) olarak bir değer gözükür ve sadece min-max aralığında değer girişini kabul eder.

Proje 13.13. İletişim Kutuları ile Veri Girişi

Kenar sayısının diyalog kutuları ile girildiği çokgen çizimi yapan programı yeniden kodlayalım.

Projemizde bir kenar sayısı (N) girildikten sonra N-gen (beşgen, altıgen gibi) çokgen çizdirmek için dönüş açısı $360 / N$ formülü ile hesaplanır ve circle() fonksiyonu ile de çizilir.

Python dili kodlaması-1: textinput ile	Ekran çıktısı
<pre>#Çokgen çizimi from turtle import * #kenar sayısı giriliyor N = int(textinput("Çokgen", "Kenar sayısı.:")) aci = 360/N #açı hesaplanıyor pensize(2) #kalem ucu kalınlığı #Çokgeni çiz for i in range(N): circle (50, aci, 1)</pre>	
Python dili kodlaması-2: numinput ile	Ekran çıktısı
<pre>#Çokgen çizimi from turtle import * #kenar sayısı giriliyor N = int(numinput("Çokgen", "Kenar sayısı.:", 6)) aci = 360/N #açı hesaplanıyor pensize(2) #kalem ucu kalınlığı #Çokgeni çiz for i in range(N): circle (50, aci, 1)</pre>	

Not. Projeye title(str(N) + "-GEN") komut satırı ile başlık yazısı ekleyebilirsiniz.

Çerçeve İçine Resim Yerleřtirmek

Turtle çizim ekranının/çerçevesinin arka planına `bgpic(dosya.gif)` ile resim yerleřtirebilirsiniz veya arka plan rengini `bgcolor(rengi)` ile deęiřtirebilirsiniz. Fakat `bgpic(dosya)` fonksiyonu sadece ".gif" formatında resimleri kabul etmektedir.

Not. Farklı resim formatlarında ('.png', '*.bmp', '*.jpeg' gibi) bir resim eklemek için Tkinter veya PyGame gibi farklı kütüphaneler kullanılabilir.

Proje 13.14. Arka Plana Resim Yükleme

Bilgisayarda yüklü bir resmi (`enKralTakim.gif`) grafik ekranın arka plan resmi olarak ayarlayan programı kodlayalım.

Burada önemli bir husus, yüklemek istedięimiz resim dosyası (`enKralTakim.gif`) ile .py uzantılı kaynak dosyamızın aynı klasör içerisinde bulunması gerekir.

Python dili kodlaması

```
from turtle import *
#resim çerçeve boyutu
setup(400, 800)
#resim dosyası .gif formatında
bgpic("enKralTakim.gif")
#çerçeve başlığı
title("En Kral Takım")
```

Ekran çıktısı



Proje 13.15. Türk Bayraęı Çizimi

600*400 çerçeve içerisinde bir Türk bayraęı çizelim.



Python dili kodlaması

```
from turtle import *
title ("Türk Bayrağı")      #çerçeve başlığı yazısı
setup(width=600,height=400) #çerçeve boyutu
bgcolor("red")              #çerçeve arka plan rengi:kırmızı
hideturtle()                #turtle nesnesini gizleyelim

def renkveKonum(renk, x, y): #konumlanma fonksiyonu:
    penup()                  #çizim kalemini kaldır
    goto(x, y)               #konumlan
    pendown()                #kalemi tekrar koy
    color(renk)              #boyama rengi: renk değişkeni
    begin_fill()             #boyamaya başla

def yildiz():                #Yıldız çizen fonksiyon:
    renkveKonum("white",80,25) #Önce konumlan
    for i in range(5):       #Sonra döngü içerisinde 5 kez
        forward(50)          #ileri 50 adım git
        right(144)           #144 derece sağa dön
        forward(50)          #ileri 50 adım git
        right(-72)           #-72 derece sağa dön
    end_fill()               #boyamayı sonlandır

def ay(cap):                 #ay çizen fonksiyon:
    circle(cap)              #daire çiz
    end_fill()               #boyamayı sonlandır

#Ana program
renkveKonum("white",-110,-120) #konum fonksiyonunu çağıralım
ay(130)                        #beyaz daire çizelim
renkveKonum("red", -70, -90)  #konumu ve rengi değiştirelim
ay(100)                        #kırmızı daire çizelim
#iki daire farklı renklere boyanarak hilal şekli elde edilmiş oldu.
yildiz()                      #yıldız çizim fonksiyonunu çağıralım
mainloop()                    #Çizim ekranının kaybolmaması için mainloopu hep çağıralım.
```

Not. Kaplumbağayı hızlandırmak: Çok fazla çizim (hareket) içeren uygulamalarda turtle hızı yavaş gelebilir. Çizimi hızlandırmak için `speed(0)` komutu kullanılabilir.

Turtle Metotlarına Toplu Bakış

<http://docs.python.org/3/library/turtle.html> adresinden en güncel Turtle metotlarının tamamını görebilirsiniz. Bu bölümde anlatılan tüm Turtle metotları ve daha fazlası aşağıdaki tabloda verilmiştir.

Sık kullanılan bazı Turtle fonksiyonları:

Turtle fonksiyonları	Açıklama
forward(mesafe) fd(mesafe)	Mesafe kadar aynı/ileri doğrultuda ilerler.
backward bk back (mesafe)	Mesafe kadar ters doğrultuda ilerler.
right(aci) rt(aci)	Belirtilen aci değeri kadar sağa döner.
left(aci) lt(aci)	Belirtilen aci değeri kadar sola döner.
degrees() radians()	Açıların derece ya da radyan cinsinden ölçüsünü belirler. Varsayılan değer: derece için 360, radyan için 2π 'dir.
goto setpos setposition(x,y)	Ekranın x,y noktasına konumlan.
setx(x)	Ekranın x noktasına konumlan.
sety(y)	Ekranın y noktasına konumlan.
setheading(aci) seth(aci)	Doğrultu açısını belirler. Standart mod için; 0-sağ, 90-yukarı, 180-sol, 270-aşağı.
home()	Başlangıç noktasına (0,0) konumlan.
reset()	Ekranı temizler ve başlangıç noktasına konumlanır.
clear()	Ekranı temizler ama konumu değiştirmez.
circle(yarıçap,açı,adım)	Yarıçapı verilen daireyi çizer.
dot(boyut,renk)	Birim noktanın boyut ve rengini belirler.
stamp()	Bulunduğu noktaya mevcut turtle simgesini (damgasını) basar.
clearstamp(no)	Belirtilen noktadaki (no) turtle simgesini kaldırır.
clearstamps(n)	n adet veya belirtilmezse tüm stamp'ları siler.
undo()	Son işlemi geri alır.
speed(n)	Turtle (kaplumbağa) hızını ayarlar, n=0 ise en hızlı, n=1 ise en yavaş hızdadır. n=2 ile 10 arası ise giderek hızlanan değeri ifade eder.
position() pos()	Turtle'ın x, y koordinat değerlerini geri döndürür.
pendown() pd() down()	Kalemi bastırır (hareket sırasında çizgi çizer).
penup() pu() up()	Kalemi kaldırır (hareket sırasında çizgi çizmez).
isdown()	Kalemin durumunu sorgular. penup() ise False, pendown() ise True değerini geri döndürür.
pensize() width()	Çizim kalemi kalınlığını belirler.
pen()	Bir sözlük yapısında kalem ile ilgili parametreleri listeler.
color()	Çizim rengini ayarlar.

<code>pencolor()</code>	Kalem rengini ayarlar.
<code>fillcolor()</code>	Dolgu rengini ayarlar.
<code>filling()</code>	Dolgu kullanıldı ise True değilse False değerini döndürür.
<code>begin_fill()</code>	Boyama işlemini başlatır.
<code>end_fill()</code>	Boyama işlemini sonlandırır.
<code>showturtle()</code> <code>st()</code>	Turtle simgesini/işaretini gösterir.
<code>hideturtle()</code> <code>ht()</code>	Turtle simgesini/işaretini gizler.
<code>isvisible()</code>	Turtle simgesi görünür durumda ise True , değilse False değerini döndürür.
<code>shape('simge')</code>	Turtle simgesini seçer/değiştirir. Simgeler; "arrow, blank, circle, classic, square, triangle, turtle" şekillerinden biri olabilir. Varsayılan shape değeri classic 'dir.
<code>getshapes()</code>	Mevcut Turtle simgelerinin isimlerini listeler: [arrow, blank, circle, classic, square, triangle, turtle] gibi.
<code>resizemode(mode)</code>	mode= "auto" ise turtle simgesi çizgi kalınlığı ile eşdeğerde; mode= "user" ise kullanıcı belirler, mode= "noresize" ise değişiklik yapılmaz.
<code>shapesize()</code> <code>turtlesize()</code>	Turtle simge boyutunu belirler (mode= "user" seçildi ise).
<code>tilt(aci)</code> <code>settiltangle(aci)</code>	Turtle simgesinin yönünü değiştirir.
<code>bgcolor()</code>	Arka plan rengini değiştirir.
<code>bgpic()</code>	Arka plan resmini değiştirir/ayarlar.
<code>setup()</code>	Ekran çerçeve boyutunu (piksel olarak) ayarlar. setup(800,600) gibi.
<code>title()</code>	Ekran pencere başlığını ayarlar.

Soru ve Cevaplarla Neler Anladık?

- 1 Python'da her türlü çizim işlemini gerçekleştirebileceğiniz modülün ismi nedir?
Turtle (Kaplumbağa)
- 2 Turtle modülü programa nasıl eklenir?
`from turtle import *`

3 Yandaki grafik çıktısını verecek programı kodlayınız?

```
from turtle import *
left(90)
forward (150)
left(90)
forward (75)
```

4 Aşağıdaki program çıktı olarak ne üretir?

```
from turtle import *
for x in range(1, 3):
    forward(200)
    left(90)
    forward(100)
    left(90)
```



5 Grafik uygulamalarında farklı yazı tiplerini (fontlarını) kullanmanız gerekebilir. Bu yazı tipleri değişik boyut ve özelliklerde olabilir. Metinlerin yazı tiplerini değiştirmek için hangi turtle metodunu kullanmanız gerekir?

font. Örnek: `'write("mesaj", font=('Arial',12,'normal'))'`

6 Grafiksel ekranın arka planına bir resim dosyası yüklemek için hangi turtle fonksiyonu kullanılır? Ve bu resim dosyasının uzantısı ne olmalıdır?

Turtle grafik ekranının arka plan rengini `bgcolor(renk)` ile değiştirebildiğiniz gibi `bgpic(dosya)` ile de arka plana ".gif" formatında bir resim yerleştirebilirsiniz.

7 Daire (çember), yay ve çokgen (poligon) çizimi için hangi fonksiyon kullanılır?

circle(yarıçap, açı, adım)

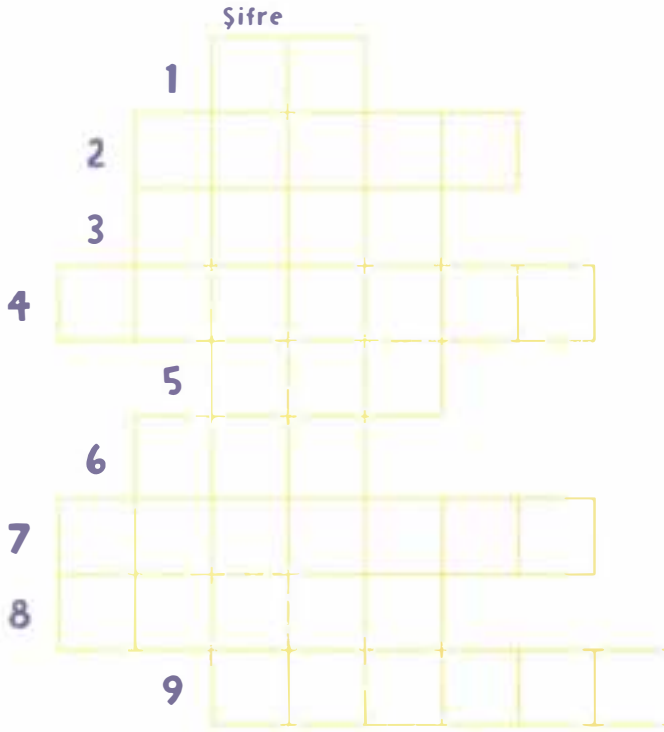
8 Çizim ekranı çerçeve boyutunu genişliğini 800, yüksekliğini 600 birim olarak ayarlamak için hangi komut kullanılır?

setup(800, 600)

9 Hareket sırasında çizgi çizmemek için çizim kalemini kaldırma da hangi komut kullanılır?

penup()

BULMACA



1. (Tersi) **backward** komut kısaltması.
2. Çizim rengini ayarlar.
3. Başlangıç noktasına (0,0) konumlandırır.
4. "İleri" komutu.
5. (Tersi) **color(255,0,0)** renk kodu.
6. Turtle çizim ekranının arka planına **bgpic(dosya.gif)** ile yerleştirilen resim dosyasının uzantısı.
7. Çizim kalemi kalınlığını belirler.
8. Turtle simgesini seçer/değiştirir.
9. Python kaplumbağa çizim modülünün adı.

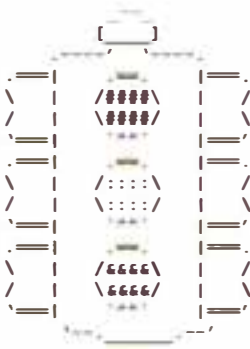
Aşağıya bulduğunuz şifreyi kodlayınız.

--	--	--	--	--	--	--	--	--	--

Bölüm

14

Olay Tabanlı Projeler ve Yılan Oyunu



Giriş

Python turtle modülü ile bir olaya bağlı (fareye tıklama veya klavyeden herhangi bir tuşa basma olayı gibi) kodlar yazılabilir. Kullanıcının fareye tıklaması veya klavyeden bir tuşa basması gibi eylemleri, olay (event) olarak adlandırılır.

Bir önceki bölümde ve aşağıdaki tabloda sunulan Turtle fonksiyonları ile olay tabanlı uygulamalar ve oyun projeleri geliştirilebilir.

Bazı Turtle olayları (events):

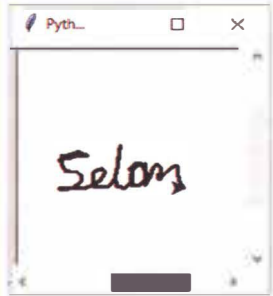
Olay fonksiyonları	Açıklama
<code>onclick(f)</code> <code>onscreenclick(f)</code>	Fareye (mouse) tıklandığında f isimli fonksiyonu çalıştırır. (Fare tıklama olayı).
<code>onrelease(f)</code>	Fare (mouse) butonu serbest bırakıldığında f fonksiyonunu çalıştırır. (Fare butonu bırakılma olayı).
<code>ondrag(f)</code>	Fare (mouse) sürüklemesi olduğunda f fonksiyonunu çalıştırır. (Fare sürüklenme olayı).
<code>listen()</code>	Tuş olaylarını yakalamak için turtle ekranını dinler.
<code>onkey(f,tus)</code> <code>onkeyrelease(f,tus)</code>	İlgili klavye tuşu bırakıldığında f isimli fonksiyonu çalıştırır. (Tuş bırakma olayı).
<code>onkeypress(f,tus)</code>	İlgili klavye tuşuna basıldığında f isimli fonksiyonu çalıştırır. (Tuşa basma olayı).
<code>ontimer(f,t)</code>	t milisaniye sonra f isimli fonksiyonu çağıran bir sayaç (timer) kurar.
<code>mainloop()</code> <code>done()</code>	Olay yakalama döngüsünü başlatır. Bu komut grafik programının en son satırında yer almalıdır.
<code>exitonclick()</code>	Fare'ye (mouse) tıklandığında <code>bye()</code> fonksiyonunu çağırır.
<code>bye()</code>	Grafik penceresini kapatır.
<code>textinput("Başlık", "mesaj")</code>	Bir iletişim (diyalog) kutusu içerisinde string tipte veri girilmesini bekler.
<code>numinput("Başlık", "Mesaj.", default, min, max)</code>	Bir iletişim (diyalog) kutusu içerisinde float tipte veri girilmesini bekler. Kutu içerisinde default değeri gösterir.
<code>write()</code>	Ekrana yazı yazar. Kullanım: <code>write("mesaj",align='center' font=('Arial',9,'normal'))</code>

delay()	Milisaniye cinsinden çizim hareketleri arasındaki gecikmeyi ayarlar. Süre delay(sure) , ne kadar uzarsa animasyon o kadar yavaş olur.
tracer(n, delay)	Animasyon ayarlarını değiştirir. Çizimleri animasyonsuz yapmak için tracer(0) kullanılır. n : n kez ekranı yeniler, delay : milisaniye cinsinden gecikme süresidir.
update()	Çizim ekranını günceller. Özellikle tracer(0,0) komutundan sonra mutlaka kullanılmalıdır.
distance()	Kaplumbağanın konumu ile bilgileri verilen nokta ya da başka bir Turtle nesnesi arasındaki mesafeyi verir.

Bu bölümdeki programların açıklamaları '#' yorum simgesi ile program içerisinde yapılmıştır.

Proje 14.1. Fare (Mouse) ile Yazı Yazma

Grafik ekrana fare (mouse) hareketi (tıklaması) ile çizim yapacak programı kodlayalım.

Python dili kodlaması	Çizim ekranı
<pre> from turtle import * pensize(3) #çizim kalemi kalınlığı wn = Screen() #wn:Ekran değişkeni #Artık program içerisinde Screen() nesnesi #yerine 'wn' kullanılacak wn.setup(200,200) #Çizim ekranı çerçeve boyutu #Tıklanılan noktanın koordinatlarını bulan fonksiyon def Nokta(x, y): goto(x, y) #Tıklanılan noktaya git wn.onclick(Nokta) #Tıklama (click) olayı olduğunda Nokta() fonksiyonunu çalıştır mainloop() #olay yakalama döngüsü ile çizim ekranını göster </pre>	

Proje 14.2. Yön Tuşları ile Çizim Yapma

Grafik ekranına klavyedeki yön tuşlarını kullanarak çizim yapan programı kodlayalım.

Klavyedeki yön okları ile kaplumbağamızı istediğimiz tarafa ilerleterek, çizim yaptırabiliriz. Python dilinde klavyedeki;

- Yukarı ↑ yön tuşu: Up
- Aşağı ↓ yön tuşu: Down
- Sola ← yön tuşu: Left
- Sağa → yön tuşu: Right

isimleri ile kodlanmıştır. Programın açıklaması kod satırları arasında yapılmıştır.

Python dili kodlaması

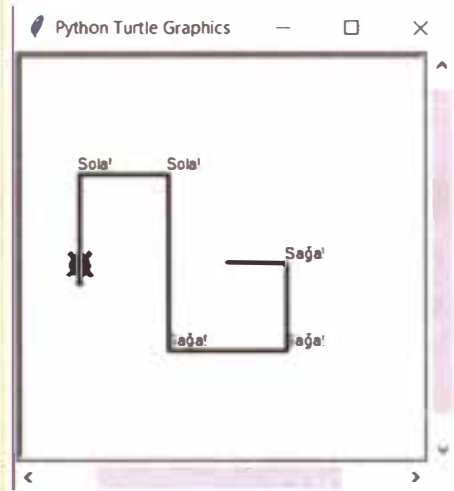
```
from turtle import *
shape('turtle') #Kaplumbağa şekli: 'turtle'
pensize(3)      #Kalem kalınlığı
wn = Screen()   #wn:Ekran değişkeni
#Artık program içerisinde Screen() nesnesi yerine
#wn' kullanılacak
wn.setup(300,300) #Çizim ekranı çerçeve boyutu

#Yön tuşları için tanımlanan fonksiyonlar:
def solaDon():
    left(90)      #Sola 90 derece dön
    write("Sola!") #Yaz 'Sola'
def sagaDon():
    right(90)     #Sağa 90 derece dön
    write("Sağa!") #Yaz 'Sağa'
def gitIleri():
    forward(20)   #20 birim ilerle

#İlgili klavye tuşuna basıldığında çalışacak
fonksiyonlar;
#Sola yön okuna basıldığında 'solaDon'
fonksiyonunu çalıştır
wn.onkeypress(solaDon, 'Left')
#Sağa yön okuna basıldığında 'sagaDon'
fonksiyonunu çalıştır
wn.onkeypress(sagaDon, 'Right')
#Yukarı yön okuna basıldığında 'gitIleri'
fonksiyonunu çalıştır
wn.onkeypress(gitIleri, 'Up')
#Yukarı yön oku

wn.listen()      #Hangi tuşa basıldığını dinle?
wn.mainloop()   #Olay yakalama döngüsünü
başlat
```

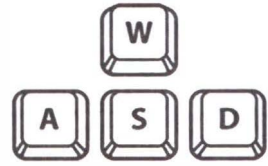
Çizim ekranı



Sıra Sende Etkinliği

Klavyedeki yön tuşlarını değiştirerek aynı çizimi yapabilir misin?

Örneğin ↑ yerine w, ← yerine a, → yerine d tuşlarını kullanabilirsiniz.



Proje 14.3. Durum Değiştiren Trafik Işıkları

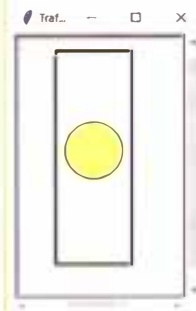
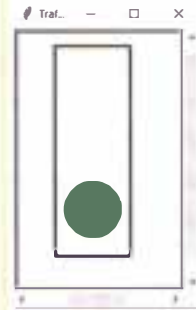
Trafik Işıkları: Klavyeden space (uzun çubuk) tuşuna basıldığında durum değiştiren bir trafik ışık sistemini kodlayalım.

Python dili kodlaması

```
from turtle import *
wn = Screen() #wn:Ekran değişkeni
#Artık program içerisinde Screen() nesnesi yerine 'wn' kullanılacak
wn.setup(300,600) #Çizim ekranı çerçeve boyutu
wn.title("Trafik ışıkları!") #çerçeve başlığı
pensize(3) #çizim kalemi uç kalınlığı
#Trafik durumunu gösteren değişken: trfkDurum
#Yeşil: 0, Sarı: 1, Kırmızı: 2
trfkDurum = 0
#Dikdörtgen levhayı çizen fonksiyon
def levhaCiz():
    for i in range(2):
        forward(80) #kısa kenar
        left(90)
        forward(220) #uzun kenar
        left(90)
#Lambayı çizen fonksiyon
def lambaCiz():
    penup() #çizim kalemini kaldır
    goto(40,40) #çizim için 40,40 koordinatlarına konumlan
    left(90) #sola dön
    fillcolor("green") #ilk önce ışığı yeşil yap
    shape("circle") #Lamba şeklini seç
    shapessize(3) #lambanın büyüklüğü
#Işık (durum) geçişlerini sağlayan durumMakinesi() fonksiyonu
def durumMakinesi():
    global trfkDurum #global değişken
    #0'dan 1'e geçiş durumu:
    if trfkDurum == 0:
        forward(60) #ileri 60 adım git
        fillcolor("yellow") #ışığı sarı yap
        trfkDurum = 1 #durumu değiştir
```

Çizim ekranı çıktısı

Klavyeden space (uzun çubuk) tuşuna basarak renkler arası geçiş sağlanır.



```

#1'den 2'ye geçiş durumu:
elif trfkDurum == 1:
    forward(60)          #ileri 60 adım git
    fillcolor("red")      #ışığı kırmızı yap
    trfkDurum = 2        #durumu değiştir
# 2'den 0'a geçiş durumu:
else:
    back(120)            #geri 120 adım git
    fillcolor("green")   #ışığı yeşil yap
    trfkDurum = 0        #durumu değiştir

#Ana program
levhaCiz() #levhaCiz fonksiyonu çağrıldı
lambaCiz() #lambaCiz fonksiyonu çağrıldı
#Işık durum geçişleri fonksiyonu için space tuşuna bas
wn.onkey(durumMakinesi, "space")
wn.listen() #Olayları dinle
wn.mainloop() #Olay yakalama döngüsünü başlat

```



Açıklama: Program, space tuşuna basınca sırası ile "yeşil, sarı, kırmızı" renklerdeki lambaları yakacaktır (daireleri boyayacaktır).

Sonlu Durum Makinesi (Finite State Machine) Algoritması

Sonlu durum makinası (veya kısaca durum makinası), sınırlı sayıda durumdan, durumlar arası geçişlerden ve eylemlerin birleşmesiyle oluşan davranışların bir modellemesidir. Özellikle elektronik sistemlerin (trafik ışık sistemi, banka ATM cihazları, jetonlu oyun makineleri gibi) modellenmesinde durum makinesi algoritmasından yararlanılır.

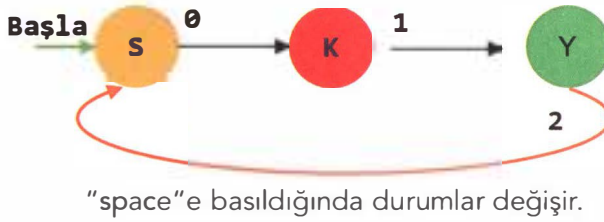
Durum makinesi (otomati) algoritmasını yukarıdaki trafik ışıkları örneğimiz üzerinden açıklayalım. Bir trafik ışığının 3 durumu vardır: **yeşil/green**, **sarı/yellow** ve **kırmızı/red**. Yukarıdaki programda bu renk (durum) geçişlerini sağlayan durum makinesi algoritmasını, **durumMakinesi()** isimli fonksiyon içerisinde kodladık. Her bir durum (renk) değişikliğini **trfkDurum** değişkeninde tuttuk.

Trafik ışık sisteminin sonlu durum makine algoritması aşağıdaki gibi çalışır;

Trafik durumunu gösteren **trfkDurum** değişkeni "0" değerinde ise renk "Sarı" rengi gösterir ve bir sonraki durumu "1" yapar. Klavyeden her space tuşuna basıldığında tablodaki durumlar arasında geçişler olur.

Mevcut Durum (trfkDurum)	İşlem (Renk değişikliği)	Koşul	Yeni durum
0	Sarı	"space" tuşuna basmak	1
1	Kırmızı	"space" tuşuna basmak	2
2	Yeşil	"space" tuşuna basmak	0

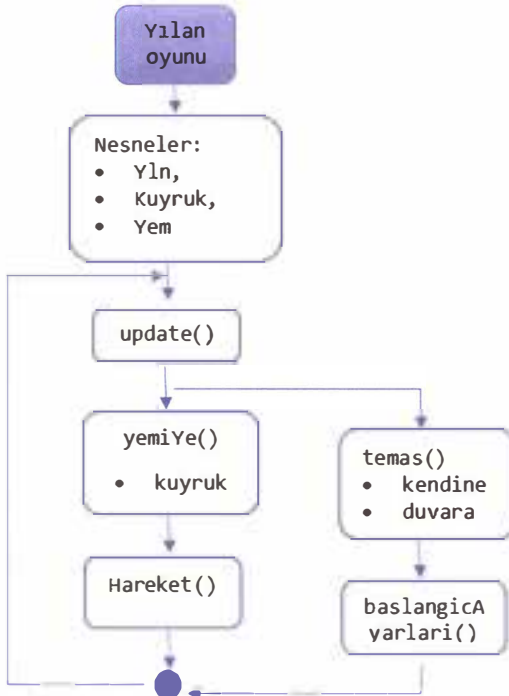
Trafik ışıklarının durum geçişlerini gösteren yukarıdaki tablo, aşağıdaki gibi durum diyagramları ile de modellenebilir.



Adım Adım Yılan Oyunu Tasarlayalım

Proje 14.4. Yılan Oyunu

Eski telefonların vazgeçilmez oyunu olan yılan oyununu Turtle ile adım adım kodlayalım.



Şekilde algoritması sunulan yılan oyunun önce çalışma mantığını anlayalım:

Oyun ekranında yılan ve yem olmak üzere iki nesne vardır. Yılan yemi yedikçe (üzerine geldikçe) yılanın kuyruk eklenir yani boyu uzar. Yılan kendi kendine veya çerçeve sınırlarına çarparsa (temas ederse) ölür ve başlangıç ayarlarına dönlür.

Oyuncunun hedefi, mümkün olduğu kadar yılanın boyunu uzatmak ve yüksek puan (skor) elde etmektir.

Şimdi oyunu nasıl kodlayacağımızı adım adım alt başlıklarla anlatalım. Programdaki her bir kod satırının açıklaması '#' yorum simgesi yanında belirtilmiştir.

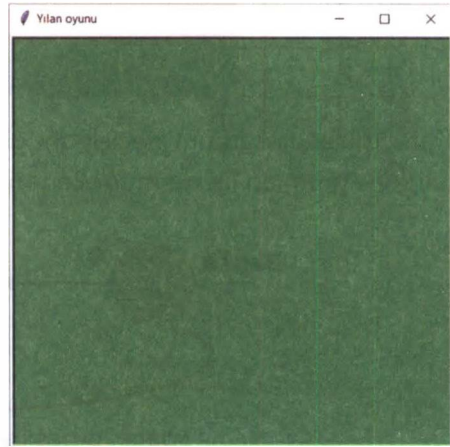
1. Oyun alan ekranı ve çerçevesi:

Öncelikle turtle modülü ve ihtiyacımız olacak diğer iki modülü programa ekleyelim;

```
import turtle
import time
import random
```

Sonra da oyun alanı çerçevesini temsil edecek 'wn' isimli nesneyi oluşturup, çerçeve ayarlarını aşağıdaki gibi yapalım;

```
#Oyun ekranı(çerçevesi) ayarları:
wn = turtle.Screen() #çerçeveyi temsil edecek nesne: wn
wn.title("Yılan oyunu") #çerçeve başlığı
wn.bgcolor("green") #çerçeve arka plan rengi: yeşil
wn.setup(600, 600) #çerçeve boyutu
wn.tracer(0) #ekran güncelleme ayarı kapalı
```



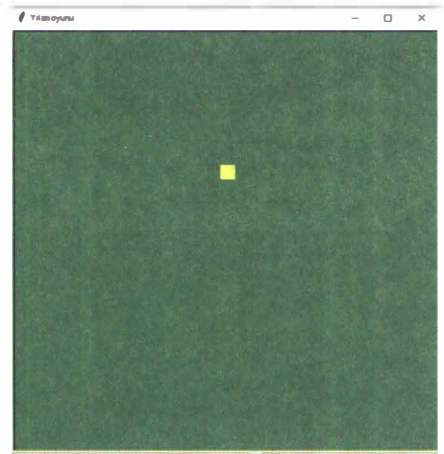
2. Yılan nesnesini oluşturalım:

Oyun çerçevesini yukarıdaki gibi ayarladıktan sonra Turtle() fonksiyonu ile yılanı temsil edecek yln isimli nesne tanımlanır ve bu nesnenin hızı, şekli, rengi gibi özellikleri aşağıdaki gibi ayarlanır;

```
#Yılan nesnesini oluşturup, başlangıç ayarlarını yapalım;
yln= turtle.Turtle() #yılan nesnesi (değişkeni):yln
yln.speed(0) #yılanın başlangıç hızı
yln.shape("square") #yılanın şekli: kare
yln.color("yellow") #yılanın kafa rengi: sarı
yln.penup() #Hareket ederken çizgi izini kaldır
yln.goto(0, 100) #yılanın başlangıç konumu
yln.yonu = "dur" #yılan hareketsiz
```

Yılan nesnesinin (veya son yaptığımız değişikliklerin) grafik ekranda görülebilmesi için ekranı/çerçeveyi her zaman güncellemek (update etmek) gerekir;

```
#Ana program sonsuz döngüsü
while True:
    wn.update() #çerçeveyi yenile/
    güncelle
```



3. Yılanı hareket ettirecek fonksiyonları yazalım:

Yılan nesnesini oluşturup, başlangıç ayarlamalarını yaptıktan sonra artık yılanı çerçeve içerisinde hareket ettirecek Hareket isimli fonksiyonu yazabiliriz;

```
def Hareket(): #“yln.yonu” değişkeninin durumuna göre hareket et
    if yln.yonu == “ust”:
        y = yln.ycor() #y ekseninde yukarı hareket et
        yln.sety(y + 20) #20 birim ilerle

    if yln.yonu == “alt”:
        y = yln.ycor() #y ekseninde aşağı hareket et
        yln.sety(y - 20) #20 birim ilerle

    if yln.yonu == “sag”:
        x = yln.xcor() #x ekseninde sağa doğru hareket et
        yln.setx(x + 20) #20 birim ilerle

    if yln.yonu == “sol”:
        x = yln.xcor() #x ekseninde sola doğru hareket et
        yln.setx(x - 20) #20 birim ilerle
```

Yılanımızı yukarı (“ust”), aşağı (“alt”), sola (“sol”) veya sağa (“sag”) hareket ettirecek fonksiyonu yazdık. Fakat bilgisayar fonksiyon içindeki ust, alt, sol ve sag’ı nasıl bilecek? Bu yönlerin her biri için de ayrı bir fonksiyon (gitYukari, gitAsagi, gitSaga, gitSola) tanımlamamız ve yln.yonu değişkenini ayarlamamız gerekiyor.

Ayrıca yılanın doğrudan tam tersi yöne (soldan sağa veya sağdan sola, yukarıdan aşağıya veya aşağıdan yukarıya) gitmemesi için bu fonksiyonlar içerisinde gerekli kontrolleri sağladık.

```
#Yılan yönü fonksiyonları ve işlevleri
def gitYukari():
    if yln.yonu != "alt":
        yln.yonu = "ust" #yılanyonu:ust

def gitAsagi():
    if yln.yonu != "ust":
        yln.yonu = "alt" #yılanyonu:alt

def gitSaga():
    if yln.yonu != "sol":
        yln.yonu = "sag" #yılanyonu:sag

def gitSola():
    if yln.yonu != "sag":
        yln.yonu = "sol" #yılanyonu:sol
```

Şimdi bu fonksiyonların her birine klavyeden bir tuş tanımlayacağız. Ve hangi tuşa basıldığını (tuş olaylarını) `wn.listen()` fonksiyonu ile dinleyeceğiz. Klavyeden ilgili yön tuşuna basıldığında hangi fonksiyonu çalıştırması gerektiği `onkeypress()` fonksiyonu içerisinde aşağıdaki gibi belirtilir. Yön tuşlarına atanan fonksiyonlar;

```
#Tuş olaylarını yakalamak için turtle çizim ekranını dinle ve ilgili
fonksiyona git;
wn.listen()

#Klavyeden yukarı (Up) yön oku ↑ tuşuna basıldığında 'gitYukari'
fonksiyonunu çalıştırır.
wn.onkeypress(gitYukari, "Up")

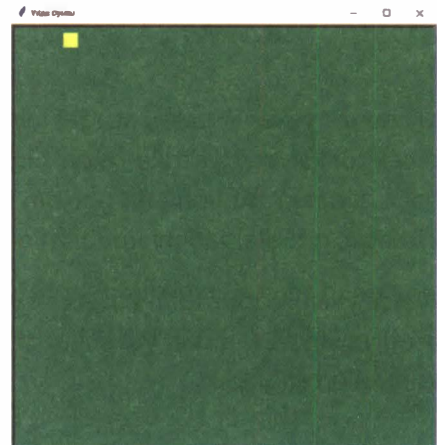
#Klavyeden aşağı (Down) yön oku ↓ tuşuna basıldığında 'gitAsagi' fonk-
siyonunu çalıştırır.
wn.onkeypress(gitAsagi, "Down")

#Klavyeden sağ (Right) yön oku → tuşuna basıldığında 'gitSaga' fonk-
siyonunu çalıştırır.
wn.onkeypress(gitSaga, "Right")

#Klavyeden sol (Left) yön oku ← tuşuna basıldığında 'gitSola' fonk-
siyonunu çalıştırır.
wn.onkeypress(gitSola, "Left")
```

Sonyaptığımız değişikliklerin grafik ekranda görülebilmesi için ekranı/çerçeveyi her zaman güncellemek (update etmek) gerekiyordu;

```
#Ana program sonsuz döngüsü:
while True:
    wn.update() #güncelle
    Hareket() #Hareket()
                fonksiyonunu çağır
    time.sleep(0.1) #0.1sn bekle
```

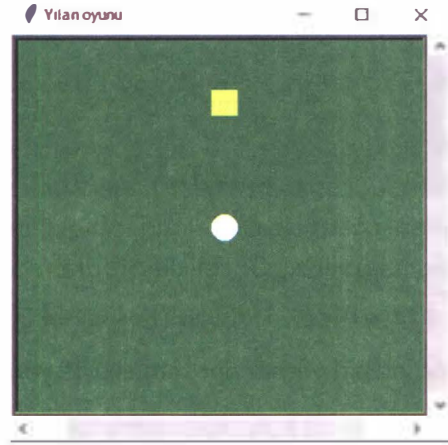


Not. Yılan nesnesinin hızını yavaşlatmak için `time.sleep(0.1)` komut satırı kullanıldı. Daha da yavaşlatmak için bu değer artırılabilir. Bu komut satırı için `time` modülünü program başına eklediğimizi hatırlatalım: `import time`.

4. Yem nesnesini oluşturalım:

Aynı yılan nesnesini oluşturduğumuz gibi yem nesnesini de oluşturalım ve özelliklerini ayarlayalım;

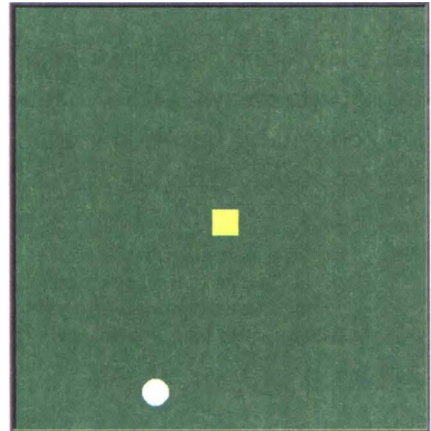
```
#Yem nesnesini oluşturup, başlangıç
ayarlarını yapalım;
yem = turtle.Turtle()
#Yem nesnesi adı: yem
yem.speed(0)
#Yemin başlangıç hızı
yem.shape("circle")
#Yemin şekli: daire
yem.color("white")
#Yemin rengi: beyaz
yem.penup()          #Kalemi kaldır
yem.goto(0,0)        #Yemi merkeze konumlandır
```



5. Yılan, yeme temas ettiğinde ne olacağı kodlayalım:

Yılan, yeme dokunduğunda onu yemesi ve çerçeve içerisinde rastgele bir yere konumlanması gerekiyor. Yılan ve yem nesneleri arasındaki mesafeyi `yn.distance(yem)` fonksiyonu ile hesaplayabiliriz. Mesafe 20 birimden az ise yılan ve yem birbirine çarpacaktır. Sonrasında ise yemin, ekranda farklı bir yere konumlanması sağlanacaktır.

Tüm bu işleri yapacak kod satırları aşağıdaki `yemiYe()` fonksiyonu içerisine yazılır. Yazılan `yemiYe()` fonksiyonunu ana program döngüsü (`while True:`) içerisinde çağırmamız yeterli olacaktır;



```
def yemiYe(): #yemiYe fonksiyonu
    if yln.distance(yem)<20: #yılan ile yem arasında 20 birimden az
        mesafe varsa çarpışma olur ve
        sonrasında yem, çerçeve içinde
        rastgele bir yere konumlanır
        x = random.randint(-280, 280) #x eksen konumu sınırları
        y = random.randint(-280, 280) #y eksen konumu sınırları
        yem.goto(x, y) #yemi rastgele bir noktaya konumlandır
```

5.1. Yılan yemi yedikçe büyümeli:

Ayrıca yılanın yeme her dokunması ile (yani yemi yedikçe) büyümesi gerekiyor. Bunun için yılanın yemle her temasında yılan yeni bir kuyruk eklemeliyiz (yani yılanı uzatmalıyız). Bunun için eklenecek her bir yeni kuyruğu bir liste değişkeninde tutabiliriz. Öncelikle program başında aşağıdaki gibi boş bir liste tanımlarız;

```
Liste = [] #kuyruk nesnelerini tutan liste
```

Ve yılanın yemle her temasında yeni kuyruğu bu listeye ekleriz.

Şimdi yemiYe() fonksiyonu içerisinde yeni bir kuyruk nesnesi tanımlayacak kodu aşağıdaki gibi if sorgusu altına yazalım:

```
def yemiYe(): #yemiYe fonksiyonu
    if yln.distance(yem)<20: #yılan yeme temas ettiğinde
        x = random.randint(-280, 280) #x eksen konumu sınırları
        y = random.randint(-280, 280) #y eksen konumu sınırları
        yem.goto(x, y) #yemi rasgele bir noktaya konumlandır

        #Yılan yeni bir kuyruk ekleyelim
        kuyruk = turtle.Turtle() #kuyruk nesnesi: kuyruk
        kuyruk.speed(0) #kuyruğun başlangıç hızı
        kuyruk.shape("square") #kuyruğun şekli: kare
        kuyruk.color("red") #kuyruğun rengi: kırmızı
        kuyruk.penup() #kalemi kaldır
        Liste.append(kuyruk) #kuyruğu listeye ekle
```

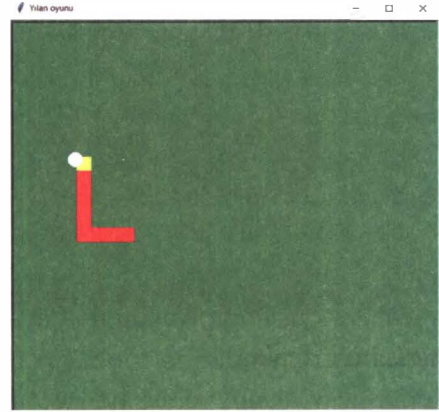
Yılan kuyruğunu ekledik. Tamam ama yılanın başı hareket ettiğinde eklenen kuyruğunun da aynı şekilde hareket etmesi lazım. Bunun için son eklenen kuyruğun konumu bir önceki kuyruğun konumu (x'den x-1'e, x-1'den x-2'ye şeklinde) olacak şekilde yer değiştirdik.

```
uz = len(Liste) #listenin uzunluğunu (kuyruk sayısını) al
for indis in range(uz-1, 0, -1): #ters sırada
    x = Liste[indis-1].xcor() # (x-1). koordinat x. koordinat yapıldı
    y = Liste[indis-1].ycor() # (y-1). koordinat y. koordinat yapıldı
    Liste[indis].goto(x, y) # Listedeki kuyruklar yeni konumlarına
    yerleştirildi
```

Ancak bu şekilde listeyi ters sırada hareket ettirdiğimizde yılanın arkasından gelen ilk kuyruk yılan başının yerine geçecektir.

Dolayısıyla listenin 0.indisli elemanı yılanın kendisi (yani yılan başı) olacaktır.

Son olarak `yemiYe()` fonksiyonu içerisine aşağıdaki kod satırları da eklenir:



```
#Listenin ilk elemanını yılan başı yapalım
if len(Liste) > 0: #Eğer listede kuyruk varsa
    x = yln.xcor()
    y = yln.ycor()
    Liste[0].goto(x,y) #0.indisli eleman yılanın yeni başı olur.
```

5.2. Skor(Puan) Ekleme:

Çerçeve başlığına (title) skor ve en yüksek skor gibi değerler yazdırabiliriz. Yılan başarılı bir şekilde yemi her yediğinde oyuncu skorunu 10 puan artıran ve en son aldığı en yüksek skoru gösteren aşağıdaki kod, `yemiYe()` fonksiyonu içerisine "`if yln.distance(yem)< 20:`" sorgusu altına eklenmelidir;

```
def yemiYe(): #yemiYe fonksiyonu
    global skor, maxSkor #yerel değişkenleri global yapalım
    if yln.distance(yem)<20: #yılan yeme temas ettiğinde

        skor = skor + 10 # Her başarılı yemede skoru 10 puan artır
        #Eğer skor, maxSkor dan büyükse skor değerini maxSkor'a ata.
        if skor > maxSkor:
            maxSkor = skor
        #skor değerlerini anlık çerçeve başlığında göster
        wn.title("Skor: {} En yüksek skorun: {}".format(skor, maxSkor))
```

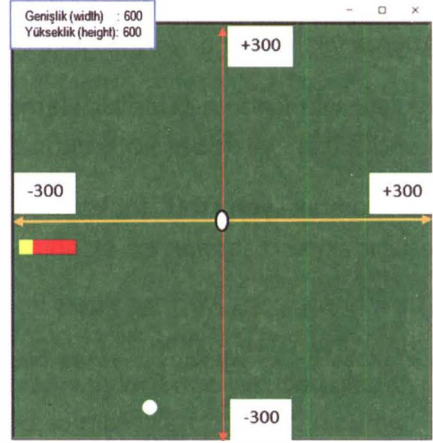
Not. Fonksiyon içerisinde kullanılan tüm değişkenlere program başında ilk değer atamaları aşağıdaki gibi yapılmalıdır;

```
#değişkenler ve başlangıç değerleri
skor = 0 #Skor değişkeni
maxSkor = 0 #maksimum skor değişkeni
Liste = [] #kuyruk dizisi
```

6. Yılan çerçeve kenarlarına çarpınca ne olacak?

Yılanın ekran/çerçeve sınırlarına çarptığında ölmesi gerekiyor. Çerçeve koordinatlarını bildiğimize göre yapmamız gereken yılanın pozisyonun bu koordinatlara denk gelip/gelmediğini sorgulamak olmalıdır. Eğer yılan pozisyonu çerçeve sınırında ise yılan duracak ve başlangıç konumuna ayarlanacaktır (sıfırlanacaktır).

Şekilde görüldüğü gibi (600, 600) bir çerçevenin sınırları -300 ile +300 değerleri arasındadır. Yılan koordinatları (xcor, ycor) bu değerlere yaklaştığında yani +290 ya da -290 değerlerini aştığında program ayarlarının sıfırlanması gerekiyor:



```
#Yılan çerçeve sınırlarına çarpınca ayarlar sıfırlayacak sorguyu
'whileTrue:' ana program döngüsü içerisine yazalım:
#Ana program sonsuz döngüsü:
while True:
    if yln.xcor()>290 or yln.xcor()<-290 or yln.ycor()>290 or yln.
    ycor()<-290:
        baslangicAyarlari()
```

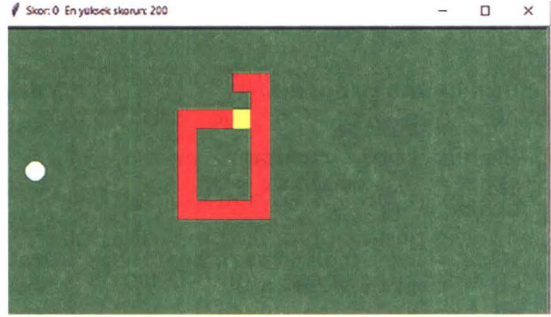
Şimdi başlangıç ayarlarını baslangicAyarlari() isimli fonksiyon içinde belirleyelim:

```
def baslangicAyarlari():
    time.sleep(1) #zamanı 1sn durdur
    yln.goto(0,0) #yılanı başlangıç (0,0) koordinatlarına al
    yln.yonu = "dur" #yılan yönünü sıfırla
    global skor #skor değişkenini global yap
    #Yılanın tüm eklemlerini (parçalarını) çerçeve dışına taşı (at)
    for eklem in Liste:
        eklem.goto(1000, 1000)
    #ve listeyi temizle
    Liste.clear()
    skor = 0 #skoru sıfırla
    #Başlıktaki Skor değerlerini güncelle
    wn.title("Skor: {} En yüksek skorun: {}".format(skor, maxSkor))
```

7. Yılan kendi kuyruğuna (vücuduna) çarpınca ne olacak?

Yılanın kendi kuyruğuna/vücuduna dokunması ya da çarpması durumunda yılanın ölmesi ve başlangıç ayarlarına dönmesi gerekiyor. Eğer yılanın başı ile eklemleri/vücudu arasındaki mesafe 20 birimden az ise kendisine çarpma durumu gerçekleşir.

Buna göre aşağıdaki kodu ana program sonsuz döngüsü içerisine yazmalıyız;

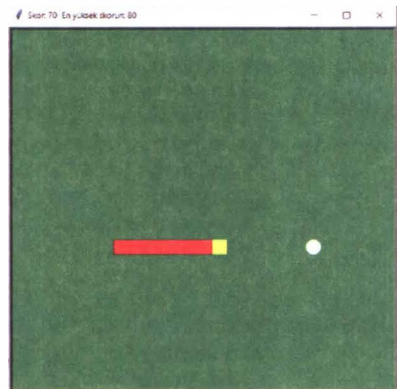


```
#veya yılan kendi vücuduna çarparsa
for eklem in Liste:
    if eklem.distance(yn) < 20: #kuyruğuna 20 birimden az yaklaşırsa
        baslangicAyarlari() #başlangıç ayarlarına dön
```

Buna göre ana program sonsuz döngüsünün son hali aşağıdaki gibi olacaktır:

```
#Yılan oyunu ana programı sonsuz döngüsü:
while True:
    wn.update() #çerçeveyi güncelle
    yemiYe() #yemiYe() fonksiyonunu çalıştır
    Hareket() #Hareket() fonksiyonunu çalıştır
    #Eğer yılan çerçeve köşelerine çarparsa
    if yn.xcor()>290 or yn.xcor()<-290 or yn.ycor()>290 or yn.ycor()<-290:
        baslangicAyarlari() #başlangıç ayarlarına dön
    #veya yılan kendi vücuduna çarparsa
    for eklem in Liste:
        if eklem.distance(yn) < 20:
            baslangicAyarlari() #tekrar başlangıç ayarlarına dön
            time.sleep(0.1) #Yılanın hızını ayarla
    wn.mainloop() #olay yakalama döngüsü aktif
```

Buna göre programın çalışan son halinin ekran görüntüsü yandaki gibi olacaktır;



Not. Ayrıca yılan oyunu dahil kitaptaki tüm kodlara <https://github.com/bcobanoglu/PythonforKids> adresinden veya www.pusula.com sitesindeki dosya merkezinden erişebilirsiniz.

Yılan Oyunu Programı (Tek parça)

Yılan oyunu programı-1

```
import turtle
import time, random
skor = 0 ; maxSkor = 0
Liste = []

#Oyun ekranı(çerçevesi) ayarları:
wn = turtle.Screen()
wn.title("Yılan oyunu")
wn.bgcolor("green")
wn.setup(600, 600)
wn.tracer(0)

#Yılan başı nesnesi ayarları:
yln= turtle.Turtle()
yln.speed(0)
yln.shape("square")
yln.color("yellow")
yln.penup()
yln.goto(0, 100)
yln.yonu = "dur"

#Yem nesnesi ayarları:
yem = turtle.Turtle()
yem.speed(0)
yem.shape("circle")
yem.color("white")
yem.penup()
yem.goto(0,0)

#Hareket fonksiyonu:
def Hareket():
    if yln.yonu == "ust":
        y = yln.ycor()
        yln.sety(y + 20)
    if yln.yonu == "alt":
        y = yln.ycor()
        yln.sety(y - 20)
    if yln.yonu == "sag":
        x = yln.xcor()
        yln.setx(x + 20)
    if yln.yonu == "sol":
        x = yln.xcor()
        yln.setx(x - 20)

#Yön tuşları fonksiyonları:
def gitYukari():
    if yln.yonu != "alt":
        yln.yonu = "ust"
def gitAsagi():
    if yln.yonu != "ust":
        yln.yonu = "alt"
def gitSaga():
    if yln.yonu != "sol":
        yln.yonu = "sag"
```

Yılan oyunu programı-2 (devamı)

```
def gitSola():
    if yln.yonu != "sag":
        yln.yonu = "sol"

#Tuş olaylarını yakala;
wn.listen()
wn.onkeypress(gitYukari, "Up")
wn.onkeypress(gitAsagi, "Down")
wn.onkeypress(gitSaga, "Right")
wn.onkeypress(gitSola, "Left")

#yemiYe() fonksiyonu:
def yemiYe():
    global skor, maxSkor
    if yln.distance(yem)<20:
        x = random.randint(-280, 280)
        y = random.randint(-280, 280)
        yem.goto(x, y)

#kuyruk nesnesi ayarları:
kuyruk = turtle.Turtle()
kuyruk.speed(0)
kuyruk.shape("square")
kuyruk.color("red")
kuyruk.penup()
Liste.append(kuyruk)

skor = skor + 10
if skor > maxSkor:
    maxSkor = skor

    wn.title("Skor: {} En
yüksek skorun: {}".format(skor,
maxSkor))

uz = len(Liste)
for indis in range(uz-1, 0, -1):
    x = Liste[indis-1].xcor()
    y = Liste[indis-1].ycor()
    Liste[indis].goto(x, y)

if len(Liste) > 0:
    x = yln.xcor()
    y = yln.ycor()
    Liste[0].goto(x,y)
```

Yılan oyunu programı-3 (devamı)

```
#baslangicAyarlari() fonksiyonu:
def baslangicAyarlari():
    time.sleep(1)
    yln.goto(0,0)
    yln.yonu = "dur"
    global skor
    for eklem in Liste:
        eklem.goto(1000, 1000)
    Liste.clear()
    skor = 0
    wn.title("Skor: {} En yüksek skorun: {}".format(skor, maxSkor))

#Ana program sonsuz döngüsü:
while True:
    wn.update()
    yemiYe()
    Hareket()
    if yln.xcor()>290 or yln.xcor()<-290 or yln.ycor()>290 or yln.ycor()<-290:
        baslangicAyarlari()
    for eklem in Liste:
        if eklem.distance(yln) < 20:
            baslangicAyarlari()
    time.sleep(0.1)

#olay yakalama döngüsü aktif
wn.mainloop()
```

Projelere Ses Ekleme

Ses veya müzik dosyaları genellikle wav veya mp3 uzantılı dosyalardır. Oyun projelerinize ses veya müzik eklemek (bilgisayarınızdaki veya bir web sitesindeki ses dosyasındaki sesi çalmak) için "playsound" modülüne ait playsound() fonksiyonu kullanılabilir. Öncesinde playsound modülü komut isteminden pip install playsound komut satırı ile bilgisayara kurulur (kurulum detayı için sayfa 142'ye bakılabilir). Kurulumdan sonra programa aşağıdaki gibi import komutu ile eklenebilir ve playsound() fonksiyonu ile ses dosyası çalıştırılabilir;

```
from playsound import playsound
playsound('sesdosyasi.mp3')
```

Parantezler arasına alınacak ses dosyası yolu ile birlikte yazılır. Örneğin nasa web sitesindeki bir ses dosyası aşağıdaki gibi programda çalıştırılabilir;

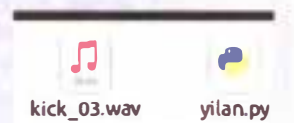
```
playsound('https://www.nasa.gov/mp3/578626main_sputnik-beep.mp3')
```

Şimdi yılan oyununda yılan köşelere veya kendi kuyruğuna çarpınca ses çıkarması için ana programın sonsuz döngüsünde playsound ile başlayan aşağıdaki satırları eklemek yeterlidir;

```
#1.playsound modülü program başında import edilir;
from playsound import playsound
#...
while True:
    #.....
    #2. Yılan köşelere çarpınca ses çıkar!
    if yln.xcor()>290 or yln.xcor()<-290 or yln.ycor()>290 or yln.
ycor()<-290:
    #çalınacak ses dosyası aşağıdaki gibi bir web adresi olabilir;
    playsound('https://www.nasa.gov/mp3/578626main_sputnik-beep.mp3')
    baslangicAyarlari()

    #3. veya yılan kendi vücuduna çarparsa ses çıkar!
    for eklem in Liste:
        if eklem.distance(yln) < 20:
            # ya da çalınacak ses dosyadaki bilgisayarımızdaki
            herhangi bir ses dosyası olabilir
            playsound('kick_03.wav')
            baslangicAyarlari()
```

Not. Bilgisayarınızdaki ses dosyası ("kick_03.wav" gibi) ile Python proje dosyasının ("yılan.py" gibi) aynı klasör içerisinde yer alması gerektiğini unutmayın.



Not. Proje14.4'ün ses dosyası eklenmiş hali olan "yılan.py" ve "kick_03.wav" ses dosyalarını bilgisayarınıza <https://github.com/bcobanoglu/Pythonfor-Kids/tree/master/Bolum14> adresinden indirebilirsiniz.

Soru ve Cevaplarla Neler Anladık?

- 1 Kullanıcının fareye tıklaması veya klavyeden bir tuşa basması gibi eylemler ne olarak adlandırılır?

Olay (Event)

- 2 Fareye (mouse) tıklandığında gerçekleşen olayın adı nedir?

onclick()

- 3 Klavye tuşuna basıldığında gerçekleşen olayın adı nedir?

onkeypress()

- 4 Tuş olaylarını yakalamak için turtle ekranını dinleyen fonksiyonun adı nedir?

listen()

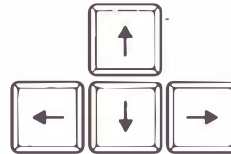
- 5 Çerçeve başlığına mesaj yazmak için ne yapmak gerekir?

title("mesaj!")

- 6 Bir turtle programının en son satırında yer alan, olay yakalama döngüsünü başlatan, grafiğin ekrandan kaybolup gitmemesini sağlayan fonksiyonun ismi nedir?

mainloop()

- 7 Yandaki yön tuşlarının isimleri nedir?



Left, Up, Down, Right

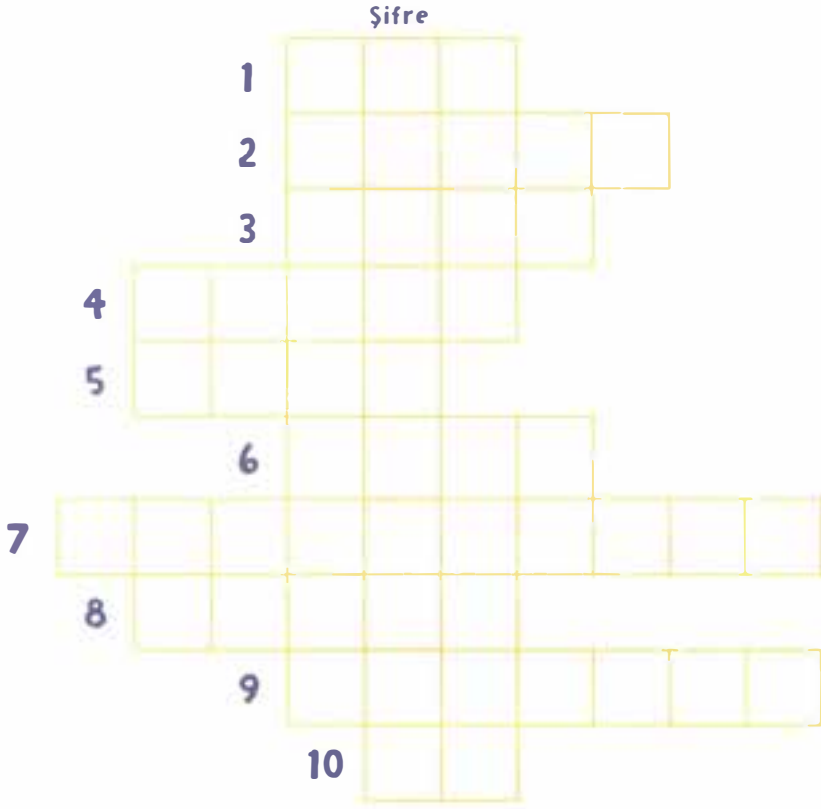
- 8 Yukarıdaki yılan oyununda yılanı biraz daha yavaşlatmak için ne yapılmalıdır?

time.sleep(0.1) satırındaki 0.1 değeri artırılmalıdır.

- 9 Yılan oyununda yılanın tam tersi yöne dönmesini (yukarıdan aşağıya veya aşağıdan yukarıya gibi) engelleyen kod satırları nelerdir?

```
def gitYukari():  
    if yln.yonu != "alt":  
        yln.yonu = "ust"  
def gitAsagi():  
    if yln.yonu != "ust":  
        yln.yonu = "alt"
```

BULMACA



1. Grafik penceresini kapatır.
2. Çerçeve başlığı.
3. Kullanıcının fareye tıklaması, klavyeden bir tuşa basması gibi eylemler, event.
4. Bekleme komutu.
5. Aşağı yön oku.
6. Git.
7. Tuşa basma olayı.
8. Çizim kalemini kaldırır.
9. Fare tıklama olayı.
10. Yukarı yön oku.

Aşağıya bulduğunuz şifreyi kodlayınız.

--	--	--	--	--	--	--	--	--	--

SONSÖZ

Evet, bu kitabı bitirerek Python ve programlama dünyasına hoş geldin. Artık ileri için bildiklerinin, öğrendiklerinin üstüne bir şeyler eklemen gerekiyor. Bu yolculuğu burada bırakmamalısın.

Kendine uzmanlaşmak, daha ileri gitmek için bir alan seçmeli ve o konu üzerinde projeler geliştirmeye devam etmelisin. Bu kitabı tamamladıktan sonra atacağın adımlar için sana birkaç önerim olacak;

- Oyun programlama alanında ilerlemek istiyorsan PyGame ve OpenCV öğrenebilirsin.
- Görsel programlama (GUI) alanında ilerlemek istiyorsan Tkinter veya PyQt öğrenebilirsin.
- Web programlama alanında ilerlemek istiyorsan Django veya Flask öğrenebilirsin.
- Robotik kodlama/programlama alanında ilerlemek istiyorsan Raspberry PI veya Arduino öğrenebilirsin.
- Veri bilimi ve Yapay Zekâ alanında ilerlemek istiyorsan NumPy, Pandas, Matplotlib, SciPy gibi paketleri öğrenebilirsin.
- Veritabanı programlama alanında ilerlemek istiyorsan SQLite, MySQL öğrenebilirsin.

Özetle, yoldasın ve yolun devam ediyor. Yolun açık olsun arkadaşım.

Ben mi? Bir şeyler danışmak, projelerini ve heyecanını paylaşmak istersen her daim buradayım.

Dr. Bülent Çobanoğlu
cobanoglubulent@gmail.com

KAYNAKÇA

1. Bülent Çobanoğlu, "Algoritma Geliştirme ve Veri Yapıları", Pusula Yayıncılık, İstanbul, 5. Baskı, Temmuz 2014.
2. Bülent Çobanoğlu, "C/C++ ve Java ile Algoritma ve Programlama", 3. Baskı, Abaküs Yayınları, ISBN: 9786059129381, 2017.
3. Bülent Çobanoğlu, "Herkes için Python", 2. Baskı, Pusula Yayıncılık, ISBN: 9786052359587, 2018.
4. Ivelin Demirov, Beth McMillan, Learn Python Visually, 2015
5. Rob, Miles, Begin to Code with Python, Microsoft, 2017.
6. <https://docs.python.org/3/library/turtle.html>
7. <https://www.edureka.co/blog/python-turtle-module/>
8. <https://www.asciart.eu/>
9. <https://realpython.com>
10. Bülent Çobanoğlu, "Algoritma Temelli Bilgisayarsız Kodlama Etkinlikleri", Pusula Yayıncılık, 2019.

PYTHON ÖĞRENMEK BU KİTAPLA ÇOK KOLAY!

- Algoritma ve Akış Şemaları
- Bilgisayar Kavramı ve Python Dili
 - Değişkenler ve Hesaplar
 - Veri Giriş-Çıkış İşlemleri
- Koşul İfadeleri (if/else) ve Karşılaştırma Operatörleri
 - Döngüler (Loops)
 - Fonksiyonlar
- String (Karakter Dizisi) İşlemleri
 - Diziler ve Listeler
 - Sözlükler
- Nesne ve Sınıf Kavramları
- Rastgele Sayı Üretimi ve Matematiksel Oyunlar
 - Turtle ile Çizim Uygulamaları
- Olay Tabanlı Projeler ve Yılan Oyunu

Bu kitapta, her bir bölümde önce programcılık kavramları anlatılmış, sonrasında kolaydan zora doğru yapılandırılmış projeler, basit anlatım tekniğiyle açıklanmıştır. Verilen örneklerde; çocukların matematik bilgisi, hayat tecrübesi ve somut kavramları anlama becerilerinin daha kuvvetli olduğu dikkate alınmış; ağırlıklı olarak oyun tabanlı ve eğlenceli olanlar seçilmiştir. Bölüm içlerinde yer alan "sıra sende" etkinlikleri ile çocuğun bölüm içinde öğrendiği kavramları tekrar etmesi sağlanmıştır. Bölüm sonlarında yer alan soru-cevap ve bulmacalar ile de pekiştirme hedeflenmiştir.

Kitapta yer alan örnek uygulamaları
www.pusulakitaplik.com
Dosya Merkezi'nden indirebilirsiniz.



BÜLENT ÇOBANOĞLU



@pusula_kitap



@pusula_kitap



@yayincilik.pusula



www.pusula.com
www.pusulakitaplik.com

PUSULA

ISBN: 978-605-2359-94-5



9 786052 359945